Lecture Notes of

# OPERATING SYSTEMS

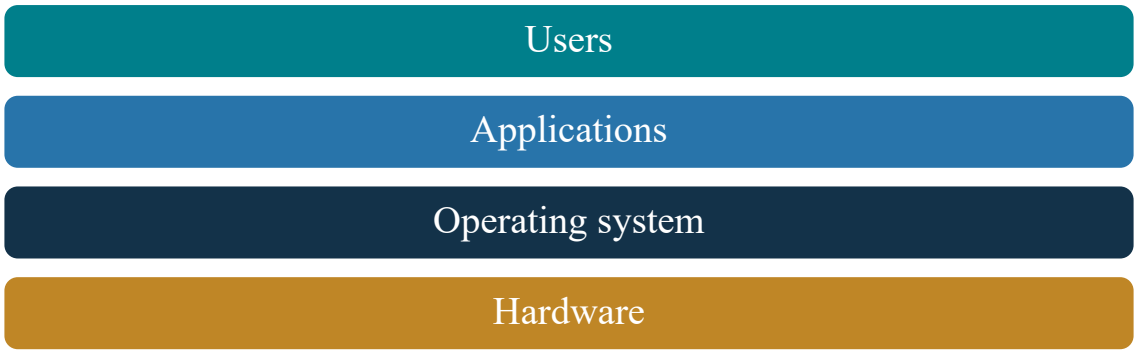
For second level

IT department

## Illustrated Student Edition

VISUAL EXPLANATION

### The big picture



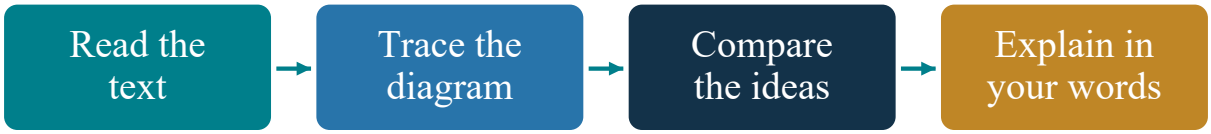

Follow the connections between people, programs, and the machine.

# How to study with this edition

Read in short steps. Use the diagrams to trace a process, compare related ideas, or check a calculation.

VISUAL EXPLANATION

**Read, trace, compare, explain**




Pause after each section and connect its explanation to its visual.

## Highlighted definitions

Pale blue blocks draw attention to definitions in the text.

## Important terms

Bold words help you find the concepts you need to remember.

## Worked examples

Tables, formulas, and result diagrams stay in their original sequence.

## Finding a topic

Use the linked contents, PDF bookmarks, and running topic headings.

# Contents

**INTRODUCTION 6**

1.1. Operating system definition . . . 6
1.2. History of Operating Systems . . . 9
1.3. Operating System Functions . . . 11
1.3.1. Memory Management . . . 13
1.3.2. Processor Management . . . 14
1.3.3. Device Management . . . 14
1.3.4. File Management . . . 15
1.3.5. Security . . . 15
1.3.6. Error detection . . . 16
1.3.7. Control over system performance . . . 16
1.3.8. Job accounting . . . 16
1.3.9. Coordination between other software and users . . . 16
1.4. Operating-System Services . . . 16
1.4.1. User interface. . . . 17
1.4.2. Program execution . . . 18
1.4.3. I/O Operation . . . 19
1.4.4. File system manipulation . . . 20
1.4.5. Communication . . . 21
1.4.6. Resource Management . . . 21
1.4.7. Protection . . . 22
1.4.8. Error detection . . . 23
1.4.9. Accounting: . . . 23

**OPERATING SYSTEMS STRUCTURE AND TYPES 25**

2.1. Operating-System Structure . . . 25
2.1.1 Simple Structure . . . 26
2.1.2. Layered Approach . . . 27
2.1.3. Microkernels . . . 30
2.1.4. Modules . . . 32
2.1.5. Hybrid Systems . . . 34

2.2. Types of Operating Systems . . . . . 35
2.2.1. Batch operating system . . . . . 35
2.2.2. Time-sharing operating systems . . . . . 36
2.2.3. Distributed operating System . . . . . 38
2.2.4. Network operating System . . . . . 39
2.2.5. Real Time operating System . . . . . 40

**PROCESS MANAGEMENT 43**

3.1. Process State . . . . . 44
3.2. Process Control Block . . . . . 45
3.3. Process Scheduling . . . . . 47
3.4. Scheduling Queues . . . . . 48
3.5. Schedulers . . . . . 50
3.5.1. Long Term Scheduler . . . . . 50
3.5.2. Short Term Scheduler . . . . . 51
3.5.3. Medium Term Scheduler . . . . . 51
3.6. Context Switch . . . . . 53
3.7. Process Creation . . . . . 56
3.8. Process Termination . . . . . 58
3.9. Interprocess Communication . . . . . 60

**CPU SCHEDULNG 64**

4.1. Preemptive Scheduling . . . . . 65
4.2. Scheduling Criteria . . . . . 66
4.3. Scheduling Algorithms . . . . . 69
4.3.1. First-Come, First-Served Scheduling . . . . . 69
4.3.2. Shortest-Job-First Scheduling . . . . . 72
4.3.3. Priority Scheduling . . . . . 75
4.3.4. Round-Robin Scheduling . . . . . 78
4.3.5. Multilevel Queue Scheduling . . . . . 82

**MEMORY MANEGEMENT 84**

5.1. Address Binding . . . . . 86
5.2. Logical Versus Physical Address Space . . . . . 88
5.3. Dynamic Loading . . . . . 90

5.4. Swapping . . . . . 90
5.4. Memory Allocation . . . . . 92
5.5. Fragmentation . . . . . 95
5.6. Segmentation . . . . . 96
5.7. Paging . . . . . 98

**STORAGE MANEGEMENT 106**

6.1. Magnetic Disks . . . . . 107
6.1.1. Disk Structure . . . . . 110
6.2. Disk Scheduling . . . . . 112
6.2.1. FCFS Scheduling . . . . . 113
6.2.2. SSTF Scheduling . . . . . 114
6.2.3. SCAN Scheduling . . . . . 116
6.2.4. C-SCAN Scheduling . . . . . 117
6.2.5. LOOK Scheduling . . . . . 118
6.2.6. Selection of a Disk-Scheduling Algorithm . . . . . 119
6.3. File -System . . . . . 121
6.3.1. File Concept . . . . . 121
6.3.2. File Attributes . . . . . 122
6.3.3. File Operations . . . . . 123
6.3.4. File Type . . . . . 126

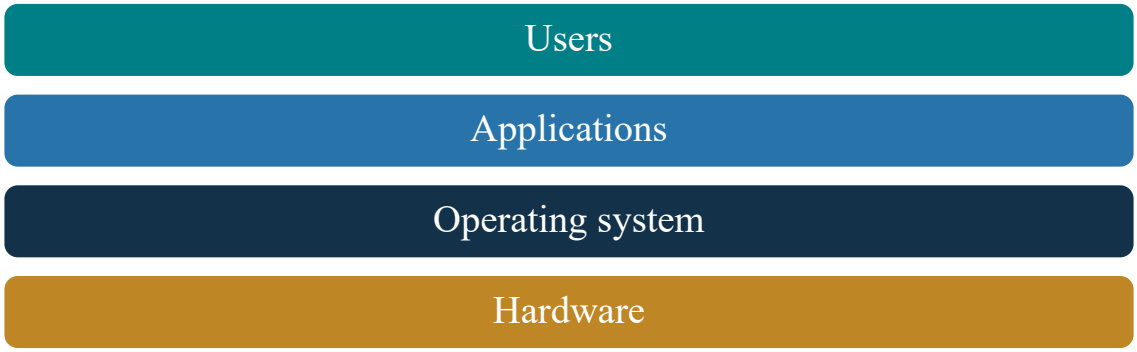
**References: 128**

CHAPTER 1

# INTRODUCTION

VISUAL EXPLANATION

## Chapter at a glance




Use this route to see how the chapter topics connect.

## 1.1. Operating system definition

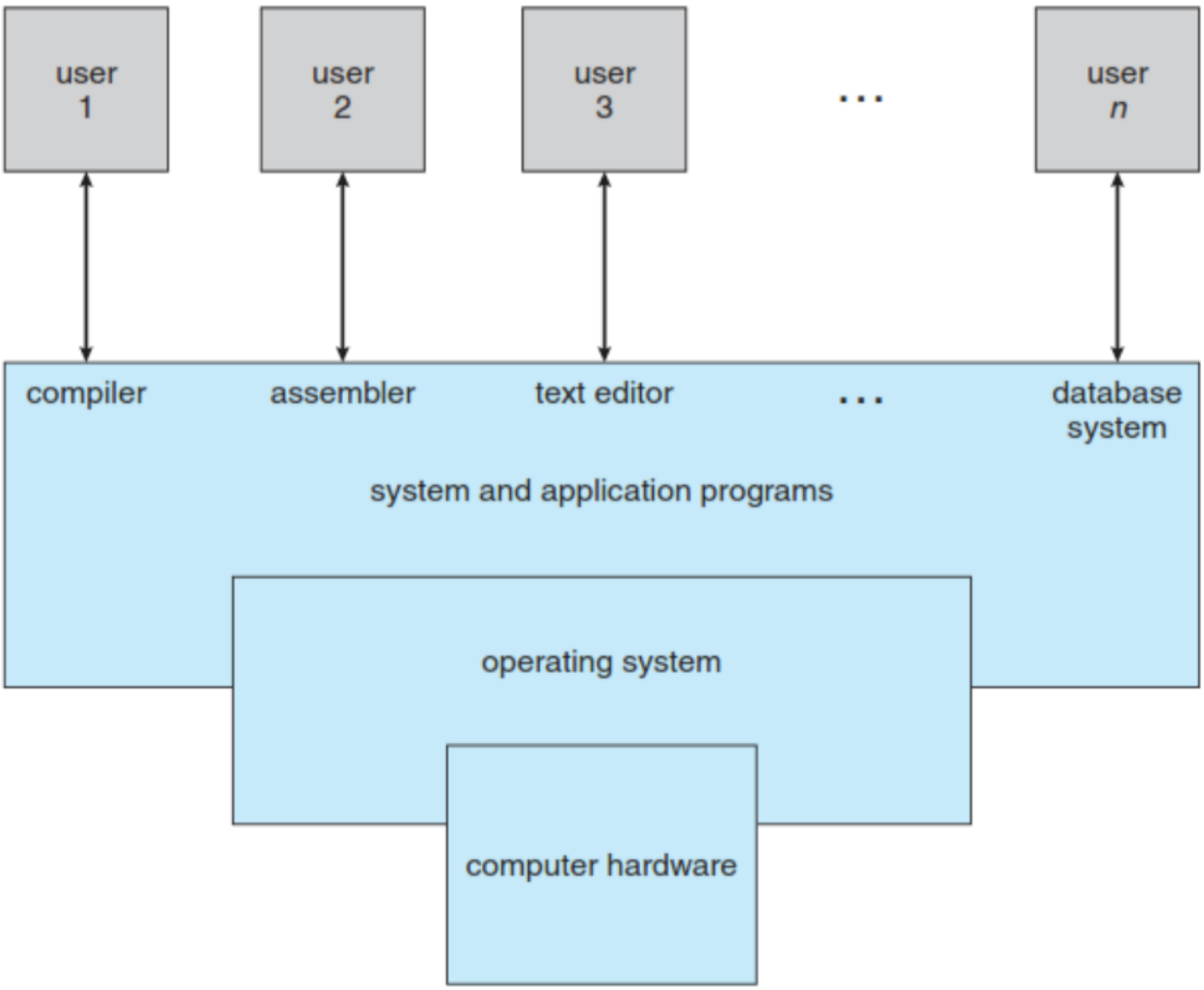
Modern general-purpose computers, including personal computers and mainframes, have an **operating system** to run other programs, such as application software. Examples of operating systems for personal computers include Microsoft Windows, Mac OS (and Darwin), Unix, and Linux.

> An ***Operating System*** (OS) is an intermediary between users and computer hardware. It provides users an environment in which a user can execute programs conveniently and efficiently. In technical terms, it is software which manages hardware.

> An **operating System** controls the allocation of resources and services such as memory, processors, devices and information.

> An ***operating system*** is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs.

The ***operating system*** could be defined also as a program that manages the hardware and software resources of a computer and acts as an intermediary between the user and the computer hardware.




Abstract view of the components of a computer system.

The kernel is a computer program that constitutes the central core of a computer's **operating system**. It has complete control over everything that occurs in the system.

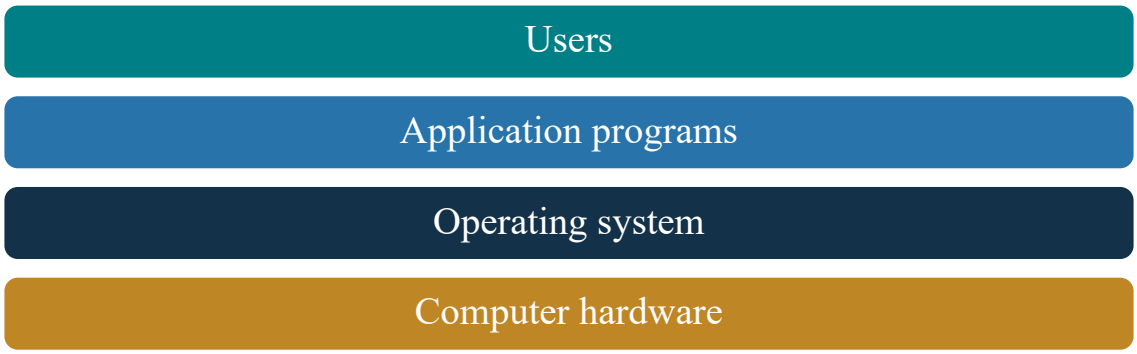
As such, it is the first program loaded on startup, and then manages the remainder of the startup, as well as input/output requests from software, translating them into data processing instructions for the central processing unit.

It is also responsible for managing memory, and for managing and communicating with computing peripherals, like printers, speakers, etc. The kernel is a fundamental part of a modern computer's **operating system**.

A kernel connects the application software to the hardware of a computer. The critical code of the kernel is usually loaded into a protected area of memory, which prevents it from being overwritten by other, less frequently used parts of the **operating system** or by applications.

VISUAL EXPLANATION

## The operating system as an interface




The operating system sits between applications and hardware and manages access to resources.

# 1.2. History of Operating Systems

The first computers did not have operating systems. By the early 1960s, commercial computer vendors were supplying quite extensive tools for streamlining the development, scheduling, and execution of jobs on batch processing systems. Examples were produced by UNIVAC and Control Data Corporation, amongst others.

Through the 1960s, several major concepts were developed, driving the development of operating systems. The development of the IBM System/360 produced a family of mainframe computers available in widely differing capacities and price points,

for which a single **operating system** OS/360 was planned. Applications written for the OS/360 can still be run on modern machines. Another key development was the concept of time-sharing: the idea of sharing the resources of expensive computers amongst multiple computer users interacting in real time with the system.

In the 1970s, commercially-popular minicomputer **operating system** was VMS. The first microcomputers did not have the capacity or need for the elaborate operating systems that had been developed for mainframes and minis; minimalistic operating systems were developed.

One notable early **operating system** was CP/M, which was supported on many early microcomputers and was largely cloned in creating MS-DOS, which became wildly popular as the **operating system** chosen for the IBM PC (IBM's version of it was called IBM-DOS or PC-DOS),

its successors making Microsoft one of the world's most profitable companies. The major alternative throughout the 1980s in the microcomputer market was Mac OS, tied intimately to the Apple Macintosh computer.

By the 1990s, the microcomputer had evolved to the point where, as well as extensive GUI facilities, the robustness and flexibility of operating systems of larger computers became increasingly desirable.

Microsoft's response to this change was the development of Windows NT, which served as the basis for Microsoft's entire **operating system** line starting in 1999. Apple rebuilt their **operating system** on top of a Unix core as Mac OS X, released in 2001.

Hobbyist-developed re-implementations of Unix, assembled with the tools from the GNU project, also became popular; versions based on the Linux kernel are by far the most popular, with the BSD derived UNIXes holding a small portion of the server market.

Today, Command line interface (or CLI) operating systems can operate using only the keyboard for input. Modern OS's use a mouse for input with a graphical user interface (GUI) sometimes implemented as a shell.

The appropriate OS may depend on the hardware architecture, specifically the CPU, with only Linux and BSD running on almost any CPU. Windows NT has been ported to other CPUs, most notably the Alpha, but not many.

Since the early 1990s the choice for personal computers has been largely limited to the Microsoft Windows family and the Unix-like

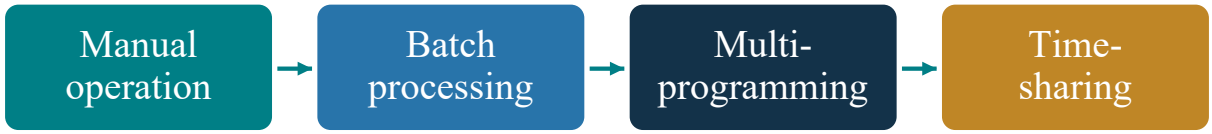
family, of which Linux and Mac OS X are becoming the major choices.

Mainframe computers and embedded systems use a variety of different operating systems, many with no direct connection to Windows or Unix, but typically more similar to Unix than Windows. The following show the OS based computer type:

- Personal computers
- IBM PC compatible - Microsoft Windows and smaller Unix-variants (like Linux and BSD)
- Apple Macintosh - Mac OS X, Windows, Linux and BSD
- Mainframes - A number of unique OS's, sometimes Linux and other Unix variants.
- Embedded systems - a variety of dedicated OS's, and limited versions of Linux or other OS's

VISUAL EXPLANATION

**From manual work to shared computing**




Conceptual progression of the historical approaches discussed in this section.

## 1.3. Operating System Functions

Following are some of important functions of an **operating System**:

- **Memory Management**
- Processor Management
- Device Management
- File Management
- Security
- Control over system performance
- Job accounting
- Error detecting aids
- Coordination between other software and users

VISUAL EXPLANATION

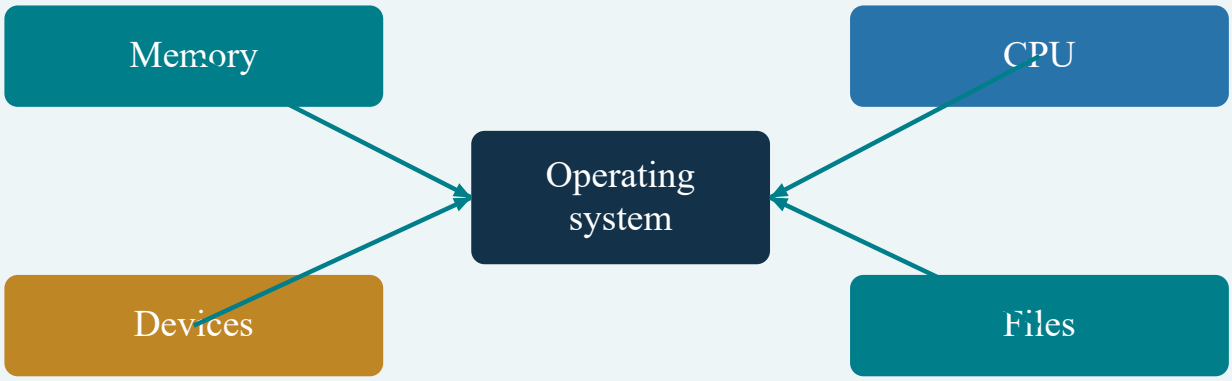
## Four resources the OS manages




These management functions coordinate the resources used by executing programs.

EXPLORE THE CONCEPT

### Resources coordinated by the OS




Processes request resources; the OS coordinates their use.

## 1.3.1. Memory Management

**Memory management** refers to management of Primary Memory or **Main Memory**. Main memory is a large array of words or bytes where each word or byte has its own address.

**Main memory** provides a fast storage that can be access directly by the CPU. So for a program to be executed, it must in the **main memory**. Operating System does the following activities for memory management.

- Keeps tracks of primary memory i.e. what part of it are in use by whom, what part are not in use.

- In multiprogramming, OS decides which process will get memory when and how much.

- Allocates the memory when the process requests it to do so.

• De-allocates the memory when the process no longer needs it or has been terminated.

## 1.3.2. Processor Management

In multiprogramming environment, OS decides which process gets the processor when and how much time. This function is called process scheduling. **Operating System** does the following activities for processor management.

- Keeps tracks of processor and status of process. Program responsible for this task is known as traffic controller.
- Allocates the processor (CPU) to a process.
- De-allocates processor when processor is no longer required.

## 1.3.3. Device Management

OS manages device communication via their respective drivers. **Operating System** does the following activities for device management.

- Keeps tracks of all devices. Program responsible for this task is known as the I/O controller.

• Decides which process gets the device when and for how much time.

• Allocates the device in the efficient way.

• De-allocates devices.

## 1.3.4. File Management

A **file system** is normally organized into directories for easy navigation and usage. These directories may contain files and other directions. **Operating System** does the following activities for file management.

- Keeps track of information, location, uses, status etc. The collective facilities are often known as **file system**.

- Decides who gets the resources.
- Allocates the resources.
- De-allocates the resources.

## 1.3.5. Security

Defense of the system against internal and external attacks likes denial-of-service, worms, or viruses. Systems generally first distinguish among users, to determine who can do what:

- User identities (**user IDs**, security IDs) include name and associated number, one per user
- User ID then associated with all files, processes of that user to determine access control
- Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
- **Privilege escalation** allows user to change to effective ID with more rights

## 1.3.6. Error detection

OS needs to be constantly aware of possible errors

- May occur in the CPU and memory hardware, in I/O devices, in user program
- For each type of error, OS should take the appropriate action to ensure correct and consistent computing
- Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

## 1.3.7. Control over system performance

It recording delays between request for a service and response from the system.

## 1.3.8. Job accounting

Keeping track of time and resources used by various jobs and users.

## 1.3.9. Coordination between other software and users

Coordination and assignment of compilers, interpreters, assemblers and other software to the various users of the computer systems.

## 1.4. Operating-System Services

> An **operating system** provides an environment for the execution of programs. It provides certain services to programs and to the users of those programs.

The specific services provided, of course, differ from one **operating system** to another, but we can identify common classes. These **operating system** services are provided for the convenience of the programmer, to make the programming task easier.

One set of **operating system** services provides functions that are helpful to the user.

VISUAL EXPLANATION

**Services used by a running program**




Selected operating-system services from this chapter; the full list remains in the chapter.

## 1.4.1. User interface.

Almost all operating systems have a user interface (UI). This interface can take several forms. One is a command- line interface (CLI), which uses text commands and a method for entering them (say, a keyboard for typing in commands in a specific format with specific options).

Another is a batch interface, in which commands and directives to control those commands are entered into files, and those files are executed. Most commonly, a graphical user interface (GUI) is used.

Here, the interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text. Some systems provide two or all three of these variations.

## 1.4.2. Program execution

**Operating system** handles many kinds of activities from user programs to system programs like printer spooler, name servers, file server etc. Each of these activities is encapsulated as a process.

A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use). Following are the major activities of an **operating system** with respect to program management.

1. Loads a program into memory.

2. Executes the program.

3. Handles program's execution.

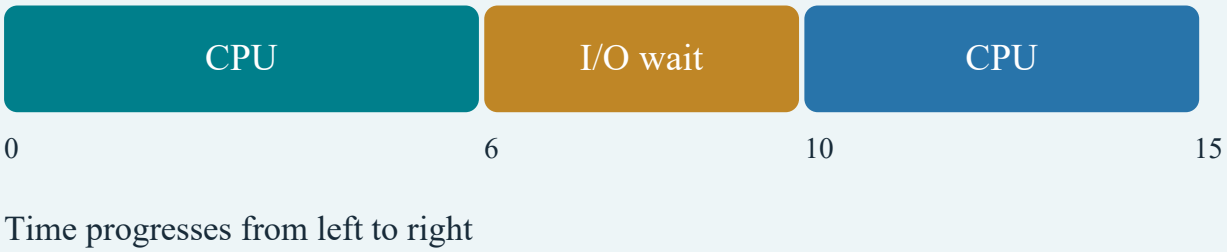
4. Provides a mechanism for process synchronization.

5. Provides a mechanism for process communication.

6. Provides a mechanism for deadlock handling.

EXPLORE THE CONCEPT

## From a program to its execution




A running program alternates between CPU work and periods of waiting.

# 1.4.3. I/O Operation

I/O subsystem comprised of I/O devices and their corresponding driver software. Drivers hides the characteristics of specific hardware devices from the user as the device driver knows the characteristics of the specific device.

**Operating System** manages the communication between user and device drivers. Following are the major activities of an **operating system** with respect to I/O Operation.

1. I/O operation means read or write operation with any

file or any specific I/O device.

2. Program may require any I/O device while running.

3. **Operating system** provides the access to the required

I/O device when required.

## 1.4.4. File system manipulation

A file represents a collection of related information. Computer can store files on the disk (secondary storage), for long term storage purpose. Few examples of storage media are magnetic tape, magnetic disk and optical disk drives like CD, DVD.

Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.

A **file system** is normally organized into directories for easy navigation and usage. These directories may contain files and other directions. Following are the major activities of an **operating system** with respect to file management.

1. Program needs to read a file or write a file.

2. The **operating system** gives the permission to the

program for operation on file.

3. Permission varies from read-only, read-write, denied

and so on.

4. **Operating System** provides an interface to the user to

create/delete files.

5. **Operating System** provides an interface to the user to

create/delete directories.

6. **Operating System** provides an interface to create the

backup of **file system**.

## 1.4.5. Communication

In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, **operating system** manages communications between processes.

Multiple processes with one another through communication lines in the network. OS handles routing and connection strategies, and the problems of contention and security. Following are the major activities of an **operating system** with respect to communication.

1. Two processes often require data to be transferred between them.
2. The both processes can be on the one computer or on different computer but are connected through computer network.
3. Communication may be implemented by two methods either by **Shared Memory** or by **Message Passing**.

## 1.4.6. Resource Management

In case of multi-user or multi-tasking environment, resources such as **main memory**, CPU cycles and files storage are to be allocated to each user or job and managed by OS. Following are the major activities of an **operating system** with respect to resource management.

1. OS manages all kind of resources using schedulers.
2. CPU scheduling algorithms are used for better

utilization of CPU.

# 1.4.7. Protection

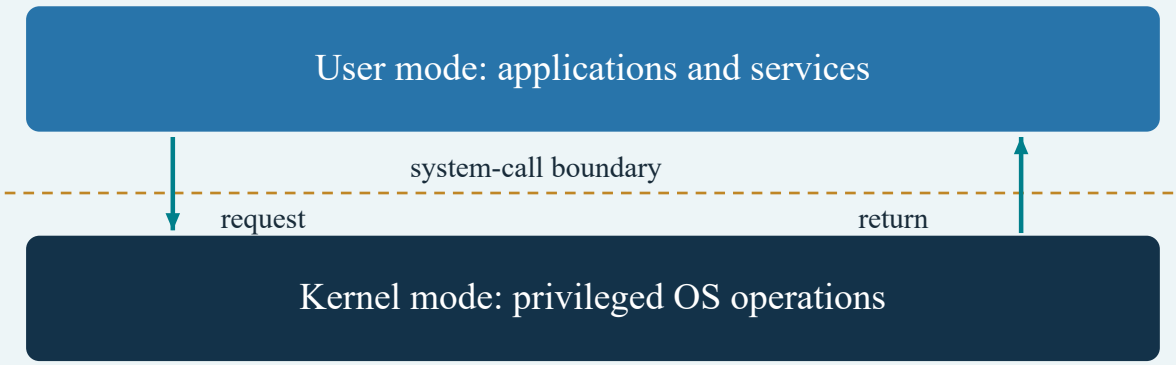
Considering computer systems having multiple users the concurrent execution of multiple processes, then the various processes must be protected from each another's activities.

Protection refers to mechanism or a way to control the access of programs, processes, or users to the resources defined by computer systems. Following are the major activities of an **operating system** with respect to protection.

1. OS ensures that all access to system resources is controlled.
2. OS ensures that external I/O devices are protected from invalid access attempts.
3. OS provides authentication feature for each user by means of a password.

EXPLORE THE CONCEPT

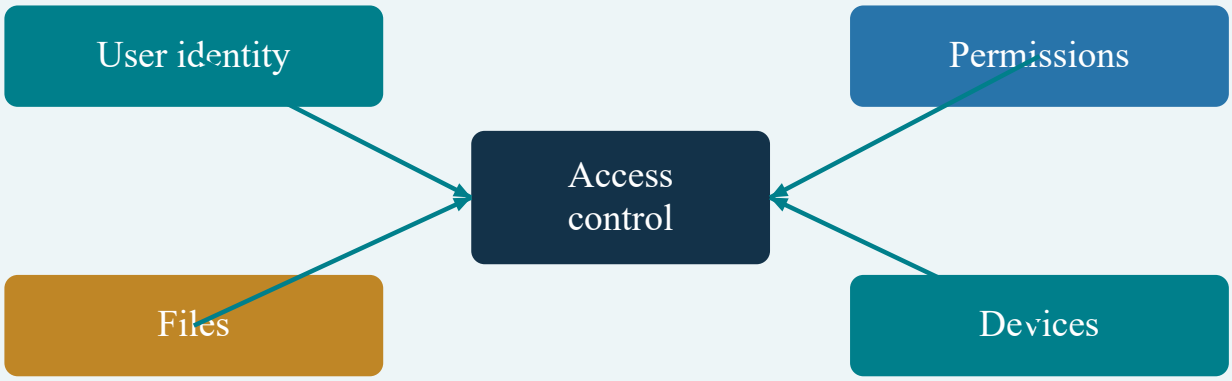
## Privileged operations cross a boundary




Applications request kernel services through a controlled interface.

EXPLORE THE CONCEPT

**Protection controls resource access**




Protection checks which users and processes may access a resource.

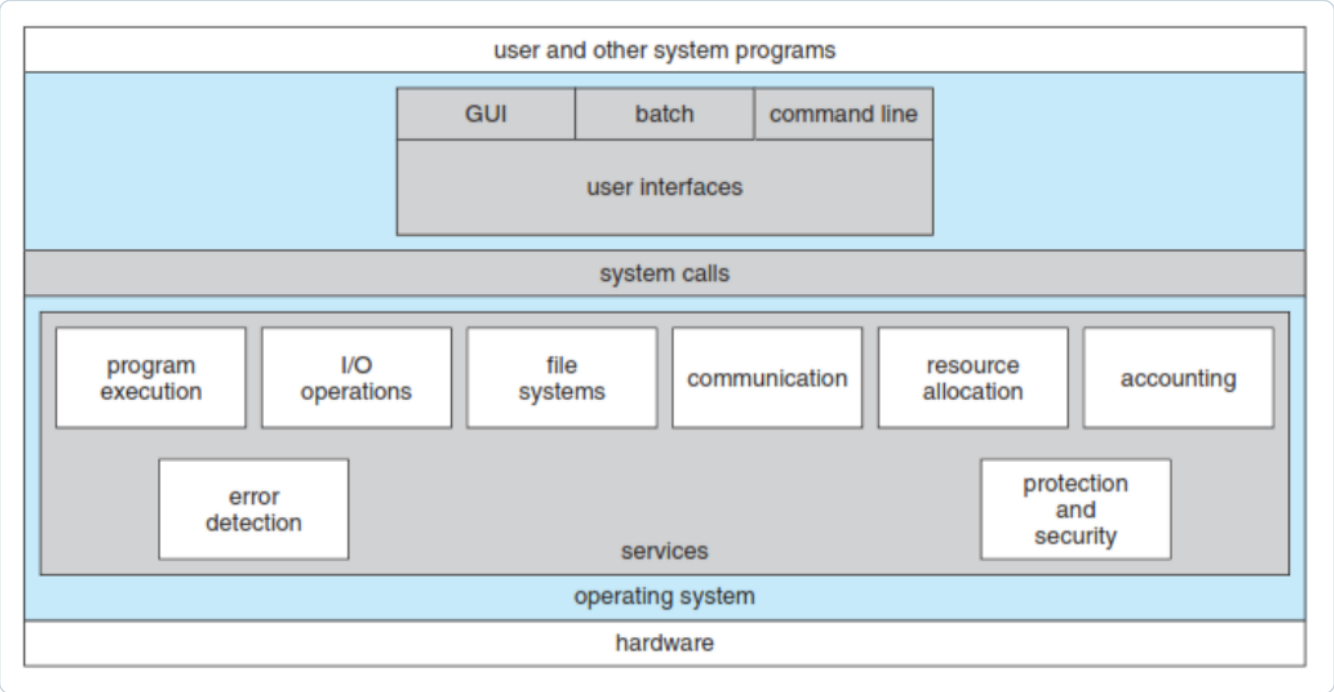
## 1.4.8. Error detection

Error can occur anytime and anywhere. Error may occur in CPU, in I/O devices or in the memory hardware. Following are the major activities of an **operating system** with respect to error handling.

1. OS constantly remains aware of possible errors.
2. OS takes the appropriate action to ensure correct and consistent computing.

## 1.4.9. Accounting:

We want to keep track of which users use how much and what kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics. Usage statistics may be a valuable tool for researchers who wish to reconfigure the system to improve computing services.



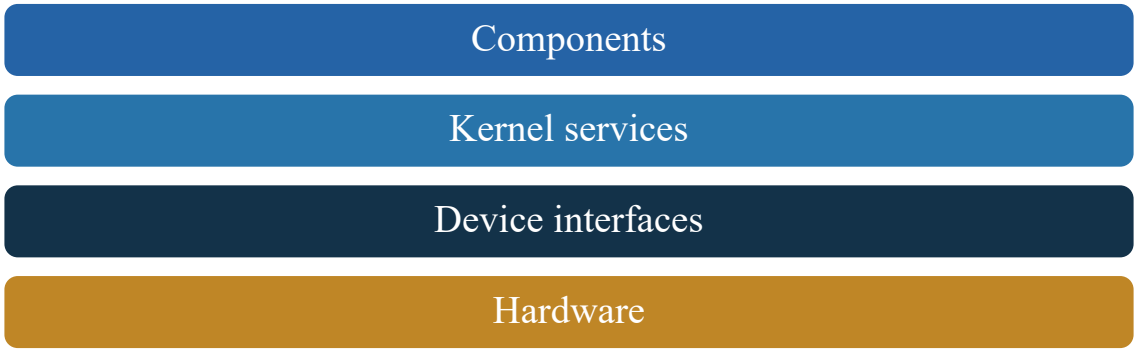

A view of **operating system** services.

CHAPTER 2

# OPERATING SYSTEMS STRUCTURE AND TYPES

VISUAL EXPLANATION

## Chapter at a glance




Use this route to see how the chapter topics connect.

## 2.1. Operating-System Structure

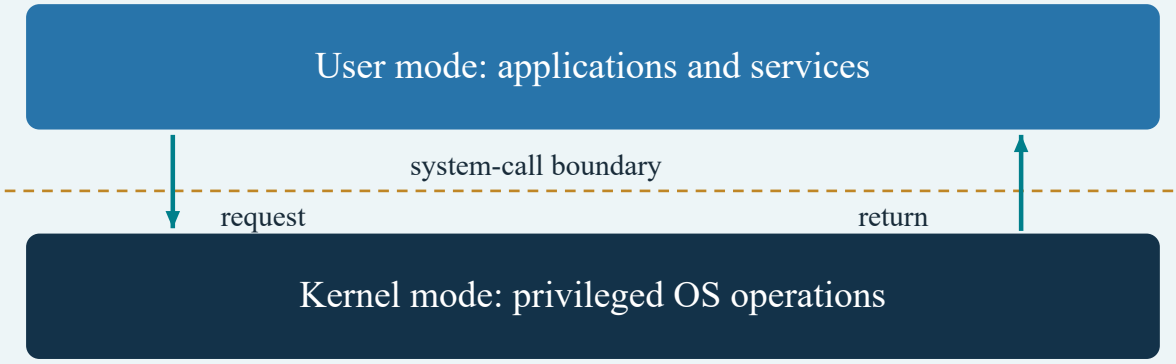
A system as large and complex as a modern **operating system** must be engineered carefully if it is to function properly and be modified easily. A common approach is to partition the task into small components, or modules, rather than have one monolithic system.

Each of these modules should be a well-defined portion of the system, with carefully defined inputs, outputs, and functions. Various ways to structure ones

- Simple structure – MS-DOS
- More complex -- UNIX
- Layered – an abstraction
- **Microkernel** -Mach

EXPLORE THE CONCEPT

### User mode and kernel mode

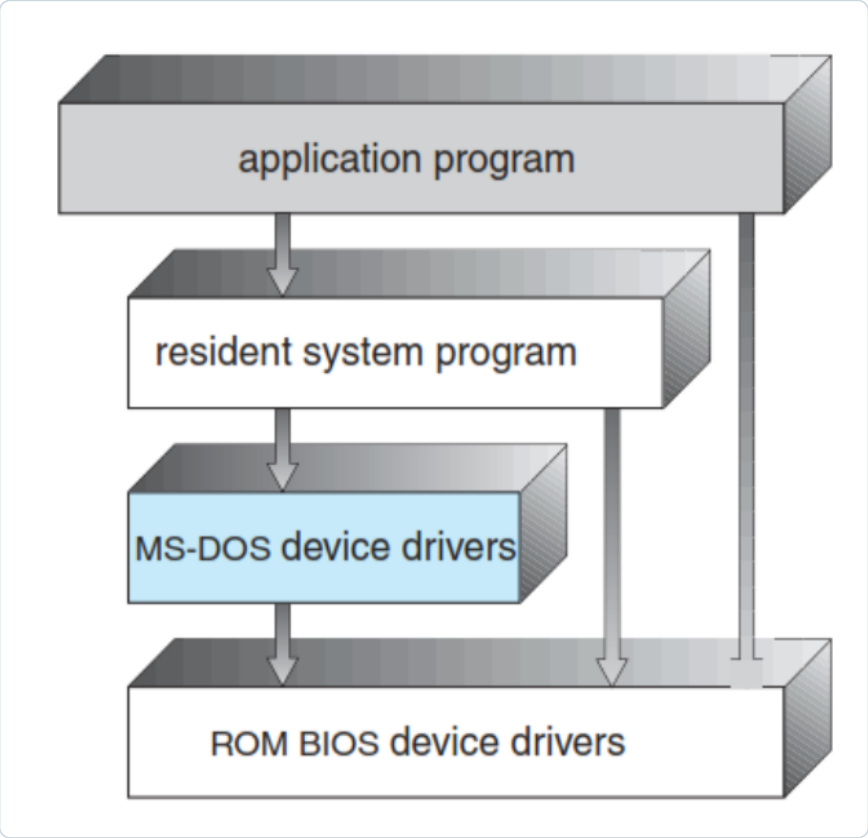



Kernel mode permits privileged operations needed to manage the machine.

## 2.1.1 Simple Structure

Many operating systems do not have well-defined structures. Frequently, such systems started as small, simple, and limited systems and then grew beyond their original scope. MS-DOS is an example of such a system.

It was originally designed and implemented by a few people who had no idea that it would become so popular. It was written to provide the most functionality in the least space, so it was not carefully divided into modules. The following figure shows its structure.




MS-DOS layer structure.

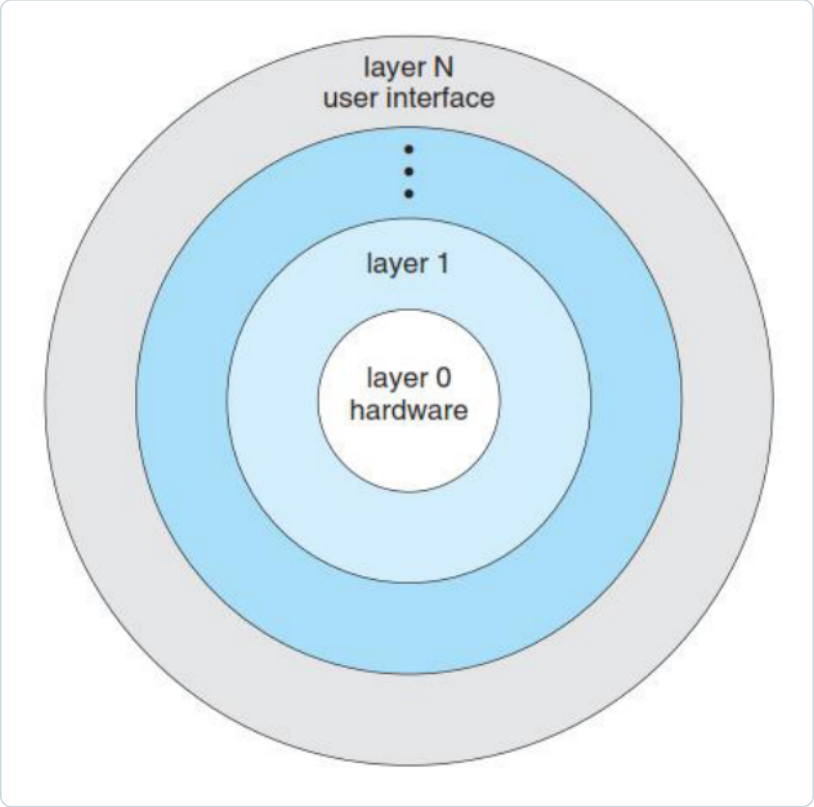
In MS-DOS, the interfaces and levels of functionality are not well separated. For instance, application programs are able to access the basic I/O routines to write directly to the display and disk drives.

## 2.1.2. Layered Approach

With proper hardware support, operating systems can be broken into pieces that are smaller and more appropriate than those allowed by the original MS-DOS.

Under a top- down approach, the overall functionality and features are determined and are separated into components. Information hiding is also important, because it leaves programmers free to implement the low-level routines as they see fit, provided that the external interface of the routine stays unchanged.

One method is the layered approach, in which the **operating system** is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface. This layering structure is depicted in the following figure




A layered **operating system**.

A typical operating-system layer—say, layer M— consists of data structures and a set of routines that can be invoked by higher-level layers. Layer M, in turn, can invoke operations on lower-level layers.

The main advantage of the layered approach is simplicity of construction and debugging. The layers are selected so that each uses functions (operations) and services of only lower-level layers. This approach simplifies debugging and system verification.

The first layer can be debugged without any concern for the rest of the system, because, by definition, it uses only the basic hardware (which is assumed correct) to implement its functions.

Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on. Each layer is implemented only with operations provided by lower- level layers. A layer does not need to know how these operations are implemented;

it needs to know only what these operations do. Hence, each layer hides the existence of certain data structures, operations, and hardware from higher-level layers.

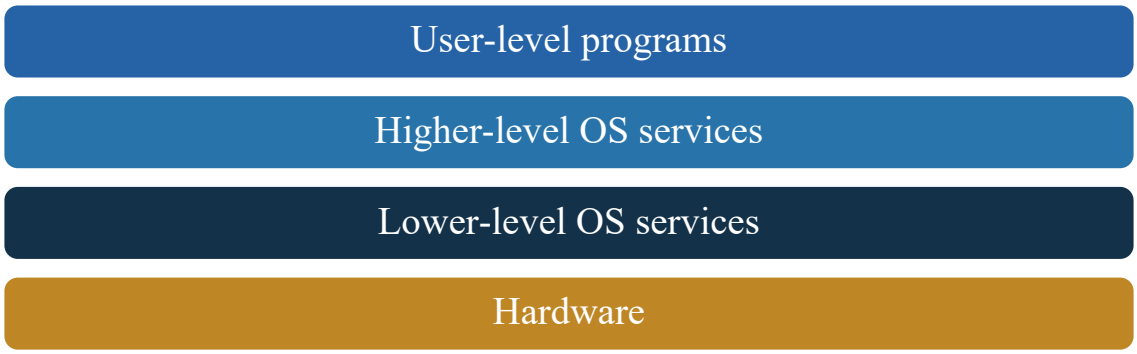
The major difficulty with the layered approach involves appropriately defining the various layers. A final problem with layered implementations is that they tend to be less efficient than other types. At each layer, the parameters may be modified;

data may need to be passed, and so on. Each layer adds overhead to the system call. The net result is a system call that takes longer than does one on a non- layered system. These limitations have caused a small backlash against layering in recent years.

Fewer layers with more functionality are being designed, providing most of the advantages of modularized code while avoiding the problems of layer definition and interaction.

VISUAL EXPLANATION

### A layered operating system

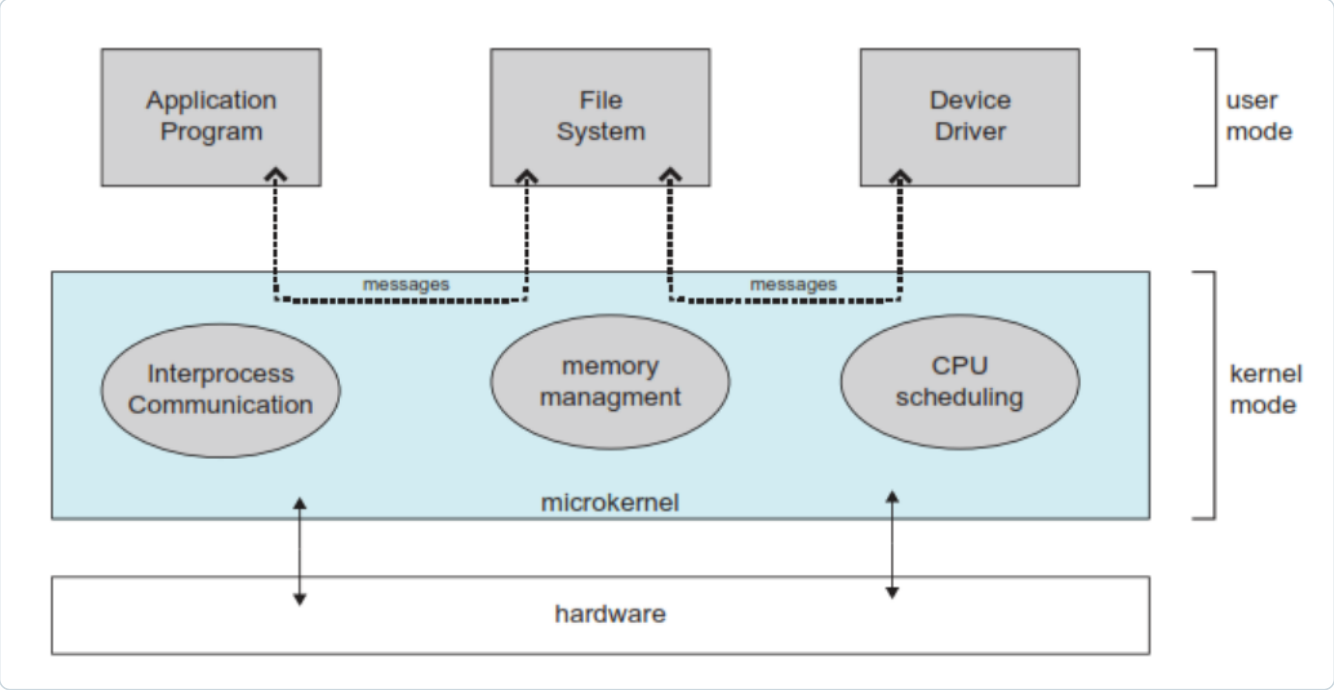



Conceptual layers: higher layers use services provided by lower layers.

## 2.1.3. Microkernels

The kernel became large and difficult to manage. This method structures the **operating system** by removing all nonessential components from the kernel and implementing them as system and user-level programs. The result is a smaller kernel.

Microkernels provide minimal process and **memory management**, in addition to a communication facility. The main function of the **microkernel** is to provide communication between the client program and the various services that are also running in user space as described in the following figure.




Architecture of a typical **microkernel**.

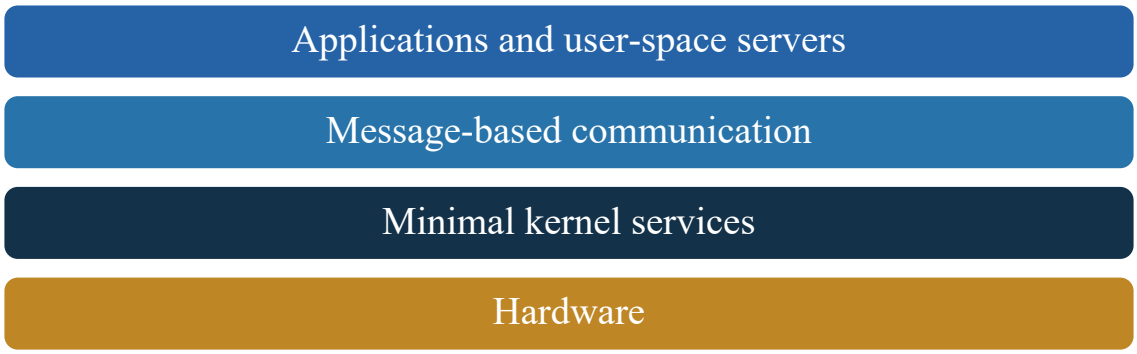
One benefit of the **microkernel** approach is that it makes extending the **operating system** easier. All new services are added to user space and consequently do not require modification of the kernel.

When the kernel does have to be modified, the changes tend to be fewer, because the **microkernel** is a smaller kernel. The resulting **operating system** is easier to port from one hardware design to another.

The **microkernel** also provides more security and reliability, since most services are running as user—rather than kernel— processes. If a service fails, the rest of the **operating system** remains untouched. Unfortunately, the performance of microkernels can suffer due to increased system-function overhead.

VISUAL EXPLANATION

### Microkernel organization




Some services run outside the kernel and communicate through the microkernel.

## 2.1.4. Modules

Perhaps the best current methodology for operating- system design involves using loadable kernel modules. Here, the kernel has a set of core components and links in additional services via modules, either at boot time or during run time.

This type of design is common in modern implementations of UNIX, such as Solaris, Linux, and Mac OS X, as well as Windows.

The idea of the design is for the kernel to provide core services while other services are implemented dynamically, as the kernel is running. Linking services dynamically is preferable to adding new features directly to the kernel, which would require recompiling the kernel every time a change was made.

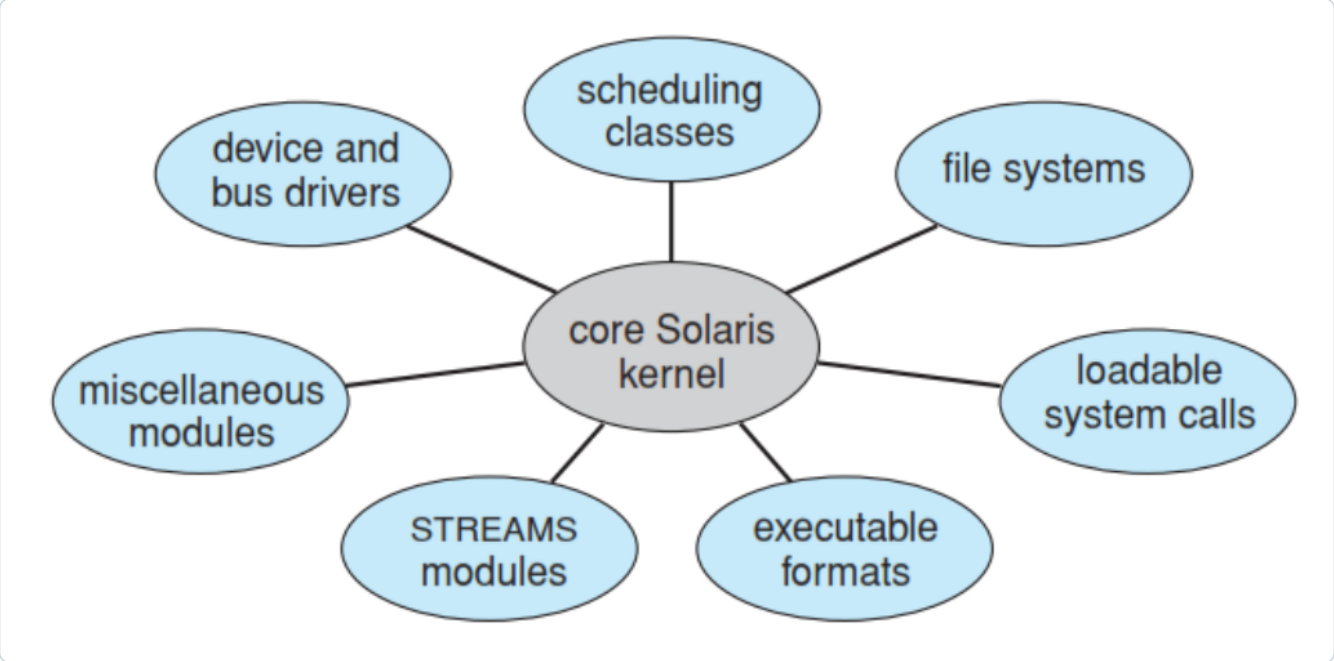
Thus, for example, we might build CPU scheduling and **memory management** algorithms directly into the kernel and then add support for different file systems by way of loadable modules.

The overall result resembles a layered system in that each kernel section has defined, protected interfaces; but it is more flexible than a layered system, because any module can call any other module.

The approach is also similar to the **microkernel** approach in that the primary module has only core functions and knowledge of how to load and communicate with other modules; but it is more efficient, because modules do not need to invoke **message passing** in order to communicate.

The Solaris **operating system** structure, shown in the following figure, is organized around a core kernel with seven types of loadable kernel modules:

1. Scheduling classes
2. File systems
3. Loadable system calls
4. Executable formats
5. STREAMS modules
6. Miscellaneous
7. Device and bus drivers




Solaris loadable modules.

Linux also uses loadable kernel modules, primarily for supporting device drivers and file systems.

## 2.1.5. Hybrid Systems

In practice, very few operating systems adopt a single, strictly defined structure. Instead, they combine different structures, resulting in hybrid systems that address performance, security, and usability issues.

For example Windows is largely monolithic, but it retains some behavior typical of **microkernel** systems, including providing support for separate subsystems that run as user-mode processes. Windows systems also provide support for dynamically loadable kernel modules.

## 2.2. Types of Operating Systems

Operating systems are there from the very first computer generation. Operating systems keep evolving over the period of time. Following are few of the important types of **operating system** which are most commonly used.

### 2.2.1. Batch operating system

The users of batch **operating system** do not interact with the computer directly. Each user prepares his job on an off- line device like punch cards and submits it to the computer operator.

To speed up processing, jobs with similar needs are batched together and run as a group. Thus, the programmers left their programs with the operator. The operator then sorts programs into batches with similar requirements.

The problems with Batch Systems are following.

- Lack of interaction between the user and job.
- CPU is often idle, because the speeds of the mechanical I/O devices are slower than CPU.
- Difficult to provide the desired **priority**.

EXPLORE THE CONCEPT

### Ready work can share the processor




Time progresses from left to right

Illustrative CPU sequence: the OS switches between ready processes.

## 2.2.2. Time-sharing operating systems

Time sharing is a technique which enables many people, located at various terminals, to use a particular computer system at the same time. Time-sharing or multitasking is a logical extension of multiprogramming.

Processor's time which is shared among multiple users simultaneously is termed as time-sharing. The main difference between Multiprogrammed Batch Systems and Time-Sharing Systems is that in case of multiprogrammed batch systems, objective is to maximize processor use,

whereas in Time- Sharing Systems objective is to minimize **response time**.

Multiple jobs are executed by the CPU by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response.

For example, in a transaction processing, processor execute each user program in a short burst or quantum of computation. That is if n users are present, each user can get **time quantum**. When the user submits the command, the **response time** is in few seconds at most.

**Operating system** uses CPU scheduling and multiprogramming to provide each user with a small portion of time. Computer systems that were designed primarily as batch systems have been modified to time- sharing systems.

Advantages of Timesharing operating systems are following

- Provide advantage of quick response.
- Avoids duplication of software.
- Reduces CPU idle time.

Disadvantages of Timesharing operating systems are following.

- Problem of reliability.
- Question of security and integrity of user programs and data.
- Problem of data communication.

VISUAL EXPLANATION

### Batch and time sharing




The key distinction here is how work is submitted and how users interact with it.

## 2.2.3. Distributed operating System

Distributed systems use multiple central processors to serve multiple real time application and multiple users. Data processing jobs are distributed among the processors accordingly to which one can perform each job most efficiently.

The processors communicate with one another through various communication lines (such as high-speed buses or telephone lines). These are referred as loosely coupled systems or distributed systems. Processors in a distributed system may vary in size and function. These processors are referred as sites, nodes, and computers and so on.

The advantages of distributed systems are following.

- With resource sharing facility user at one site may be able to use the resources available at another.
- Speedup the exchange of data with one another via electronic mail.

- If one site fails in a distributed system, the remaining sites can potentially continue operating.

- Better service to the customers.
- Reduction of the load on the host computer.
- Reduction of delays in data processing.

## 2.2.4. Network operating System

Network **Operating System** runs on a server and provides server the capability to manage data, users, groups, security, applications, and other networking functions.

The primary purpose of the network **operating system** is to allow shared file and printer access among multiple computers in a network, typically a local area network (LAN), a private network or to other networks.

Examples of network operating systems are Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD.

The advantages of network operating systems are following.

- Centralized servers are highly stable.
- Security is server managed.

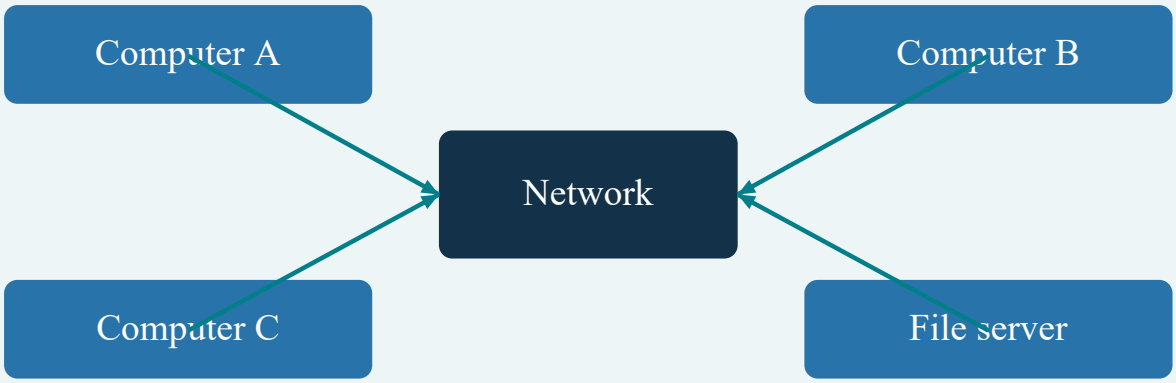
- Upgrades to new technologies and hardware can be easily integrated into the system.
- Remote access to servers is possible from different locations and types of systems.

The disadvantages of network operating systems are following.

- High cost of buying and running a server.
- Dependency on a central location for most operations.
- Regular maintenance and updates are required.

EXPLORE THE CONCEPT

**Computers connected by a network**




Network services allow computers to communicate and share selected resources.

## 2.2.5. Real Time operating System

Real time system is defines as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. Real time processing is always on line whereas on line system need not be real time.

The time taken by the system to respond to an input and display of required updated information is termed as **response time**. So in this method **response time** is very less as compared to the online processing.

Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application.

Real-time **operating system** has well-defined, fixed time constraints otherwise system will fail. For example Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, and home-appliance controllers, Air traffic control system etc.

There are two types of real-time operating systems.

*Hard real-time systems*

Hard real-time systems guarantee that critical tasks complete on time. In hard real-time systems secondary storage is limited or missing with data stored in ROM. In these systems virtual memory is almost never found.

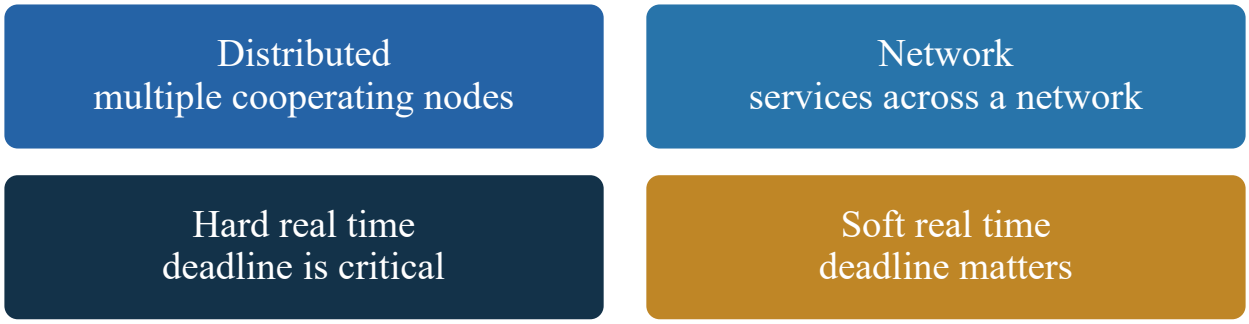
*Soft real-time systems*

Soft real time systems are less restrictive. Critical real- time task gets **priority** over other tasks and retains the **priority** until it completes. Soft real-time systems have limited utility than hard real-time systems.

For example, Multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers etc.

VISUAL EXPLANATION

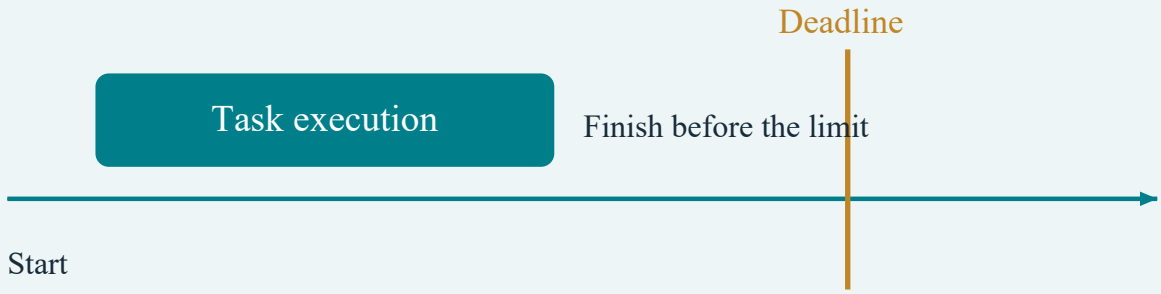
## Operating-system types




A conceptual comparison of the types described in this section.

EXPLORE THE CONCEPT

## Real-time work has a deadline




Timing is part of correctness in hard real-time systems.

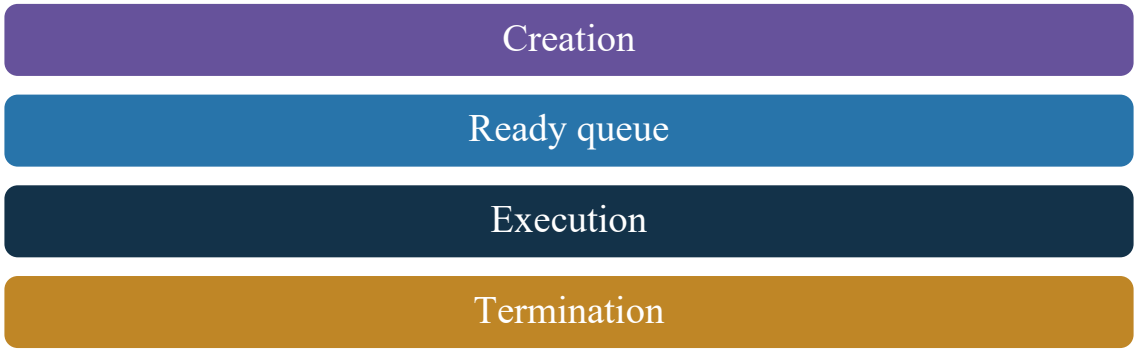
In hard real-time systems, completing after the deadline can make a result unusable.

CHAPTER 3

# PROCESS MANAGEMENT

VISUAL EXPLANATION

## Chapter at a glance




Use this route to see how the chapter topics connect.

A process can be thought of as a program in execution. A process will need certain resources—such as CPU time, memory, files, and I/O devices —to accomplish its task. These resources are allocated to the process either when it is created or while it is executing.

A process is the unit of work in most systems. Systems consist of a collection of processes. The execution of a process must progress in a sequential fashion. So, the process is defines as an entity which represents the basic unit of work to be implemented in the system.

A process generally also includes the process stack, which contains temporary data (such as function parameters, return addresses, and local variables), and a data section, which contains global variables.

A program by itself is not a process. It is a static entity made up of program statement while process is a dynamic entity. Program contains the instructions to be executed by processor. A program takes a space at single place in **main memory** and continues to stay there. A program does not perform any action by itself.

## 3.1. Process State

As a process executes, it changes state. The state of a process is defined in part by the current activity of that process. A process may be in one of the following states:

• New. The process is being created.

• Running. Instructions are being executed.

• Waiting. The process is waiting for some event to occur (such as an I/O completion or reception of a signal).

• Ready. The process is waiting to be assigned to a processor.

• Terminated. The process has finished execution.

The state diagram corresponding to these states is presented in the following figure.

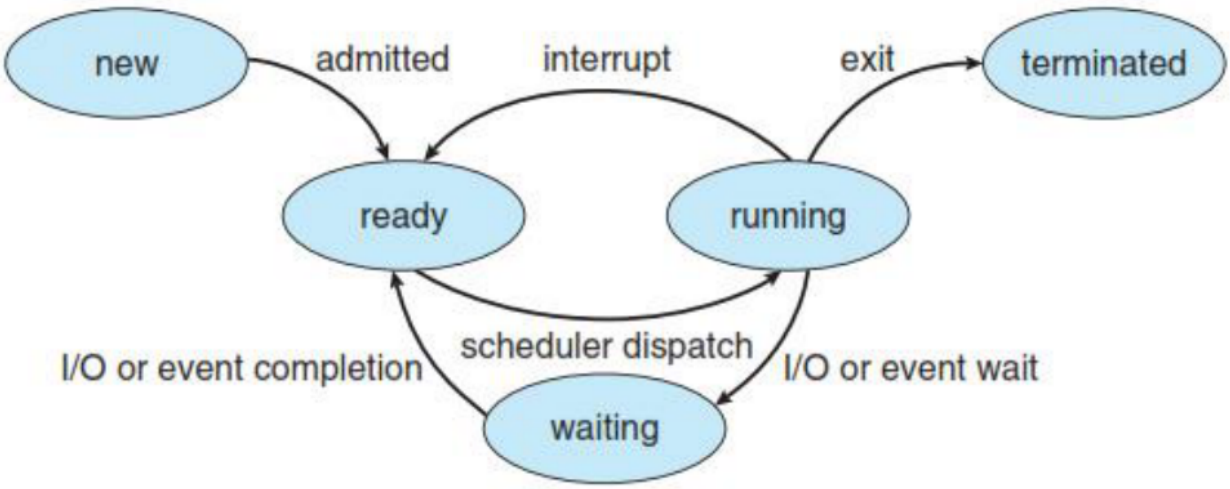
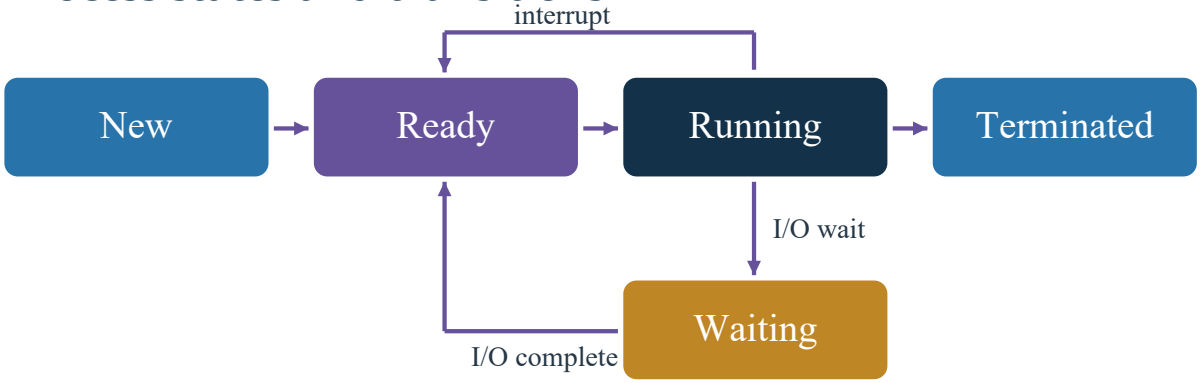



Diagram of process state.

VISUAL EXPLANATION

**Process states and transitions**




Dispatch selects a ready process. I/O may block it; I/O completion makes it ready again.

## 3.2. Process Control Block

Each process is represented in the **operating system** by a **process control block** (PCB) also called a task control block. PCB is the data structure used by the operating system. Operating system groups all information that needs about particular process.

PCB contains many pieces of information associated with a specific process which is described below.

- Process state: The state may be new, ready, running, waiting, halted, and so on.
- Program counter: The counter indicates the address of the next instruction to be executed for this process.
- CPU registers: The registers vary in number and type, depending on the computer architecture.

They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.

- CPU-scheduling information: This information includes a process **priority**, pointers to scheduling queues, and any other scheduling parameters.
- Memory-management information: This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the **operating system**
- Accounting information: This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- I/O status information: This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

VISUAL EXPLANATION

### What a PCB records




The process control block holds information needed to manage and resume a process.

## 3.3. Process Scheduling

The objective of multiprogramming is to have some process running at all times, to maximize **CPU utilization**. The objective of time sharing is to switch the CPU among processes so frequently that users can interact with each program while it is running.

To meet these objectives, the process scheduler selects an available process (possibly from a set of several available processes) for program execution on the CPU. For a single-processor system, there will never be more than one running process.

If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled.

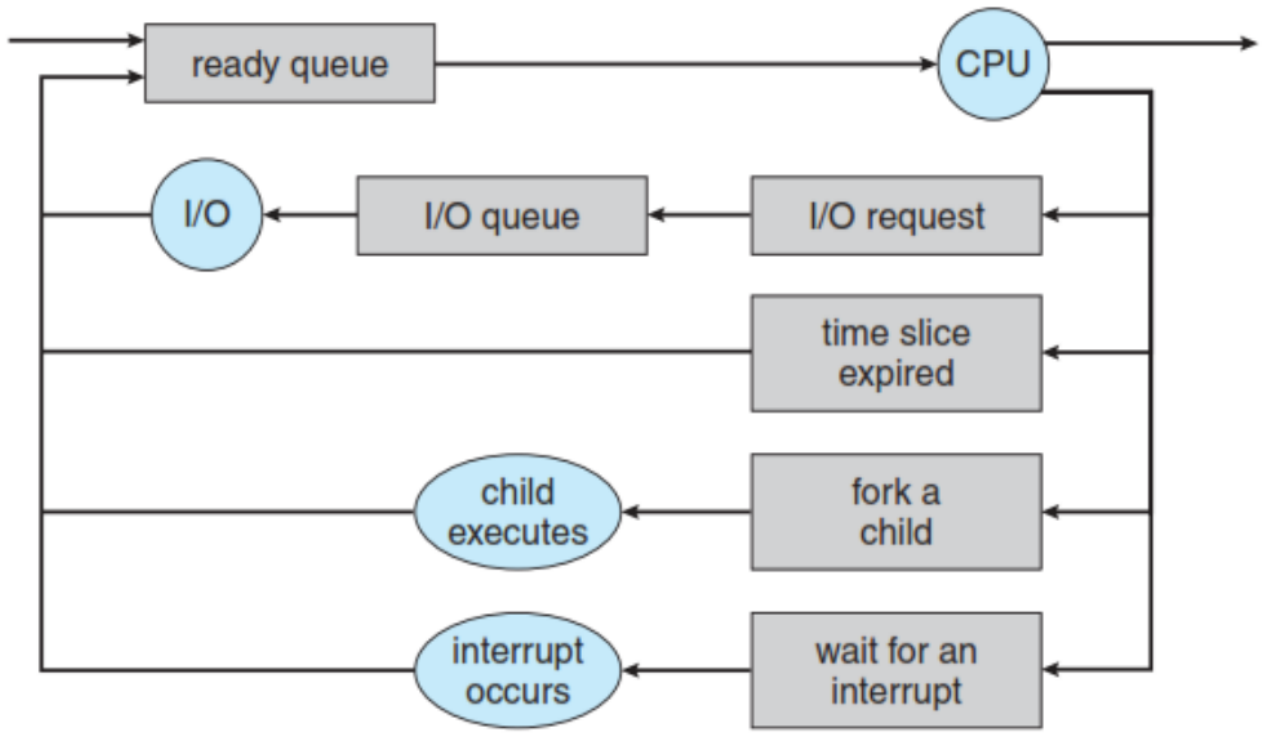
# 3.4. Scheduling Queues

Scheduling queues refers to queues of processes or devices. When the process enters into the system, then this process is put into a job queue. This queue consists of all processes in the system. The **operating system** also maintains other queues such as device queue.

Device queue is a queue for which multiple processes are waiting for a particular I/O device. Each device has its own device queue.

This figure shows the queuing diagram of process scheduling.

- Queue is represented by rectangular box.
- The circles represent the resources that serve the queues.
- The arrows indicate the process flow in the system.




Queuing-diagram representation of process scheduling.

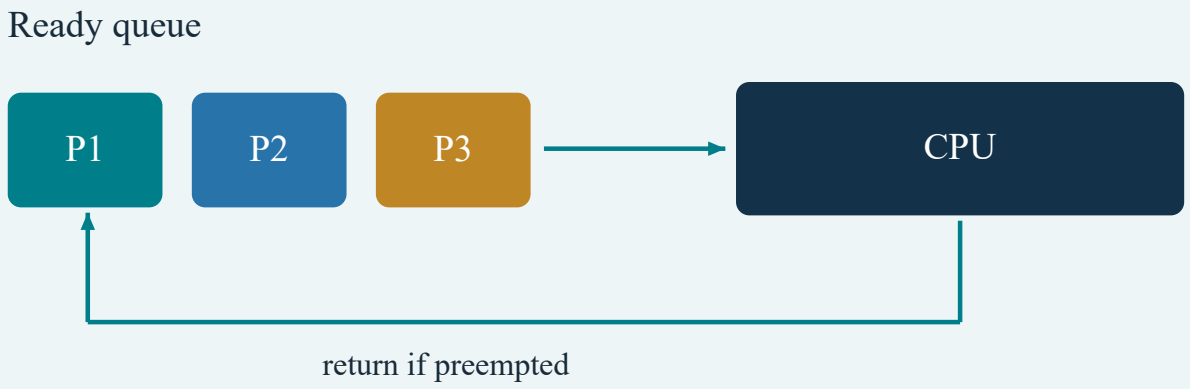
A new process is initially put in the **ready queue**. It waits there until it is selected for execution, or dispatched. Once the process is allocated the CPU and is executing, one of several events could occur:

- The process could issue an I/O request and then be placed in an I/O queue.
- The process could create a new child process and wait for the child's termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the **ready queue**.

In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the **ready queue**. A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

EXPLORE THE CONCEPT

## The ready queue feeds the CPU




Ready processes wait for CPU allocation; a preempted process can return to the queue.

## 3.5. Schedulers

A process migrates among the various scheduling queues throughout its lifetime. The **operating system** must select, for scheduling purposes, processes from these queues in some fashion. The selection process is carried out by the appropriate scheduler.

Schedulers are special system software which handles process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types

- Long Term Scheduler
- Short Term Scheduler
- Medium Term Scheduler

## 3.5.1. Long Term Scheduler

It is also called job scheduler. Long term scheduler determines which programs are admitted to the system for processing. Job scheduler selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

On some systems, the long term scheduler may not be available or minimal. Time-sharing operating systems have no long term

scheduler. When process changes the state from new to ready, then there is use of long term scheduler.

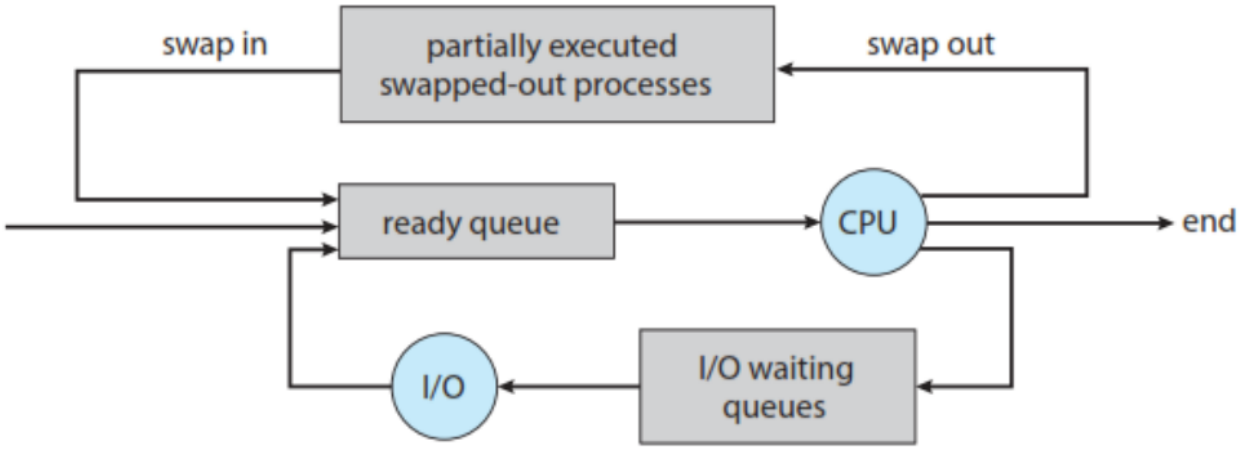
## 3.5.2. Short Term Scheduler

It is also called CPU scheduler. Main objective is increasing system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects process among the processes that are ready to execute and allocates CPU to one of them.

Short term scheduler also known as dispatcher, execute most frequently and makes the fine grained decision of which process to execute next. Short term scheduler is faster than long term scheduler.

## 3.5.3. Medium Term Scheduler

Medium term scheduling is part of the **swapping**. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium term scheduler is in-charge of handling the swapped out-processes.




Addition of medium-term scheduling to the queuing diagram

Running process may become suspended if it makes an I/O request. Suspended processes cannot make any progress towards completion.

In this condition, to remove the process from memory and make space for other process, the suspended process is moved to the secondary storage. This process is called **swapping**, and the process is said to be swapped out or rolled out. **Swapping** may be necessary to improve the process mix.

| S.N. | Long Term Scheduler | Short Term Scheduler | Medium Term Scheduler |
|---|---|---|---|
| 1 | It is a job scheduler | It is a CPU scheduler | It is a process swapping scheduler. |
| 2 | Speed is lesser than short term scheduler | Speed is fastest among other two | Speed is in between both short and long term scheduler. |
| 3 | It controls the degree of multiprogramming | It provides lesser control over degree of multiprogramming | It reduces the degree of multiprogramming. |
| 4 | It is almost absent or minimal in time sharing system | It is also minimal in time sharing system | It is a part of Time sharing systems. |
| 5 | It selects processes from pool and loads them into memory for execution | It selects those processes which are ready to execute | It can re-introduce the process into memory and execution can be continued. |

VISUAL EXPLANATION

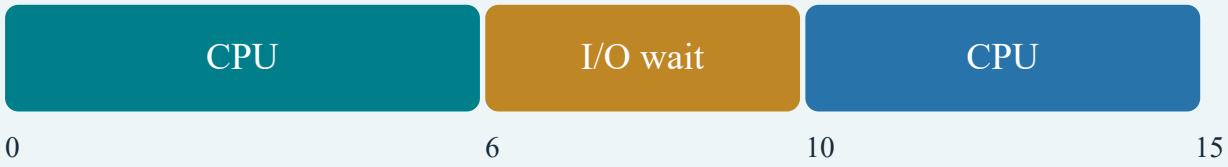
### Three scheduling decisions




These decisions operate at different points in the life of a process.

EXPLORE THE CONCEPT

### CPU bursts and I/O waiting




Time progresses from left to right

Waiting for I/O allows another ready process to use the processor.

## 3.6. Context Switch

A **context switch** is the mechanism to store and restore the state or context of a CPU in **Process Control block** so that a process execution can be resumed from the same point at a later time.

Using this technique a context switcher enables multiple processes to share a single CPU. Context switching is an essential part of a multitasking **operating system** features.

When the scheduler switches the CPU from executing one process to execute another, the context switcher saves the content of all processor registers for the process being removed from the CPU, in its process descriptor. The context of a process is represented in the **process control block** of a process. **Context switch** time is pure overhead.

Context switching can significantly affect performance as modern computers have a lot of general and status registers to be saved. Content switching times are highly dependent on hardware support.

**Context switch** requires ( n + m ) b*K time units to save the state of the processor with n general registers, assuming b are the store operations are required to save n and m registers of two process control blocks and each store instruction requires K time units.

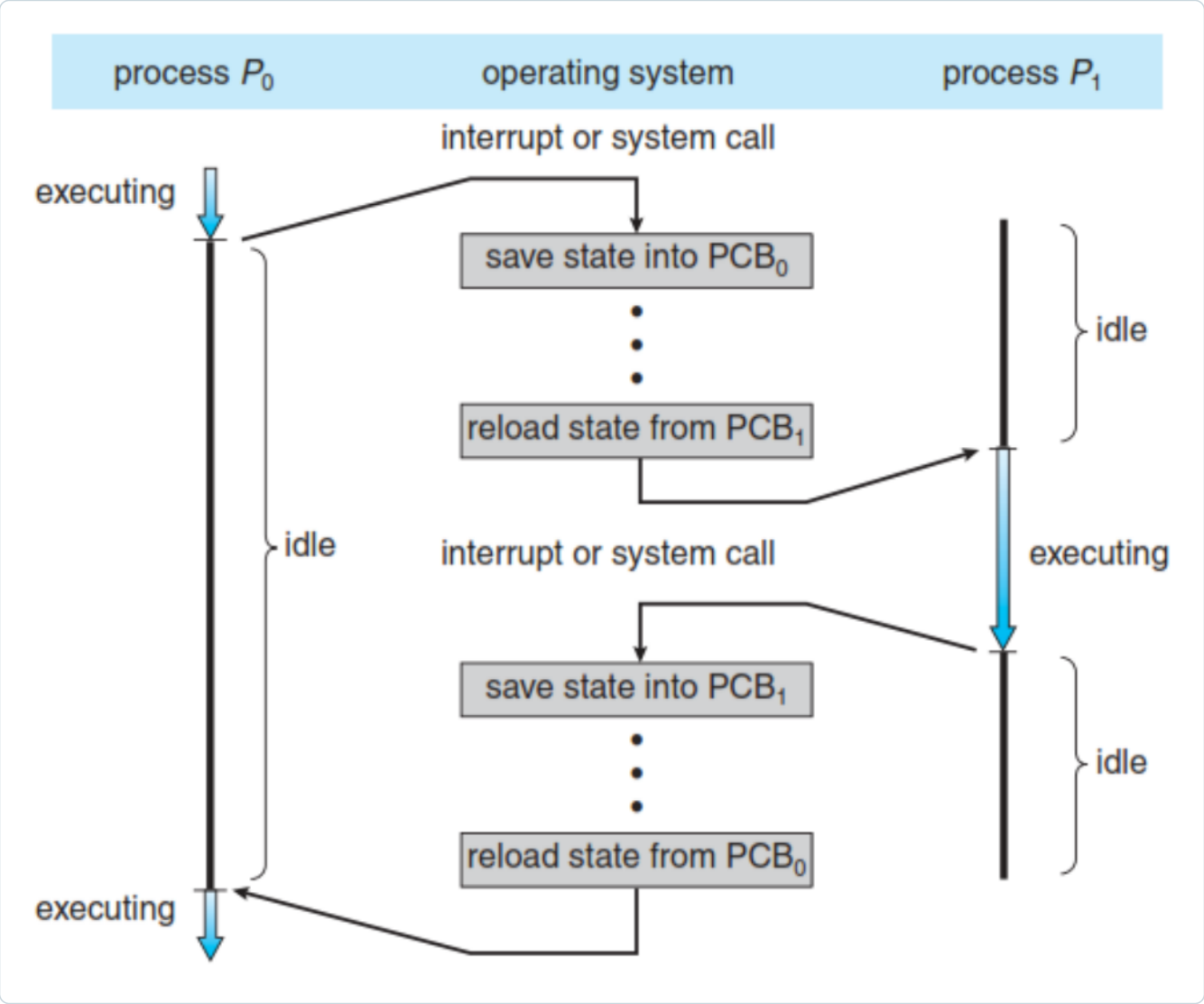



Diagram showing CPU switch from process to process.

Some hardware systems employ two or more sets of processor registers to reduce the amount of context switching time. When the process is switched, the following information is stored.

- Program Counter
- Scheduling Information
- Base and limit register value
- Currently used register
- Changed State
- I/O State

- Accounting

VISUAL EXPLANATION

**Context switch**



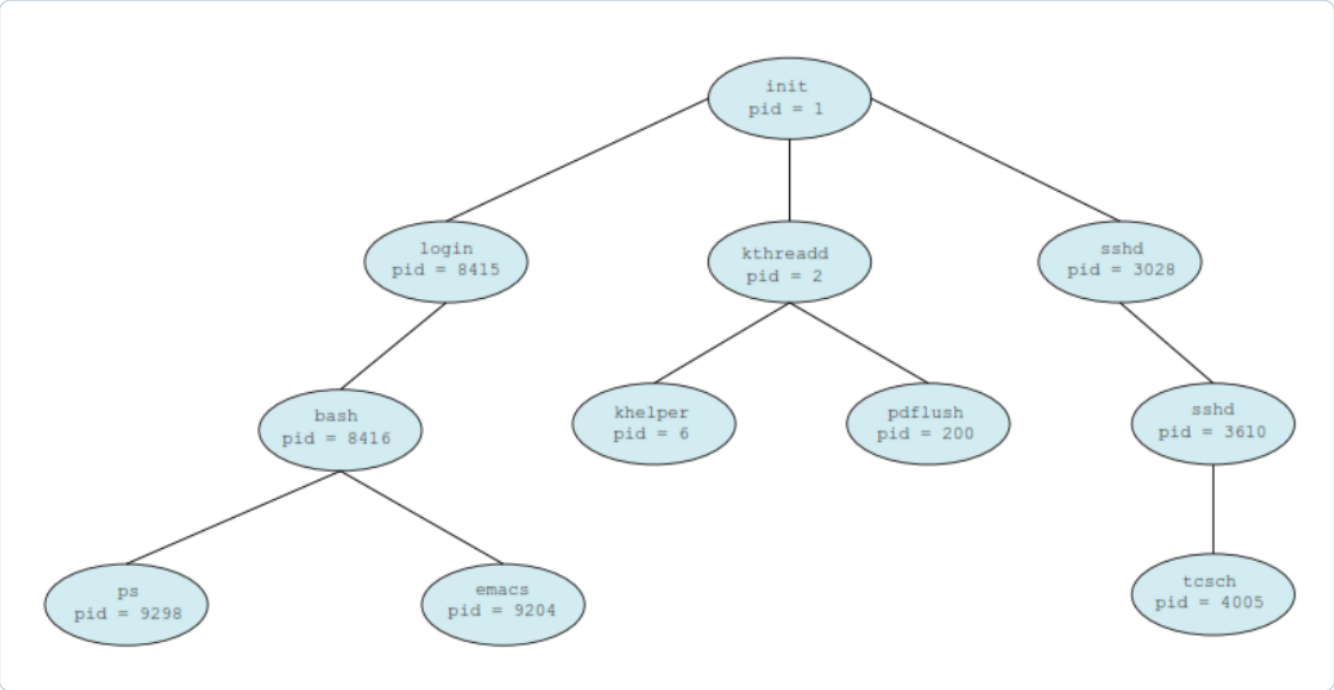

The OS preserves one execution context and restores another before execution continues.

## 3.7. Process Creation

During the course of execution, a process may create several new processes. As mentioned earlier, the creating process is called a parent process, and the new processes are called the children of that process. Each of these new processes may in turn create other processes, forming a tree of processes.

Most operating systems (including UNIX, Linux, and Windows) identify processes according to a unique process identifier (or pid), which is typically an integer number. The pid provides a unique value for each process in the system.

the following figure illustrates a typical process tree for the Linux **operating system**, showing the name of each process and its pid.




When a process creates a child process, that child process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task.

A child process may be able to obtain its resources directly from the **operating system**, or it may be constrained to a subset of the resources of the parent process.

The parent may have to partition its resources among its children, or it may be able to share some resources (such as memory or files) among several of its children. Restricting a child process to a subset of the parent's resources prevents any process from overloading the system by creating too many child processes.

The parent process may pass along initialization data (input) to the child process. For example, consider a process whose function is to display the contents of a file say, image.jpg—on the screen of a terminal.

When the process is created, it will get, as an input from its parent process, the name of the file image.jpg. Using that file name, it will open the file and write the contents out. It may also get the name of the output device.

Alternatively, some operating systems pass resources to child processes. On such a system, the new process may get two open files, image.jpg and the terminal device, and may simply transfer the data between the two.

When a process creates a new process, two possibilities for execution exist:

1. The parent continues to execute concurrently with its children.
2. The parent waits until some or all of its children have terminated. There are also two address-space possibilities for the new process:

1. The child process is a duplicate of the parent process (it has the same program and data as the parent).
2. The child process has a new program loaded into it.

## 3.8. Process Termination

A process terminates when it finishes executing its final statement and asks the **operating system** to delete it by using the exit() system call.

At that point, the process may return a status value (typically an integer) to its parent process (via the wait() system call). All the resources of the process—including physical and virtual memory, open files and I/O buffers—are deallocated by the **operating system**.

Termination can occur in other circumstances as well. A process can cause the termination of another process via an appropriate system call (for example, TerminateProcess() in Windows).

Usually, such a system call can be invoked only by the parent of the process that is to be terminated. Otherwise, users could arbitrarily kill each other's jobs. Note that a parent needs to know the identities of its children if it is to terminate them.

Thus, when one process creates a new process, the identity of the newly created process is passed to the parent.

A parent may terminate the execution of one of its children for a variety of reasons, such as these:

- The child has exceeded its usage of some of the resources that it has been allocated.
- The task assigned to the child is no longer required.
- The parent is exiting, and the **operating system** does not allow a child to continue if its parent terminates. Some systems do not allow a child to exist if its parent has terminated.

  In such systems, if a process terminates (either normally or abnormally), then all its children must also be terminated. This phenomenon, referred to as cascading termination, is normally initiated by the **operating system**.

# 3.9. Interprocess Communication

Processes executing concurrently in the **operating system** may be either independent processes or cooperating processes. A process is independent if it cannot affect or be affected by the other processes executing in the system.

Any process that does not share data with any other process is independent. A process is cooperating if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

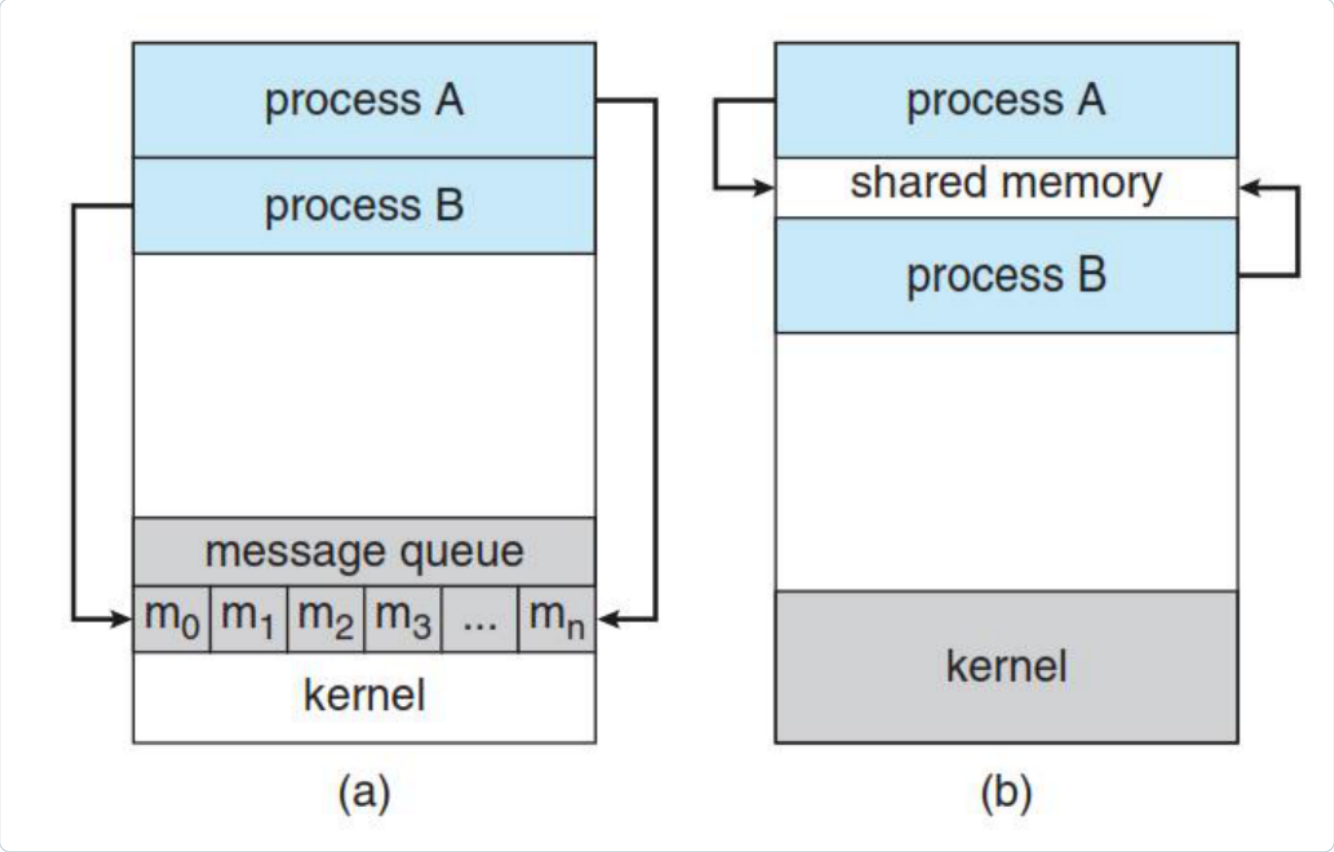
There are several reasons for providing an environment that allows process cooperation:

- Information sharing. Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to such information.
- Computation speedup. If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing cores.
- Modularity. We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.
- Convenience. Even an individual user may work on many tasks at the same time. For instance, a user may be editing, listening to music, and compiling in parallel.

Cooperating processes require an interprocess communication (IPC) mechanism that will allow them to exchange data and information. There are two fundamental models of interprocess communication: **shared memory** and **message passing**.

In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.

In the message-passing model, communication takes place by means of messages exchanged between the cooperating processes. The two communications models are contrasted in the following figure.




Communications models. (a) **Message passing**. (b) **Shared memory**.

Both of the models just mentioned are common in operating systems, and many systems implement both. **Message passing** is useful for

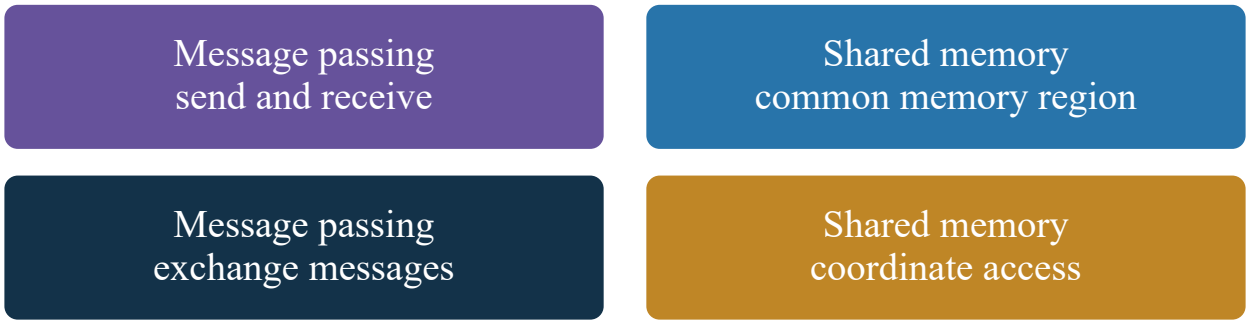
exchanging smaller amounts of data, because no conflicts need be avoided.

**Message passing** is also easier to implement in a distributed system than **shared memory**. Shared memory can be faster than message passing, since message-passing systems are typically implemented using system calls and thus require the more time-consuming task of kernel intervention.

In shared-memory systems, system calls are required only to establish shared-memory regions. Once **shared memory** is established, all accesses are treated as routine memory accesses, and no assistance from the kernel is required.

VISUAL EXPLANATION

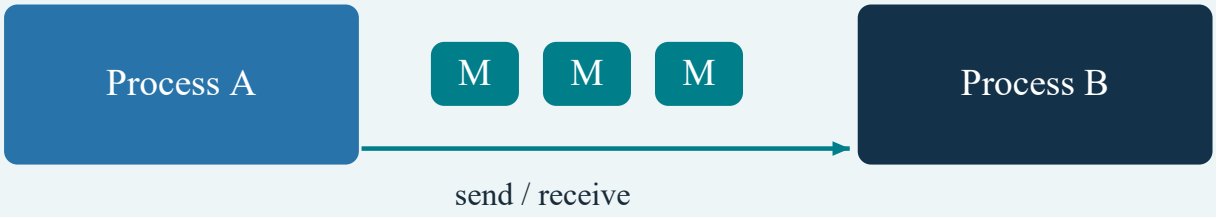
## Two communication models




Both models support interprocess communication; their mechanisms differ.

EXPLORE THE CONCEPT

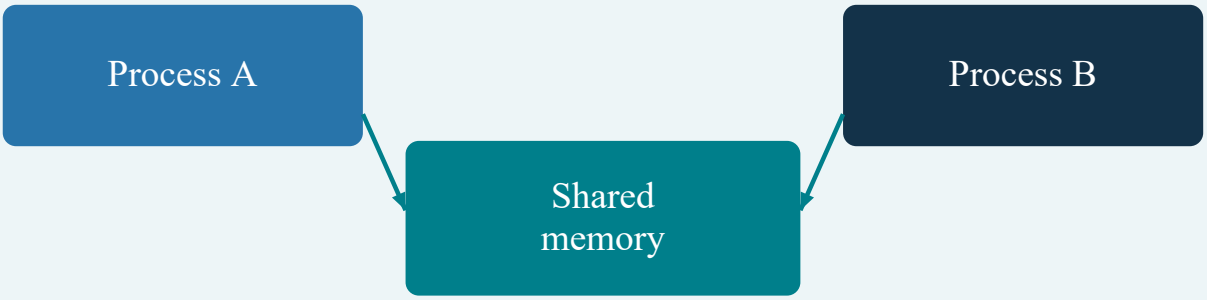
## Message passing exchanges data




Processes communicate by sending and receiving messages.

EXPLORE THE CONCEPT

## Shared memory holds common data



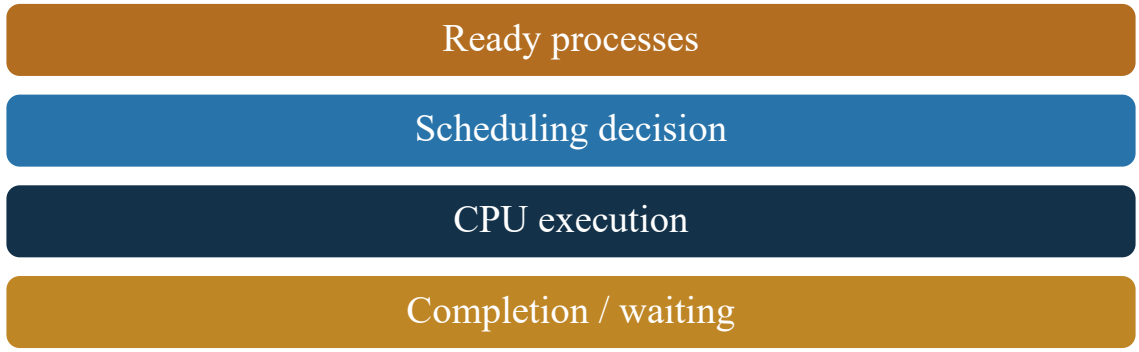

Processes access a common region and must coordinate their access.

CHAPTER 4

# CPU SCHEDULNG

VISUAL EXPLANATION

## Chapter at a glance




Use this route to see how the chapter topics connect.

CPU scheduling is the basis of multiprogrammed operating systems. By switching the CPU among processes, the **operating system** can make the computer more productive. In a single-processor system, only one process can run at a time. Others must wait until the CPU is free and can be rescheduled.

The objective of multiprogramming is to have some process running at all times, to maximize **CPU utilization**. The idea is relatively simple. A process is executed until it must wait, typically for the completion of some I/O request.

In a simple computer system, the CPU then just sits idle. All this **waiting time** is wasted; no useful work is accomplished.

With multiprogramming, we try to use this time productively. Several processes are kept in memory at one time. When one process

has to wait, the **operating system** takes the CPU away from that process and gives the CPU to another process. This pattern continues.

Every time one process has to wait, another process can take over use of the CPU. Scheduling of this kind is a fundamental operating-system function. Almost all computer resources are scheduled before use. The CPU is, of course, one of the primary computer resources. Thus, its scheduling is central to operating-system design.

## 4.1. Preemptive Scheduling

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the running state to the waiting state (for example, as the result of an I/O request or an invocation of wait() for the termination of a child process)
2. When a process switches from the running state to the ready state (for example, when an interrupt occurs)
3. When a process switches from the waiting state to the ready state (for example, at completion of I/O)
4. When a process terminates For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the **ready queue**) must be selected for execution. There is a choice, however, for situations 2 and 3.

When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is **nonpreemptive** or cooperative. Otherwise, it is **preemptive**.

Under **nonpreemptive** scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.

This scheduling method was used by Microsoft Windows 3.x. Windows 95 introduced **preemptive** scheduling, and all subsequent versions of Windows operating systems have used **preemptive** scheduling.

The Mac OS X **operating system** for the Macintosh also uses **preemptive** scheduling; previous versions of the Macintosh operating system relied on cooperative scheduling.

## 4.2. Scheduling Criteria

Different CPU-scheduling algorithms have different properties, and the choice of a particular algorithm may favor one class of processes over another. In choosing which algorithm to use in a particular situation, we must consider the properties of the various algorithms.

Many criteria have been suggested for comparing CPU- scheduling algorithms. Which characteristics are used for comparison can make a substantial difference in which algorithm is judged to be best. The criteria include the following:

- **CPU utilization**: We want to keep the CPU as busy as possible. Conceptually, **CPU utilization** can range from 0 to 100 percent. In

a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily loaded system).

- **Throughput**: If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called **throughput**. For long processes, this rate may be one process per hour; for short transactions, it may be ten processes per second.

- **Turnaround time**: From the point of view of a particular process, the important criterion is how long it takes to execute that process. The interval from the time of submission of a process to the time of completion is the **turnaround time**.

  **Turnaround time** is the sum of the periods spent waiting to get into memory, waiting in the **ready queue**, executing on the CPU, and doing I/O.

- **Waiting time**: The CPU-scheduling algorithm does not affect the amount of time during which a process executes or does I/O. It affects only the amount of time that a process spends waiting in the **ready queue**. Waiting time is the sum of the periods spent waiting in the ready queue.

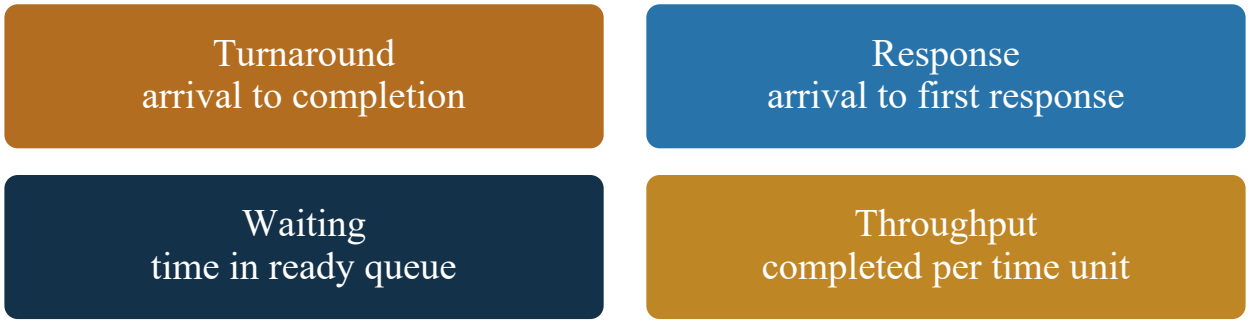
- **Response time:** In an interactive system, **turnaround time** may not be the best criterion.

Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is the time from the submission of a request until the first response is produced.

This measure, called **response time**, is the time it takes to start responding, not the time it takes to output the response. The **turnaround time** is generally limited by the speed of the output device.

VISUAL EXPLANATION

## Scheduling measurements




Use the definitions in this chapter to distinguish the scheduling criteria.

EXPLORE THE CONCEPT

## Three times to distinguish

| | |
|---|---|
| Turnaround | completion - arrival |
| Response | first run - arrival |
| Waiting | total in ready queue |

Definitions use arrival, first execution, completion, and time spent in the ready queue.

## 4.3. Scheduling Algorithms

CPU scheduling deals with the problem of deciding which of the processes in the **ready queue** is to be allocated the CPU. There are many different CPU-scheduling algorithms. In this section, we describe several of them.

### 4.3.1. First-Come, First-Served Scheduling

By far the simplest CPU-scheduling algorithm is the first-come, first-served (FCFS) scheduling algorithm. With this scheme, the process that requests the CPU first is allocated the CPU first. The implementation of the FCFS policy is easily managed with a FIFO queue.

When a process enters the **ready queue**, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The running process is then removed from the queue.

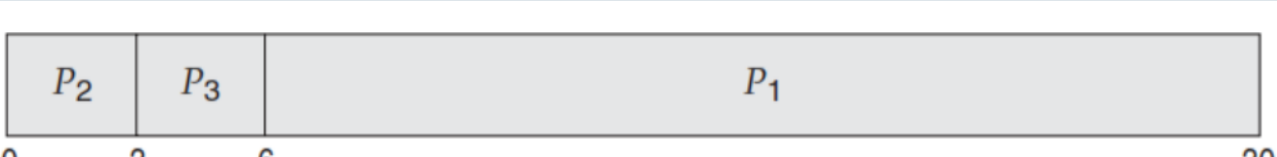
On the negative side, the average **waiting time** under the FCFS policy is often quite long. Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

| Process | Execution Time |
|---|---|
| $P_1$ | 24 |
| $P_2$ | 3 |
| $P_3$ | 3 |

The processes arrive in the order *P1*, *P2*, *P3* and are served in FCFS order, we get the result shown in the following Gantt chart, which is a bar chart that illustrates a particular schedule, including the start and finish times of each of the participating processes:




**Waiting time** is 0 milliseconds for process *P1*, 24 milliseconds for process *P2*, and 27 milliseconds for process *P3*. Thus, the average **waiting time** is (0 + 24 + 27)/3 = 17 milliseconds. If the processes arrive in the order *P1*, *P2*, *P3* however, the results will be as shown in the following Gantt chart:

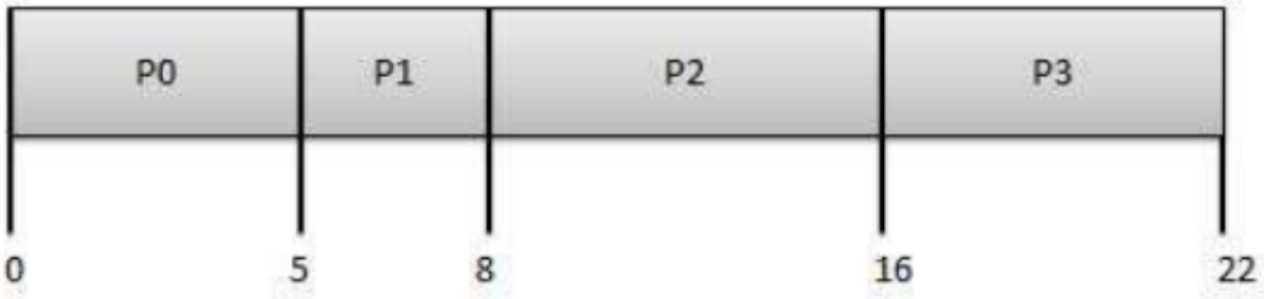



The average **waiting time** is now (6 + 0 + 3)/3 = 3 milliseconds. This reduction is substantial. Thus, the average **waiting time** under an FCFS policy is generally not minimal and may vary substantially if the processes' CPU times vary greatly.

Another example, assume the following processes:

| Process | Arrival Time | Execute Time |
|---|---|---|
| P0 | 0 | 5 |
| P1 | 1 | 3 |
| P2 | 2 | 8 |
| P3 | 3 | 6 |

The Gantt chart for the service time for the processes as follows:




Wait time of each process is following:

| Process | Wait Time : Service Time - Arrival Time |
|---|---|
| P0 | 0 - 0 = 0 |
| P1 | 5 - 1 = 4 |
| P2 | 8 - 2 = 6 |
| P3 | 16 - 3 = 13 |

Average Wait Time: (0 + 4 + 6 + 13) / 4 = 5.55

VISUAL EXPLANATION

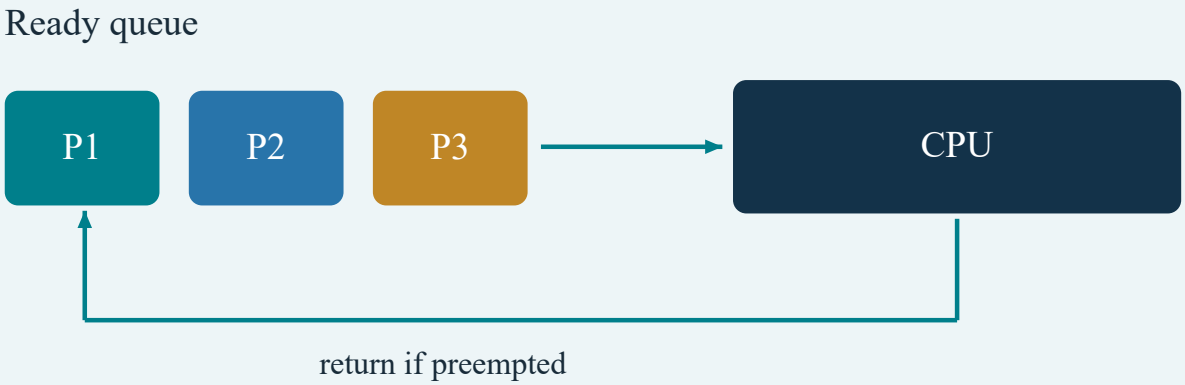
## FCFS: arrival order matters




Illustrative example, worked visual example: each process runs in arrival order.

EXPLORE THE CONCEPT

## FCFS follows arrival order



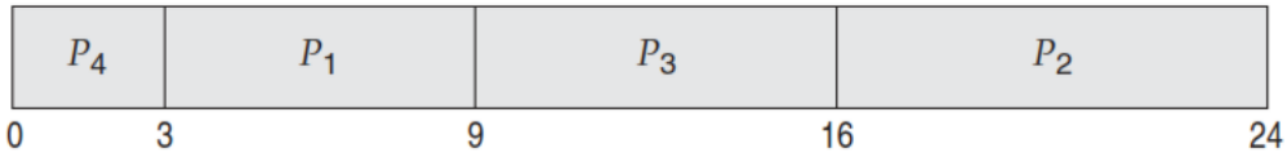

The process at the front of the ready queue receives the CPU first.

# 4.3.2. Shortest-Job-First Scheduling

When the CPU is available, it is assigned to the process that has the smallest next process execution time. If the execution time of two processes is the same, FCFS scheduling is used to break the tie.

| Process | Execution Time |
|---|---|
| $P_1$ | 6 |
| $P_2$ | 8 |
| $P_3$ | 7 |
| $P_4$ | 3 |

Using SJF scheduling, we would schedule these processes according to the following Gantt chart:

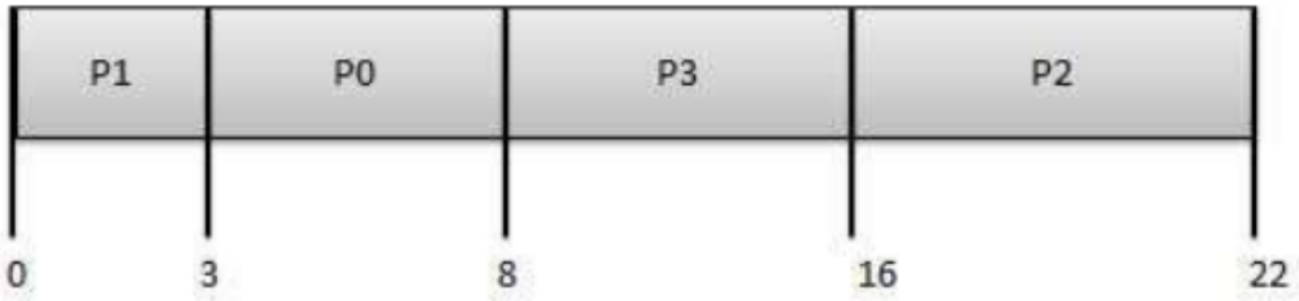



The **waiting time** is 3 milliseconds for process *P1*, 16 milliseconds for process *P2*, 9 milliseconds for process *P3*, and 0 milliseconds for process *P4*. Thus, the average **waiting time** is (3 + 16 + 9 + 0)/4 = 7 milliseconds. By comparison, if we were using the FCFS scheduling scheme, the average waiting time would be 10.25 milliseconds.

Another example on SJF, assume the following processes:

| Process | Arrival Time | Execute Time |
|---|---|---|
| P0 | 0 | 5 |
| P1 | 1 | 3 |
| P2 | 2 | 8 |
| P3 | 3 | 6 |

The Gantt chart for the service time for the processes as follows:




Wait time of each process is following

| Process | Wait Time : Service Time - Arrival Time |
|---|---|
| P0 | 3 - 0 = 3 |
| P1 | 0 - 0 = 0 |
| P2 | 16 - 2 = 14 |
| P3 | 8 - 3 = 5 |

> Average Wait Time: (3 + 0 + 14 + 5) / 4 = 5.50

The SJF scheduling algorithm is provably optimal, in that it gives the minimum average **waiting time** for a given set of processes. Moving a short process before long one decrease the **waiting time** of the short process more than it increases the waiting time of the long process. Consequently, the average waiting time decreases.

The real difficulty with the SJF algorithm knows the length of the next CPU request. For long-term (job) scheduling in a batch system,

we can use the process time limit that a user specifies when he submits the job.

Although the SJF algorithm is optimal, it cannot be implemented at the level of short-term CPU scheduling. With short-term scheduling, there is no way to know the length of the next process execution time.

One approach to this problem is to try to approximate SJF scheduling. We may not know the length of the next process execution time, but we may be able to predict its value. We expect that the process execution time will be similar in length to the previous ones.

By computing an approximation of the length of the next process execution time, we can pick the process with the shortest predicted process execution time.

VISUAL EXPLANATION

**SJF: shortest available job first**




Illustrative nonpreemptive order when all three jobs arrive together; ties follow FCFS.

## 4.3.3. Priority Scheduling

Apriority is associated with each process, and the CPU is allocated to the process with the highest **priority**. Equal- **priority** processes are scheduled in FCFS order.

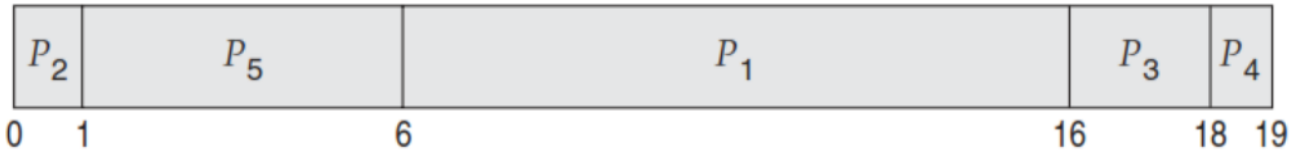
An SJF algorithm is simply a **priority** algorithm where the **priority** ($p$) is the inverse of the next process execution time. The larger the CPU burst, the lower the priority, and vice versa.

Priorities are generally indicated by some fixed range of numbers, such as 0 to 7 or 0 to 4,095. However, there is no general agreement on whether 0 is the highest or lowest **priority**. Some systems use low numbers to represent low **priority**; others use low numbers for high priority.

As an example, consider the following set of processes, assumed to have arrived at time 0 in the order P1, P2, ⋯, P5 (we assume that low numbers represent high **priority**):

| Process | Execution Time | Priority |
|---|---|---|
| $P_1$ | 10 | 3 |
| $P_2$ | 1 | 1 |
| $P_3$ | 2 | 4 |
| $P_4$ | 1 | 5 |
| $P_5$ | 5 | 2 |

Using **priority** scheduling, we would schedule these processes according to the following Gantt chart:



The average **waiting time** is 8.2 milliseconds. Priorities can be defined either internally or externally. Internally defined priorities use some measurable quantity or quantities to compute the **priority** of a process.

For example, time limits, memory requirements, and the number of open files, have been used in computing priorities.

External priorities are set by criteria outside the **operating system**, such as the importance of the process, the type and amount of funds being paid for computer use, the department sponsoring the work, and other, often political, factors.

A major problem with **priority** scheduling algorithms is leaving some low **priority** processes waiting indefinitely. Generally, one of two things will happen.

Either the process will eventually be run or the computer system will eventually crash and lose all unfinished low-**priority** processes. A solution to the problem is aging;

Aging involves gradually increasing the **priority** of processes that wait in the system for a long time. For example, if priorities range from 127 (low) to 0 (high), we could increase the **priority** of a waiting process by 1 every 15 minutes.

VISUAL EXPLANATION

### Priority and aging




Aging helps a long-waiting process gain an opportunity to execute.

## 4.3.4. Round-Robin Scheduling

The round-robin (RR) scheduling algorithm is designed especially for timesharing systems. It is similar to FCFS scheduling, but preemption is added to enable the system to switch between processes. A small unit of time, called a **time quantum** or time slice, is defined.

A **time quantum** is generally from 10 to 100 milliseconds in length. The **ready queue** is treated as a circular queue.

The CPU scheduler goes around the **ready queue**, allocating the CPU to each process for a time interval of up to 1 **time quantum**. To implement RR scheduling, we again treat the ready queue as a FIFO queue of processes. New processes are added to the tail of the ready queue.

The CPU scheduler picks the first process from the **ready queue**, sets a timer to interrupt after 1 **time quantum**, and dispatches the process.

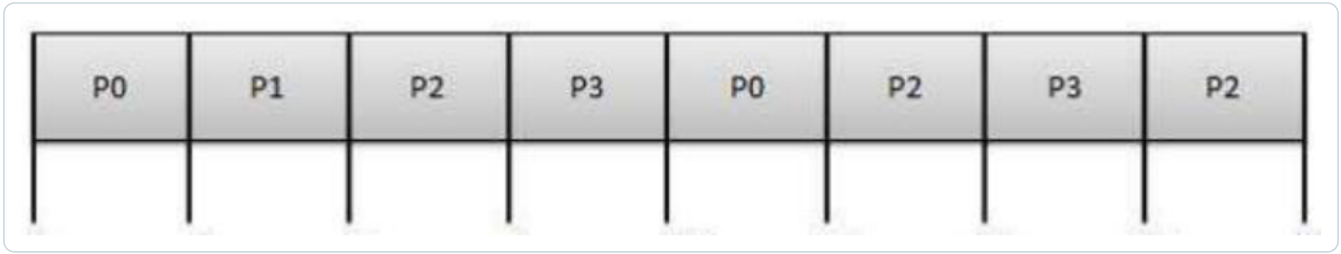
One of two things will then happen. The process may have a process execution time of less than 1 **time quantum**. In this case, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the **ready queue**.

If the process execution time of the currently running process is longer than 1 **time quantum**, the timer will go off and will cause an interrupt to the **operating system**. A context switch will be executed, and the process will be put at the tail of the ready queue. The CPU scheduler will then select the next process in the ready queue.

The average **waiting time** under the RR policy is often long. Consider the following set of processes that arrive at time 0, with the length of the process execution time given in milliseconds:

| Process | Arrival Time | Execute Time |
|---|---|---|
| P0 | 0 | 5 |
| P1 | 1 | 3 |
| P2 | 2 | 8 |
| P3 | 3 | 6 |

Quantum = 3.



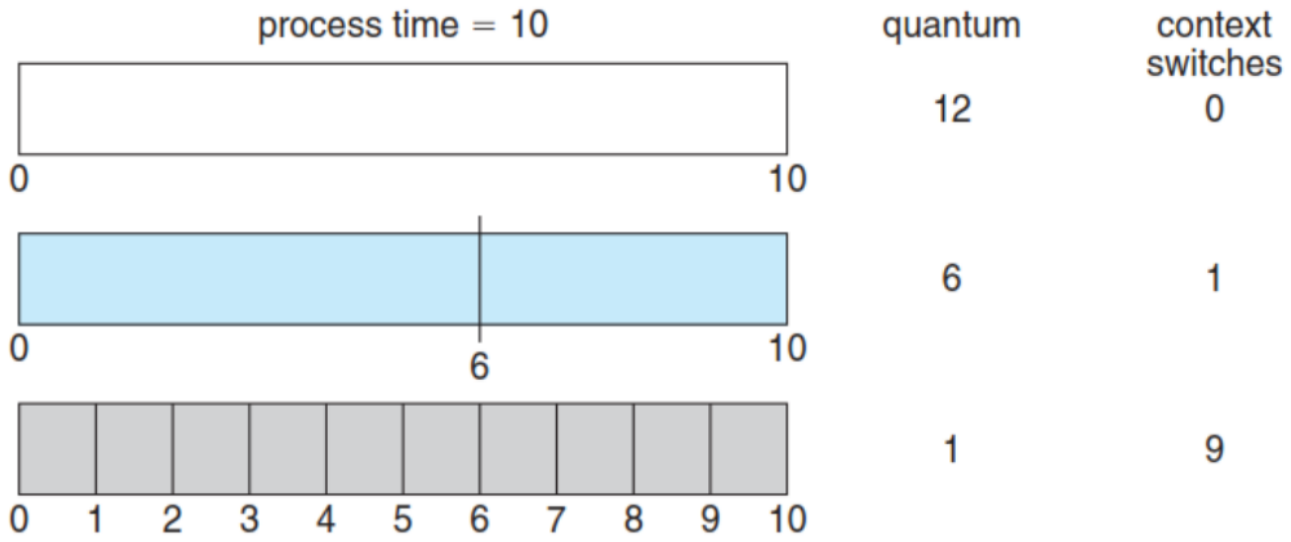
| Process | Waiting time |
|---|---|
| P0 | (0 - 0) + (12 - 3) = 9 |
| P1 | (3 -1) = 2 |
| P2 | (6 - 2) + (14 - 6) + (20 - 17) = 15 |
| P3 | (9 - 3) + (17 - 12) = 11 |
| | Average waiting time = (9 + 2 + 15 + 11)/ 4= 6.75 |

The performance of the RR algorithm depends heavily on the size of the **time quantum**. At one extreme, if the **time quantum** is extremely large, the RR policy is the same as the FCFS policy.

In contrast, if the **time quantum** is extremely small (say, 1 millisecond), the RR approach can result in a large number of context switches. Assume, for example, that we have only one process of 10 time units.

If the quantum is 12 time units, the process finishes in less than 1 **time quantum**, with no overhead. If the quantum is 6 time units, however, the process requires 2 quanta, resulting in a **context switch**.

If the **time quantum** is 1 time unit, then nine context switches will occur, slowing the execution of the process accordingly as shown inthe following figure.




VISUAL EXPLANATION

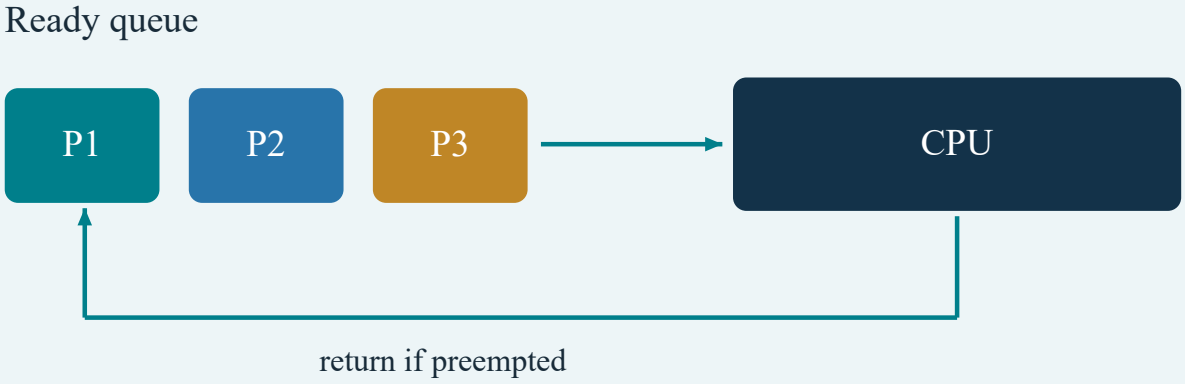
## Round robin shares CPU time




Illustrative quantum of 2 units; unfinished processes return to the ready queue.

EXPLORE THE CONCEPT

## Round robin returns unfinished work




When a time quantum expires, an unfinished process returns to the ready queue.

EXPLORE THE CONCEPT

### A short time slice causes more switches




Time progresses from left to right

One 8-unit job with a 2-unit quantum uses four slices; switches add overhead.

## 4.3.5. Multilevel Queue Scheduling

A multilevel queue scheduling algorithm partitions the **ready queue** into several separate queues. The processes are permanently assigned to one queue, generally based on some property of the process, such as memory size, process **priority**, or process type.

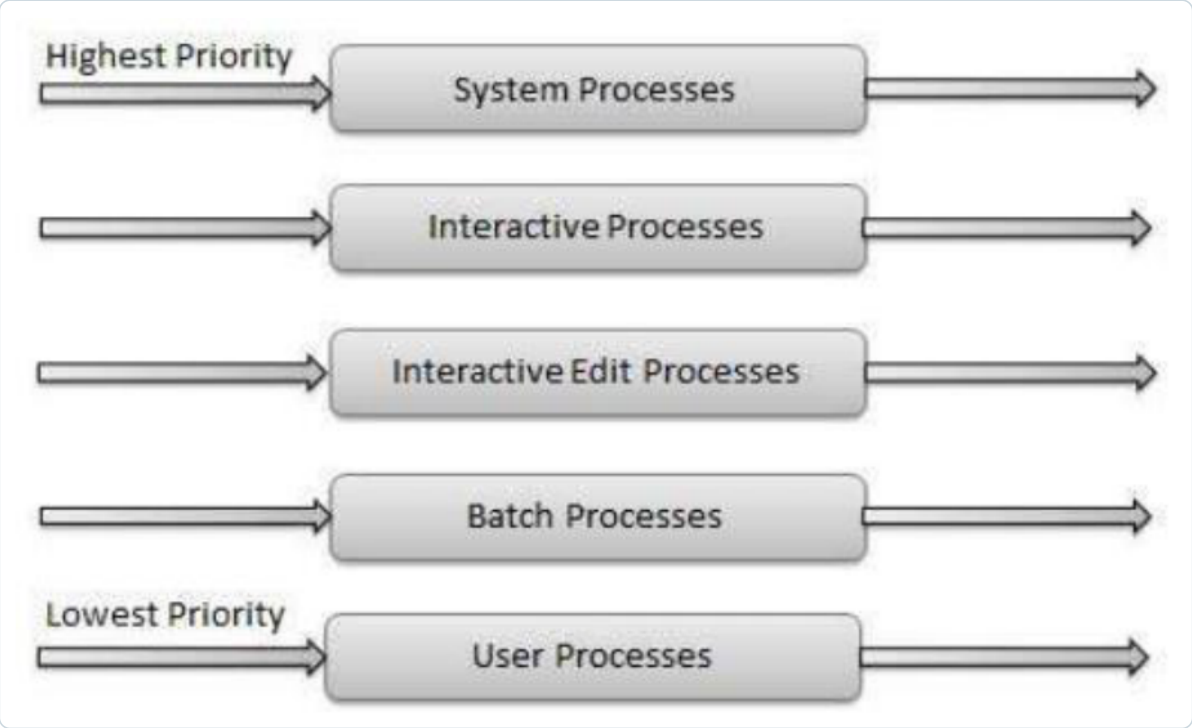
Each queue has its own scheduling algorithm. There must be scheduling among the queues, which is commonly implemented as fixed-**priority preemptive** scheduling.

Let’s look at an example of a multilevel queue scheduling algorithm with five queues, listed below in order of **priority**:

1. System processes
2. Interactive processes
3. Interactive editing processes

4. Batch processes

5. User processes




Each queue has absolute **priority** over lower-**priority** queues. No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty.

If an interactive editing process entered the **ready queue** while a batch process was running, the batch process would be preempted.

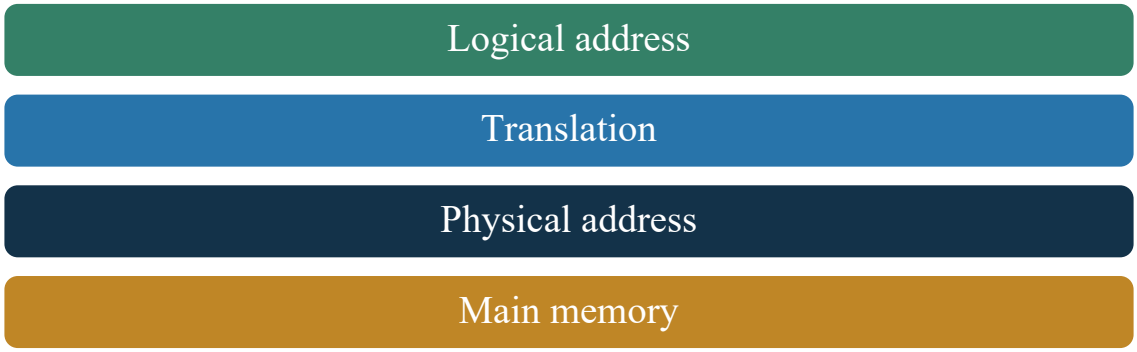
Another possibility is to time-slice among the queues. Here, each queue gets a certain portion of the CPU time, which it can then schedule among its various processes.

CHAPTER 5

# MEMORY MANEGEMENT

VISUAL EXPLANATION

## Chapter at a glance



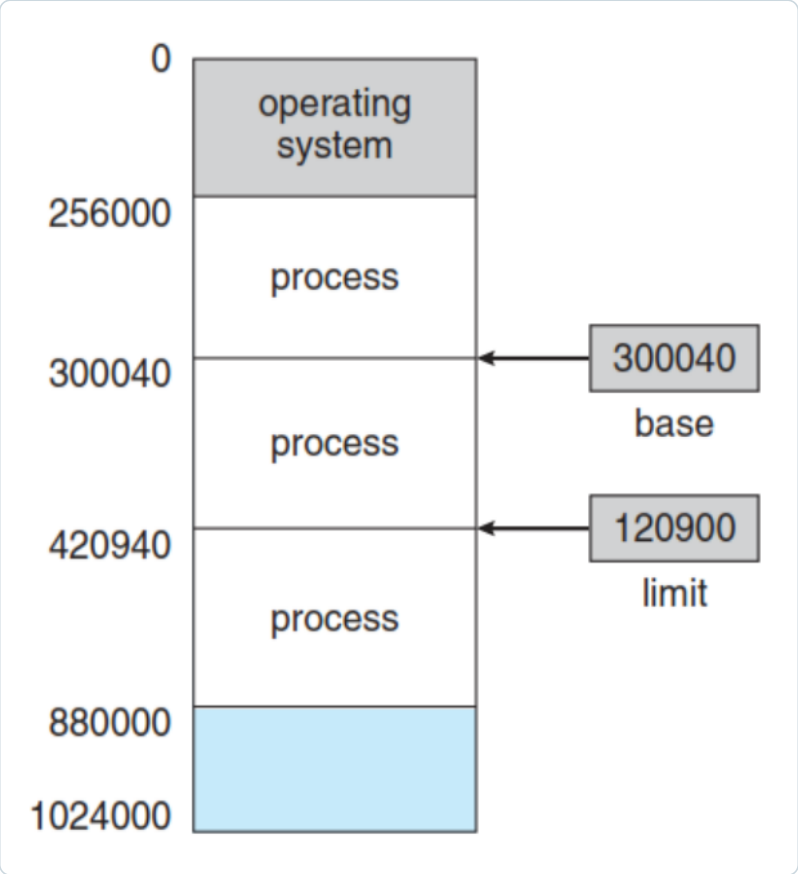

Use this route to see how the chapter topics connect.

Memory is central to the operation of a modern computer system. Memory consists of a large array of bytes, each with its own address. The CPU fetches instructions from memory according to the value of the program counter. These instructions may cause additional loading from and storing to specific memory addresses.

A typical instruction-execution cycle, for example, first fetches an instruction from memory. The instruction is then decoded and may cause operands to be fetched from memory. After the instruction has been executed on the operands, results may be stored back in memory.

We first need to make sure that each process has a separate memory space. Separate per-process memory space protects the processes from each other and is fundamental to having multiple processes loaded in memory for concurrent execution.

To separate memory spaces, we need the ability to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses. We can provide this protection by using two registers, usually a base and a limit, as illustrated in the following figure.




The base register holds the smallest legal physical memory address; the limit register specifies the size of the range. For example, if the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 300040 through 420939

EXPLORE THE CONCEPT

### Base and limit define a valid range

| | |
|---|---|
| Base | 300040 |
| Limit | 120900 |
| Last valid address | 420939 |

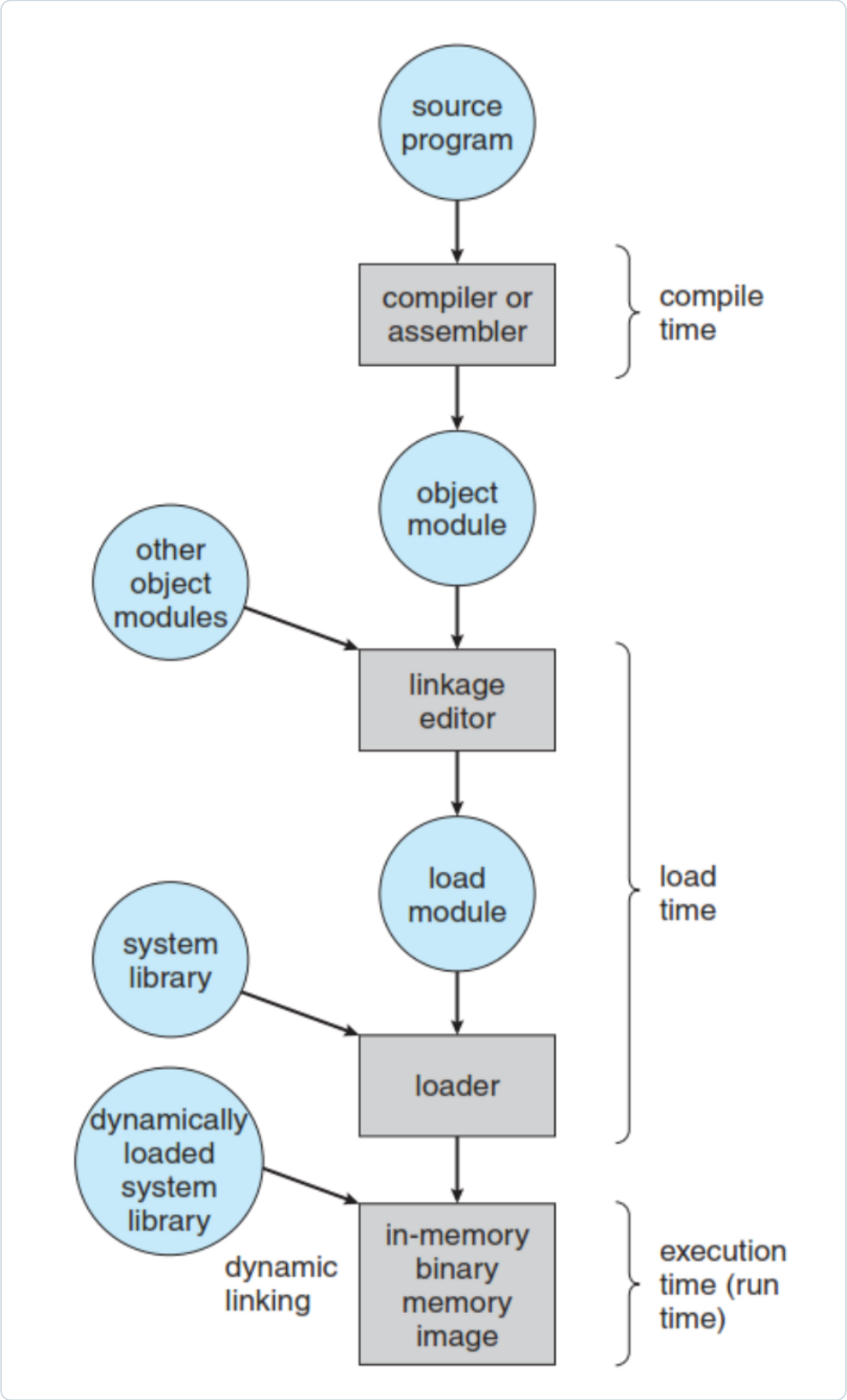
Worked address range: the last valid address is base + limit - 1.

## 5.1. Address Binding

Usually, a program resides on a disk as a binary executable file. To be executed, the program must be brought into memory and placed within a process. Depending on the **memory management** in use, the process may be moved between disk and memory during its execution.

The processes on the disk that are waiting to be brought into memory for execution form the input queue.

Most systems allow a user process to reside in any part of the physical memory. In most cases, a user program goes through several steps—some of which may be optional— before being executed as shown in the following figure. Addresses may be represented in different ways during these steps.




Classically, the binding of instructions and data to memory addresses can be done at any step along the way:

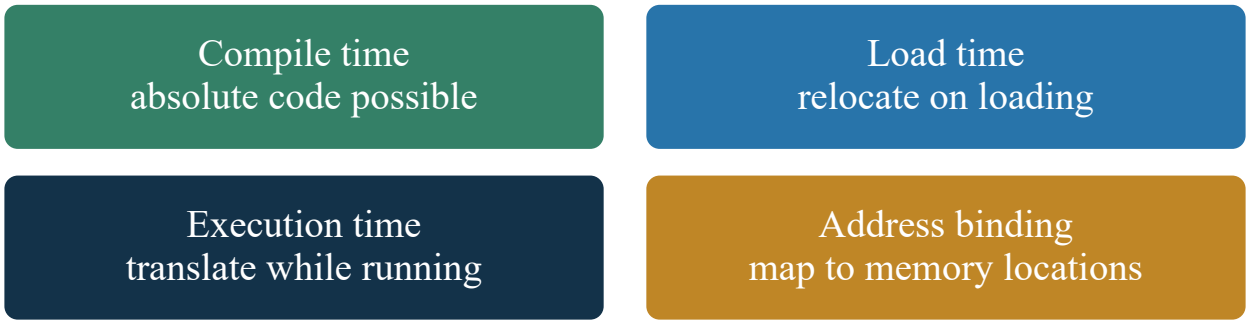
- **Compile time**: If you know at compile time where the process will reside in memory, then absolute code can be generated. For

example, if you know that a user process will reside starting at location R, then the generated compiler code will start at that location and extend up from there.

- **Load time**: If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code. In this case, final binding is delayed until load time. If the starting addresses changes, we need only reload the user code to incorporate this changed value.
- **Execution time**: If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

VISUAL EXPLANATION

**When addresses are bound**




Binding can occur at different stages, as described in this section.

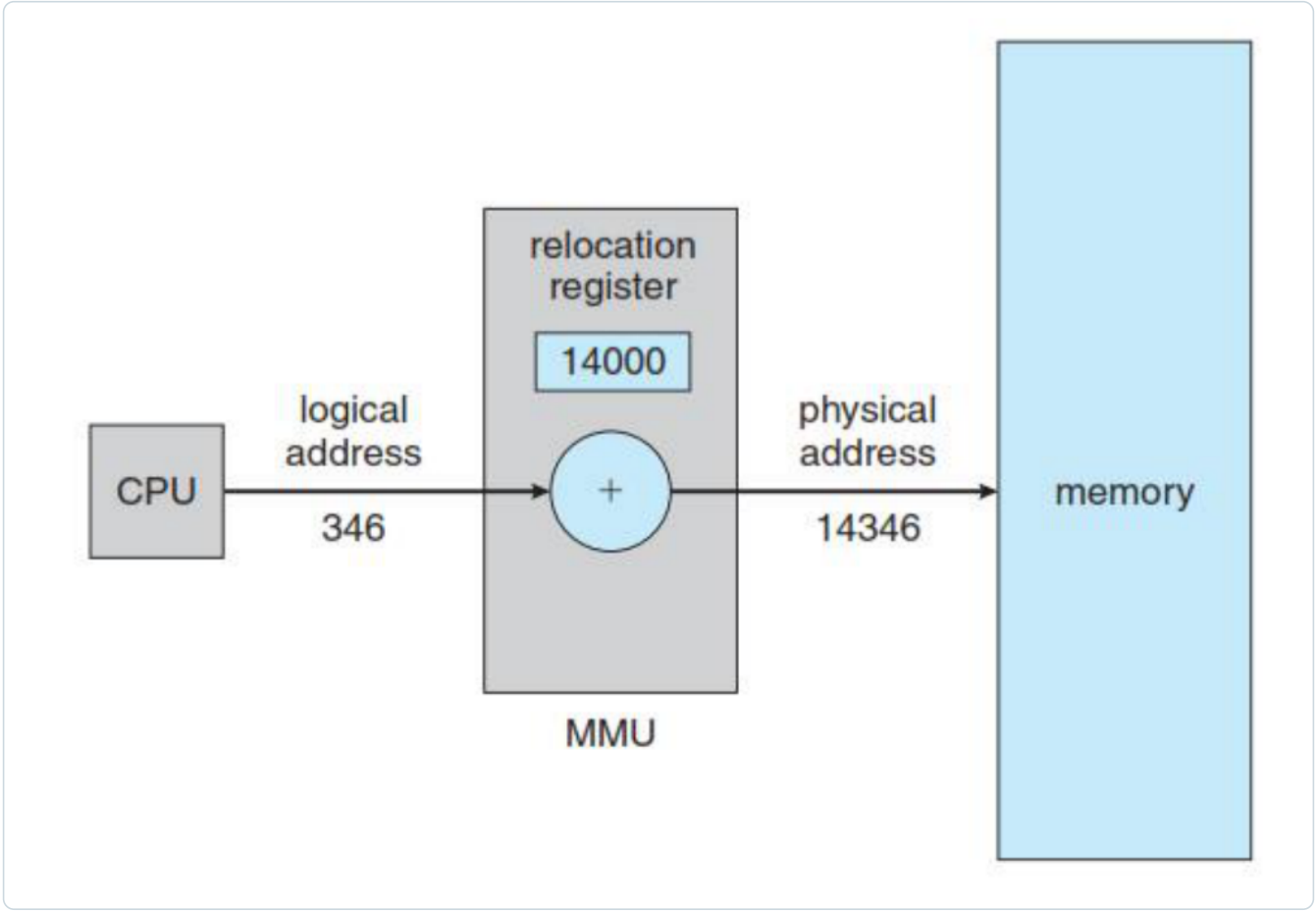
## 5.2. Logical Versus Physical Address Space

An address generated by the CPU is commonly referred to as a **logical address**, whereas an address seen by the memory unit—that is, the one loaded into the memory- address register of the memory—is commonly referred to as a **physical address**.

The run-time mapping from virtual to physical addresses is done by a hardware device called the memory-management unit (MMU). The base register is now called a relocation register.

The value in the relocation register is added to every address generated by a user process at the time the address is sent to memory as shown in the following figure.

For example, if the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000; an access to location 346 is mapped to location 14346.




We now have two different types of addresses: logical addresses (in the range 0 to max) and physical addresses (in the range R +0 to R + max for a base value R). The user program generates only logical addresses and thinks that the process runs in locations 0 to max.

However, these logical addresses must be mapped to physical addresses before they are used.

## 5.3. Dynamic Loading

To obtain better memory-space utilization, we can use dynamic loading. With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format. The main program is loaded into memory and is executed.

When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded.

If it has not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.

The advantage of dynamic loading is that a routine is loaded only when it is needed. Dynamic loading does not require special support from the **operating system**.

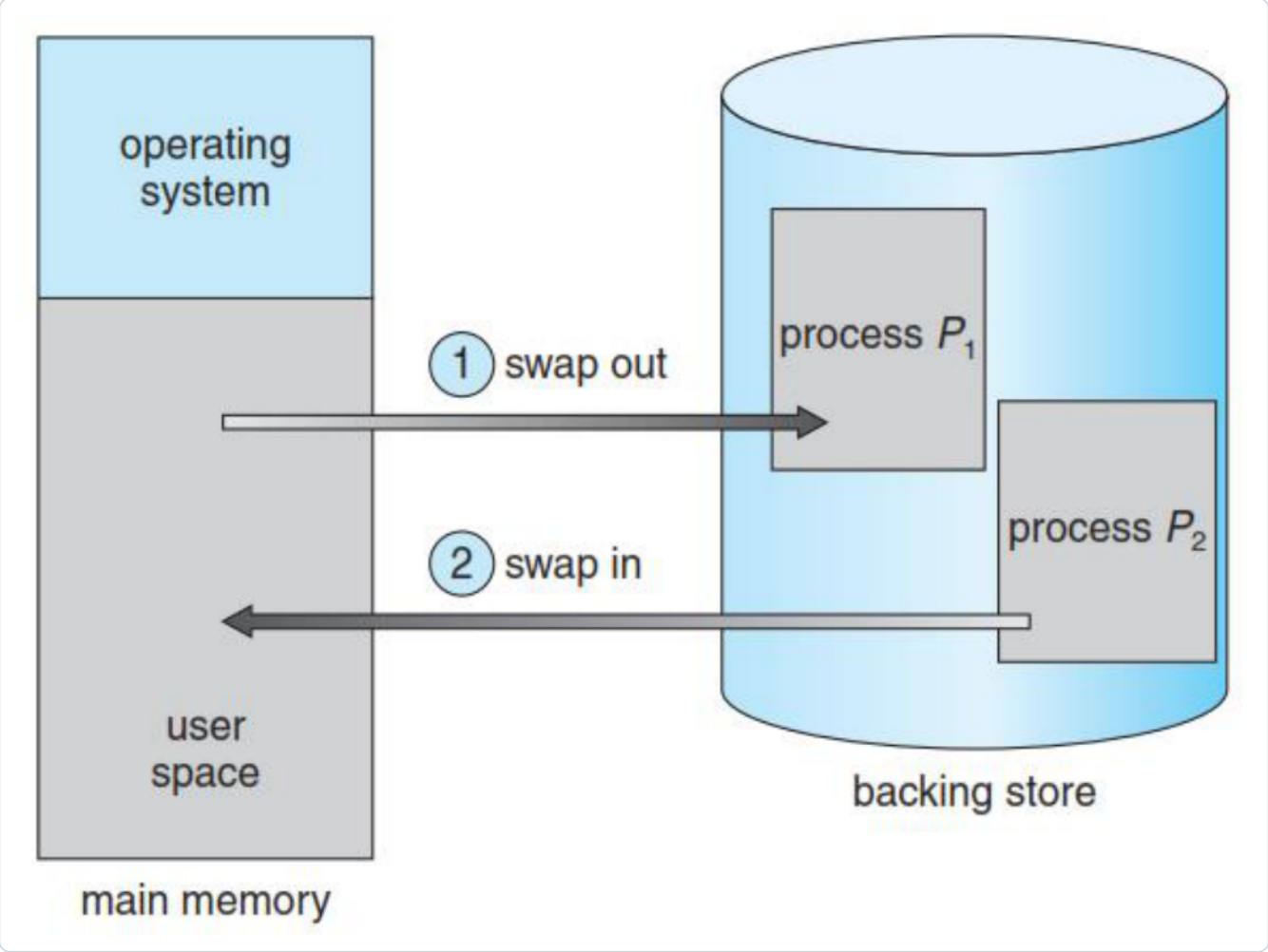
## 5.4. Swapping

> **Swapping** is a mechanism in which a process can be swapped temporarily out of **main memory** to a backing store, and then brought back into memory for continued execution.

Backing store is a usually a hard disk drive or any other secondary storage which fast in access and large enough to accommodate copies

of all memory images for all users. It must be capable of providing direct access to these memory images.

Major time consuming part of **swapping** is transfer time. Total transfer time is directly proportional to the amount of memory swapped.




Let's assume that the user process is 100 MB in size and the backing store is a standard hard disk with a transfer rate of 50 MB per second. The actual transfer of the 100-MB process to or from **main memory** takes

100 MB/50 MB per second = 2 seconds.

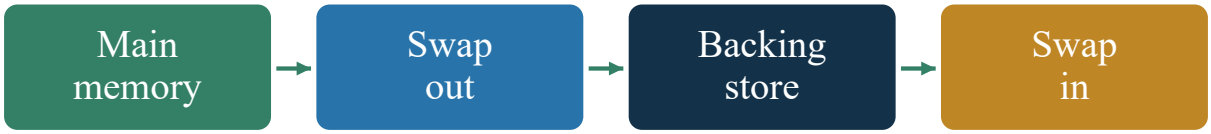
The swap time is 200 milliseconds. Since we must swap both out and in, the total swap time is about 4,000 milliseconds. If we have a computer system with 4 GB of **main memory** and a resident **operating system** taking 1 GB, the maximum size of the user process is 3 GB.

However, many user processes may be much smaller than this—say, 100 MB. A 100-MB process could be swapped out in 2 seconds, compared with the 60 seconds required for **swapping** 3 GB.

Clearly, it would be useful to know exactly how much memory a user process is using, not simply how much it might be using. Then we would need to swap only what is actually used, reducing swap time.

VISUAL EXPLANATION

**Swapping moves a process image**




Conceptual movement between main memory and backing store; memory is released while swapped out.

## 5.4. Memory Allocation

One of the simplest methods for allocating memory is to divide memory into several fixed-sized partitions. Each partition may contain exactly one process. Thus, the degree of multiprogramming is bound by the number of partitions.

In this multiplepartition method, when a partition is free, a process is selected from the input queue and is loaded into the free partition.

When the process terminates, the partition becomes available for another process.

In the variable-partition scheme, the **operating system** keeps a table indicating which parts of memory are available and which are occupied. Initially, all memory is available for user processes and is considered one large block of available memory, a hole. Eventually, as you will see, memory contains a set of holes of various sizes.

In general, as mentioned, the memory blocks available comprise a set of holes of various sizes scattered throughout memory. When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process.

If the hole is too large, it is split into two parts. One part is allocated to the arriving process; the other is returned to the set of holes. When a process terminates, it releases its block of memory, which is then placed back in the set of holes.

If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole. At this point, the system may need to check whether there are processes waiting for memory and whether this newly freed and recombined memory could satisfy the demands of any of these waiting processes.

This procedure is a particular instance of the general dynamic storage allocation problem, which concerns how to satisfy a request of size n from a list of free holes. There are many solutions to this problem.

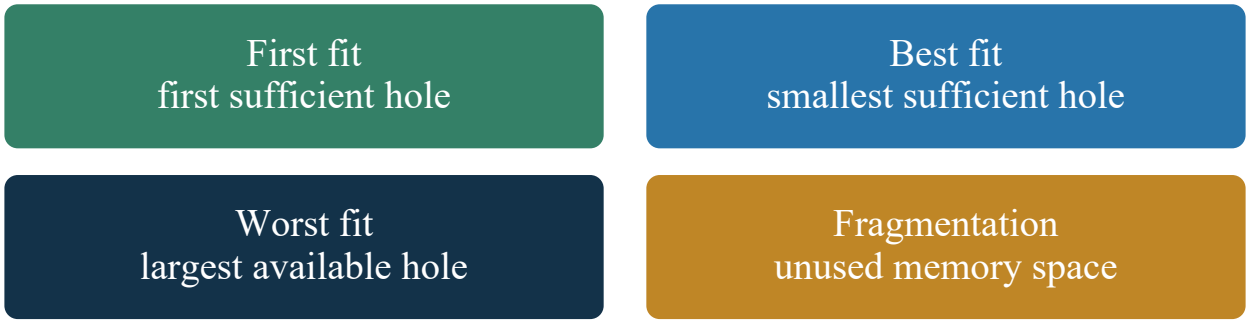
The first-fit, best-fit, and worst-fit strategies are the ones most commonly used to select a free hole from the set of available holes.

- First fit. Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.
- Best fit. Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.
- Worst fit. Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.

Simulations have shown that both first fit and best fit are better than worst fit in terms of decreasing time and storage utilization. Neither first fit nor best fit is clearly better than the other in terms of storage utilization, but first fit is generally faster.

VISUAL EXPLANATION

**Choosing a free memory hole**




These placement strategies choose among free holes during contiguous memory allocation.

## 5.5. Fragmentation

**Fragmentation** is a phenomenon in which storage space is used inefficiently, reducing capacity or performance and often both. There are two different but related forms of **fragmentation**: external fragmentation and internal fragmentation.

1. External **fragmentation** exists when there is enough total memory space to satisfy a request but the available spaces are not contiguous: storage is fragmented into a large number of small holes.

   That happen when the processes are loaded and removed from memory, the free memory space is broken into little pieces. If all these small pieces of memory were in one big free block instead, we might be able to run several more processes this solution called compaction.

2. Internal **fragmentation** exists when the memory allocated to a process may be slightly larger than the requested memory. Consider a multiple-partition allocation scheme with a hole of 18,464 bytes. Suppose that the next process requests 18,462 bytes.

   If we allocate exactly the requested block, we are left with a hole of 2 bytes. The overhead to keep track of this hole will be substantially larger than the hole itself. The general approach to avoiding this problem is to break the physical memory into fixed-sized blocks and allocate memory in units based on block size.

EXPLORE THE CONCEPT

### Free space can be fragmented




Allocated blocks Free holes

Free space exists in separate regions.

External fragmentation can leave enough total free memory spread across separate holes.

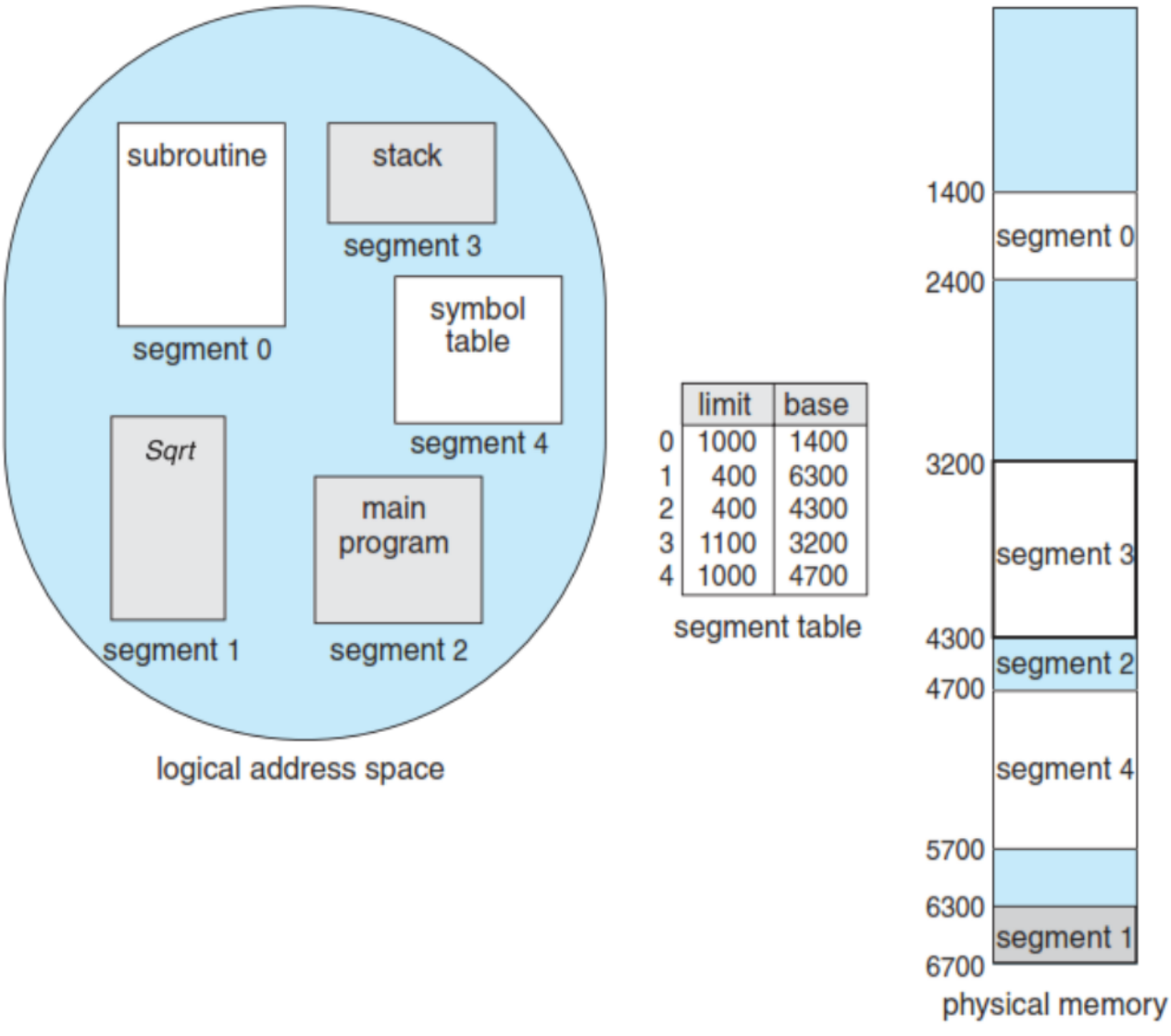
## 5.6. Segmentation

**Segmentation** is a memory-management scheme that supports this programmer view of memory. a programmer thinks of it as a main program with a set of methods, procedures, or functions. It may also include various data structures: objects, arrays, stacks, variables, and so on. Each of these modules or data elements is referred to by name.

Segments vary in length, and the length of each is intrinsically defined by its purpose in the program. Each segment has a name and a length. Segments are numbered and are referred to by a segment number, rather than by a segment name. Each segment has a segment base and a segment limit stored in segment table.

Consider the situation shown in the following figure. We have five segments numbered from 0 through 4. The segments are stored in physical memory as shown.

The segment table has a separate entry for each segment, giving the beginning address of the segment in physical memory (or base) and the length of that segment (or limit). For example, segment 2 is 400 bytes long and begins at location 4300.



VISUAL EXPLANATION

### Segmentation follows program structure




Segments are logical units of variable length; their table entries locate and bound them.

EXPLORE THE CONCEPT

### Segments match logical program units




Logical units may have different lengths.

A segment table stores a base and a limit for each segment.

Segmentation organizes memory around units such as code, data, and stack.

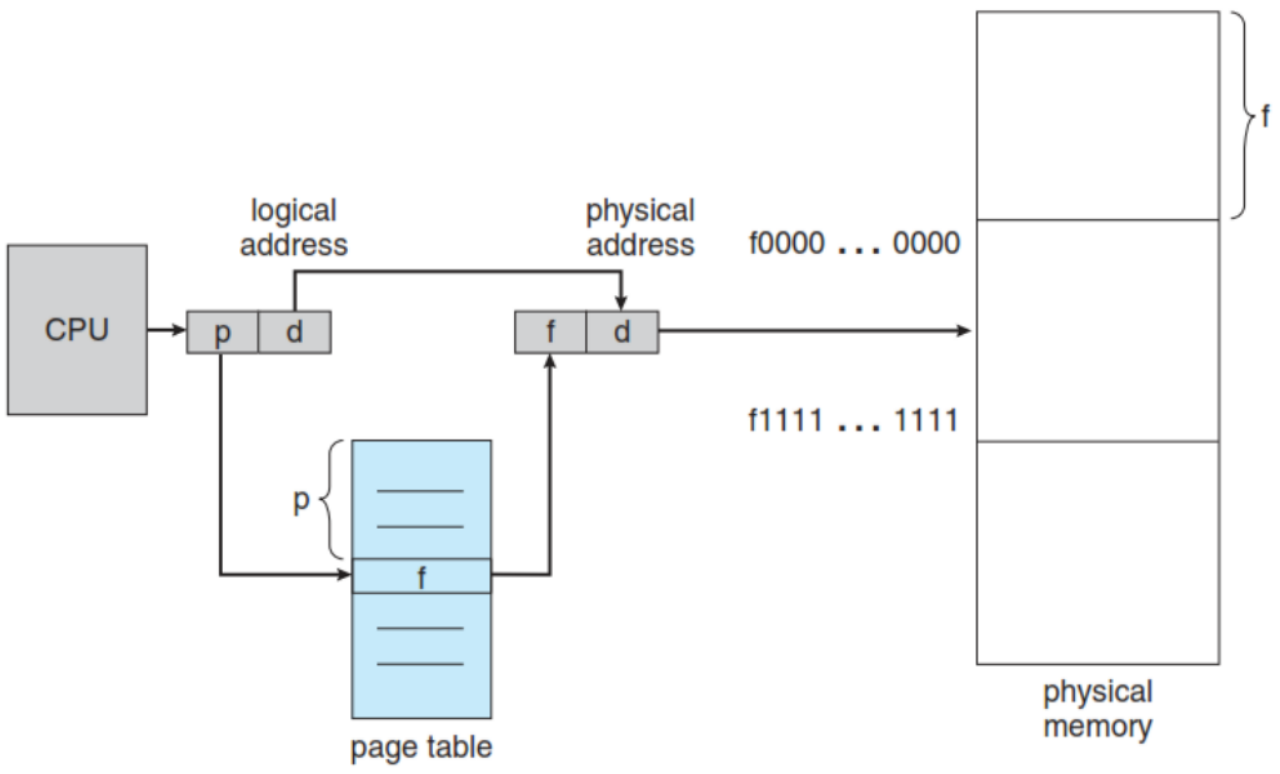
## 5.7. Paging

**Paging** is another memory-management scheme. The basic method for implementing **paging** involves breaking physical memory into fixed-sized blocks called frames and breaking logical memory into blocks of the same size called pages.

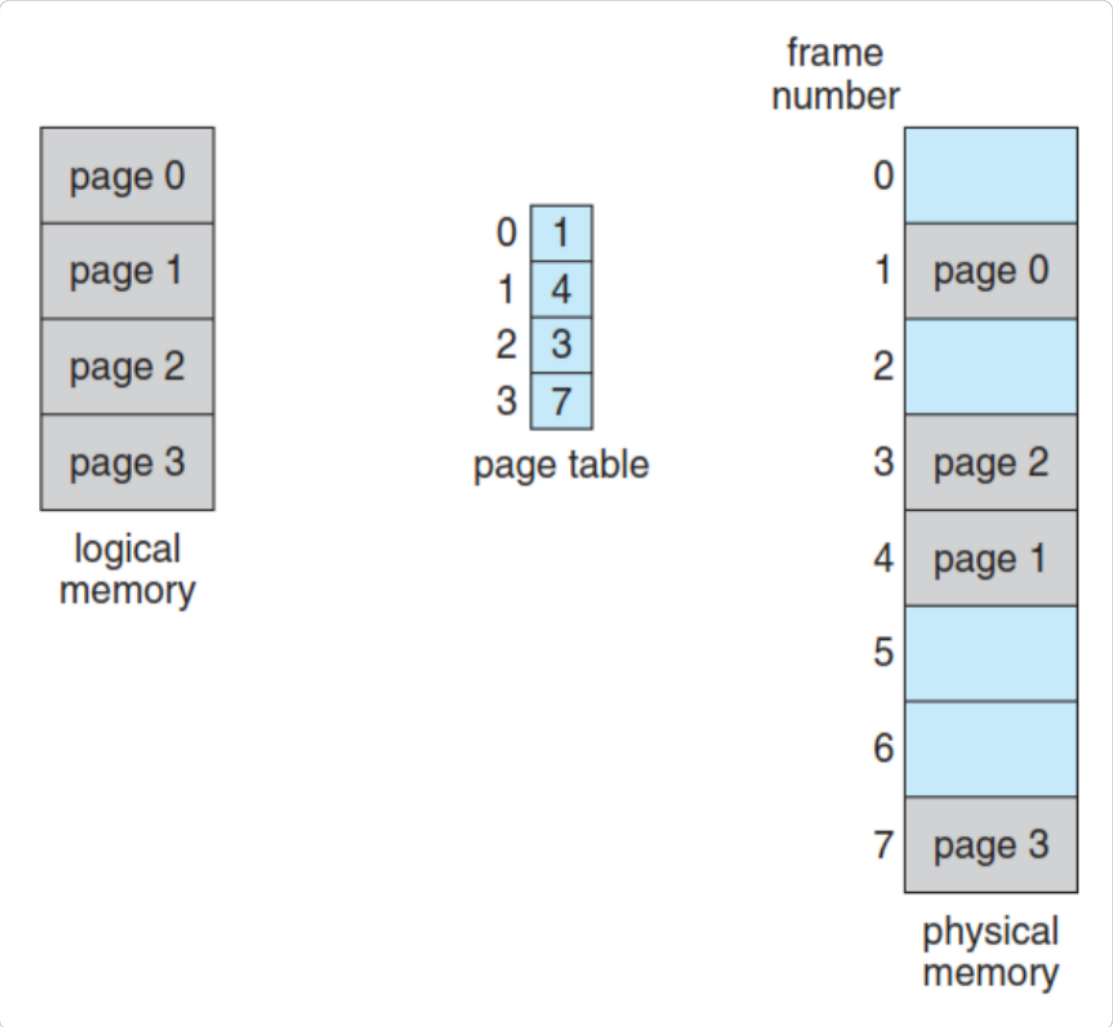
When a process is to be executed, its pages are loaded into any available memory frames from their source. The backing store is divided into fixed-sized blocks that are the same size as the memory frames or clusters of multiple frames.

Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d). The page number is used as an index into a **page table**. The **page table** contains the base address of each page in physical memory.

This base address is combined with the page offset to define the physical memory address that is sent to the memory unit as shown in the following figure.



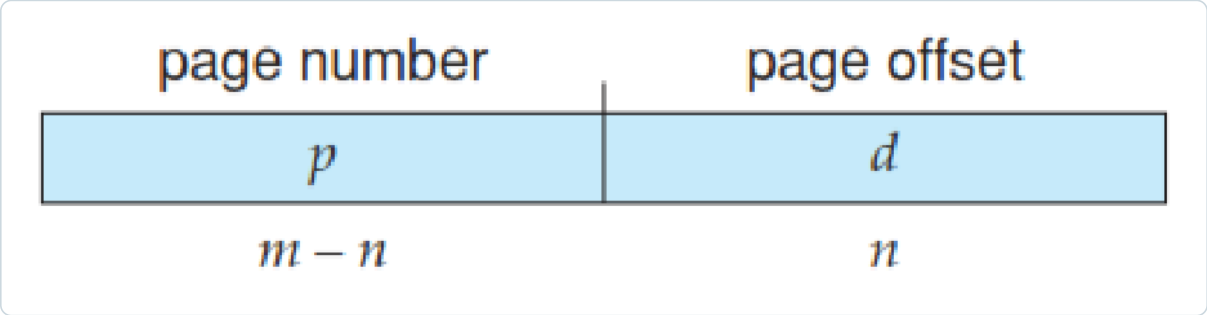

The **paging** model of memory is in the following figure.



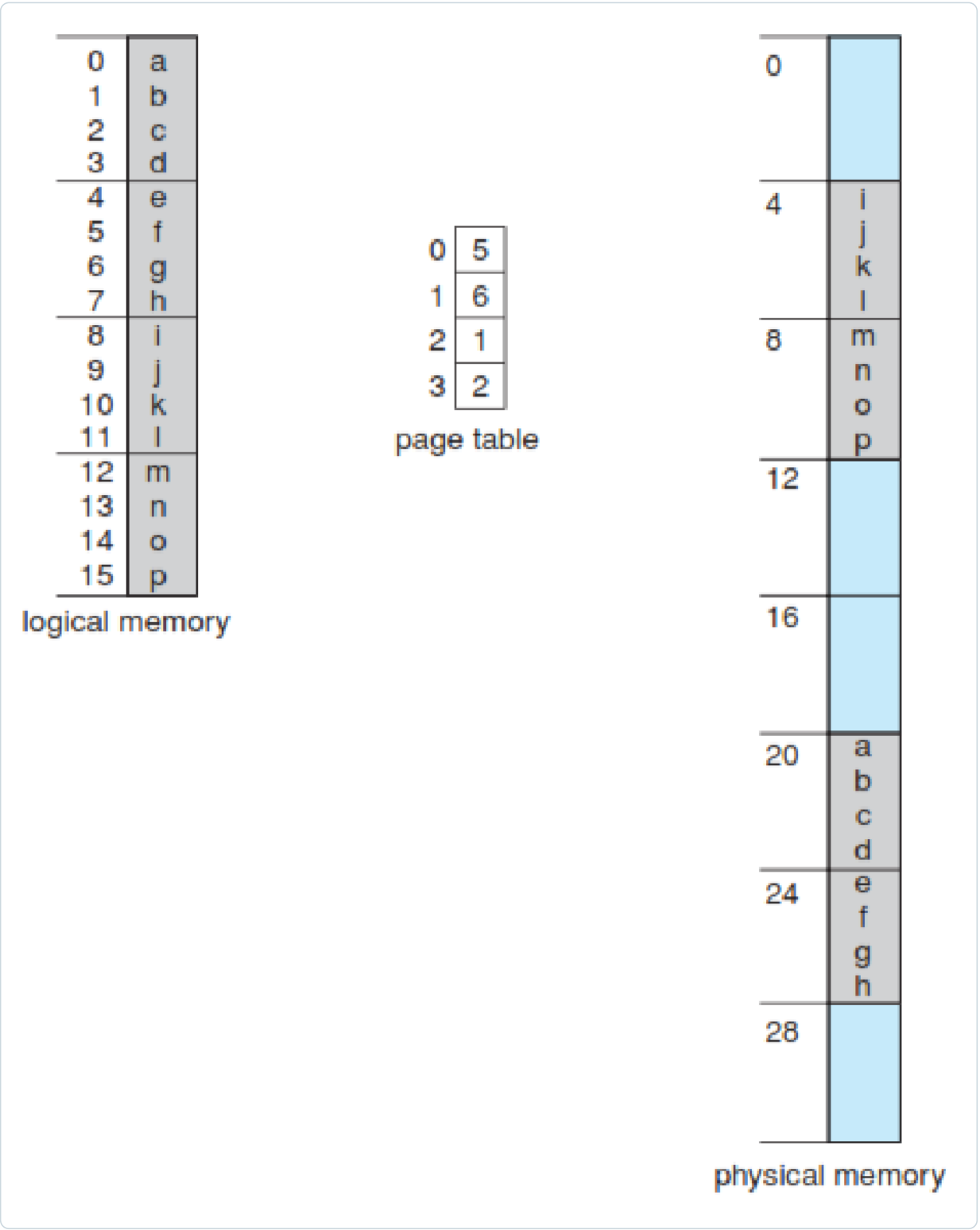

> The page size (like the frame size) is defined by the hardware. The size of a page is a power of 2, varying between 512 bytes and 1 GB per page, depending on the computer architecture.

The selection of a power of 2 as a page size makes the translation of a **logical address** into a page number and page offset particularly easy.

If the size of the **logical address** space is $2^m$, and a page size is $2^n$ bytes, then the high-order m - n bits of a **logical address** designate the page number, and the n low-order bits designate the page offset. Thus, the logical address is as follows:




Where p is an index into the **page table** and d is the displacement within the page. As a concrete (although minuscule) example, consider the memory in the following figure.




Here, in the **logical address**, n = 2 and m = 4. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages), we show how the programmer's view of memory can be mapped into physical memory. **Logical address** 0 is page 0, offset 0.

Indexing into the **page table**, we find that page 0 is in frame 5. Thus, **logical address** 0 maps to physical address 20 [= (5 × 4) + 0]. Logical address 3 (page 0, offset 3) maps to physical address 23 [= (5 × 4) + 3]. Logical address 4 is page 1, offset 0;

according to the **page table**, page 1 is mapped to frame 6. Thus, **logical address** 4 maps to physical address 24 [= (6 × 4) + 0]. Logical address 13 maps to physical address 9.

When we use a **paging** scheme, we have no external **fragmentation**: any free frame can be allocated to a process that needs it. However, we may have some internal fragmentation. Notice that frames are allocated as units.

If the memory requirements of a process do not happen to coincide with page boundaries, the last frame allocated may not be completely full. For example, if page size is 2,048 bytes, a process of 72,766 bytes will need 35 pages plus 1,086 bytes.

It will be allocated 36 frames, resulting in internal **fragmentation** of 2,048 − 1,086 = 962 bytes. In the worst case, a process would need n pages plus 1 byte. It would be allocated n + 1 frames, resulting in internal **fragmentation** of almost an entire frame.

If process size is independent of page size, we expect internal **fragmentation** to average one-half page per process. This consideration suggests that small page sizes are desirable.

However, overhead is involved in each page- table entry, and this overhead is reduced as the size of the pages increases. Also, disk I/O is more efficient when the amount data being transferred is larger.

Generally, page sizes have grown over time as processes, data sets, and **main memory** have become larger. Today, pages typically are between 4 KB and 8 KB in size and some systems support even larger page sizes. Some CPUs and kernels even support multiple page sizes.

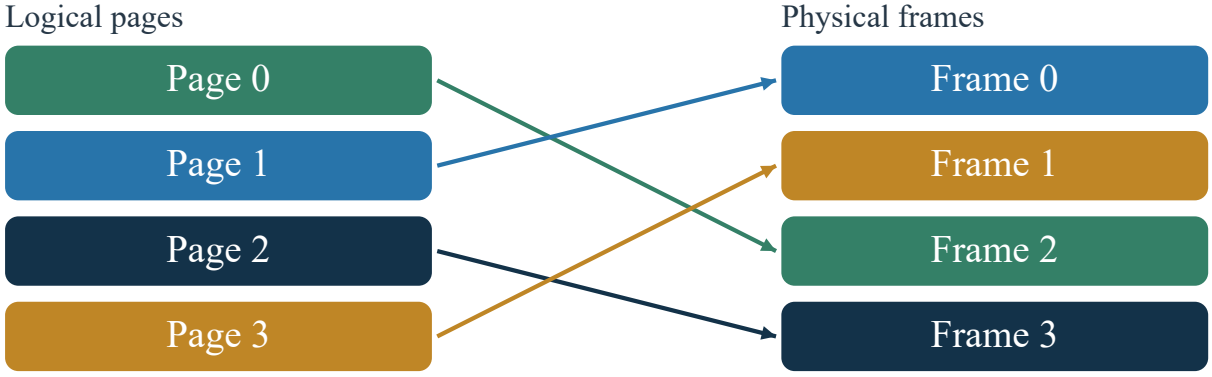
For instance, Solaris uses page sizes of 8 KB and 4 MB, depending on the data stored by the pages.

Researchers are now developing support for variable on- the-fly page size. Frequently, on a 32-bit CPU, each page- table entry is 4 bytes long, but that size can vary as well.

A32-bit entry can point to one of $2^{32}$ If frame size is 4 KB ($2^{12}$ physical page frames.), then a system with 4-byte entries can address 2 bytes (or 16 TB) of physical memory. We should note here that the size of physical memory in a paged memory system is different from the maximum logical size of a process.

VISUAL EXPLANATION

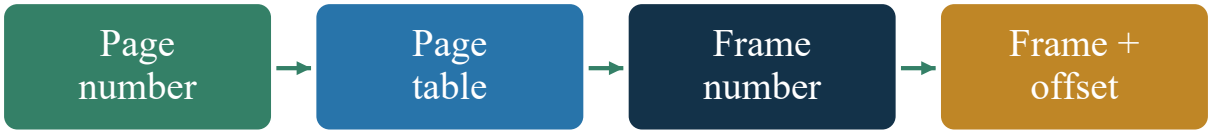
**Paging maps pages to frames**




Illustrative mapping: equal-size pages can occupy noncontiguous frames in physical memory.

VISUAL EXPLANATION

## Translate a paged address




The page number selects a table entry. The page offset is retained in the physical address.

EXPLORE THE CONCEPT

## A logical address has two parts



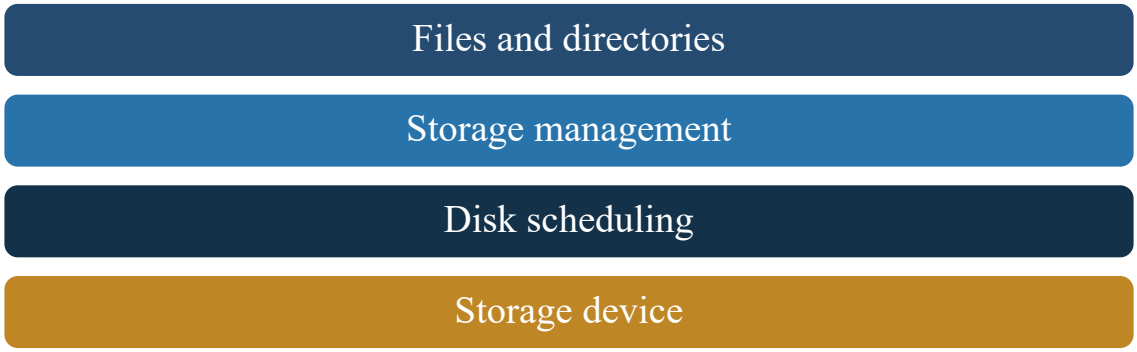

Paging splits an address into a page number and an offset within that page.

CHAPTER 6

# STORAGE MANEGEMENT

VISUAL EXPLANATION

## Chapter at a glance




Use this route to see how the chapter topics connect.

The computer system must provide secondary storage to back up **main memory**. Modern computer systems use disks as the primary on-line storage medium for information (both programs and data).

The **file system** provides the mechanism for on-line storage of and access to both data and programs residing on the disks. A file is a collection of related information defined by its creator. The files are mapped by the **operating system** onto physical devices. Files are normally organized into directories for ease of use.

The devices that attach to a computer vary in many aspects. Some devices transfer a character or a block of characters at a time. Some can be accessed only sequentially, others randomly. Some transfer data synchronously, others asynchronously.

Some are dedicated, some shared. They can be read-only or read–write. They vary greatly in speed. In many ways, they are also the slowest major component of the computer.

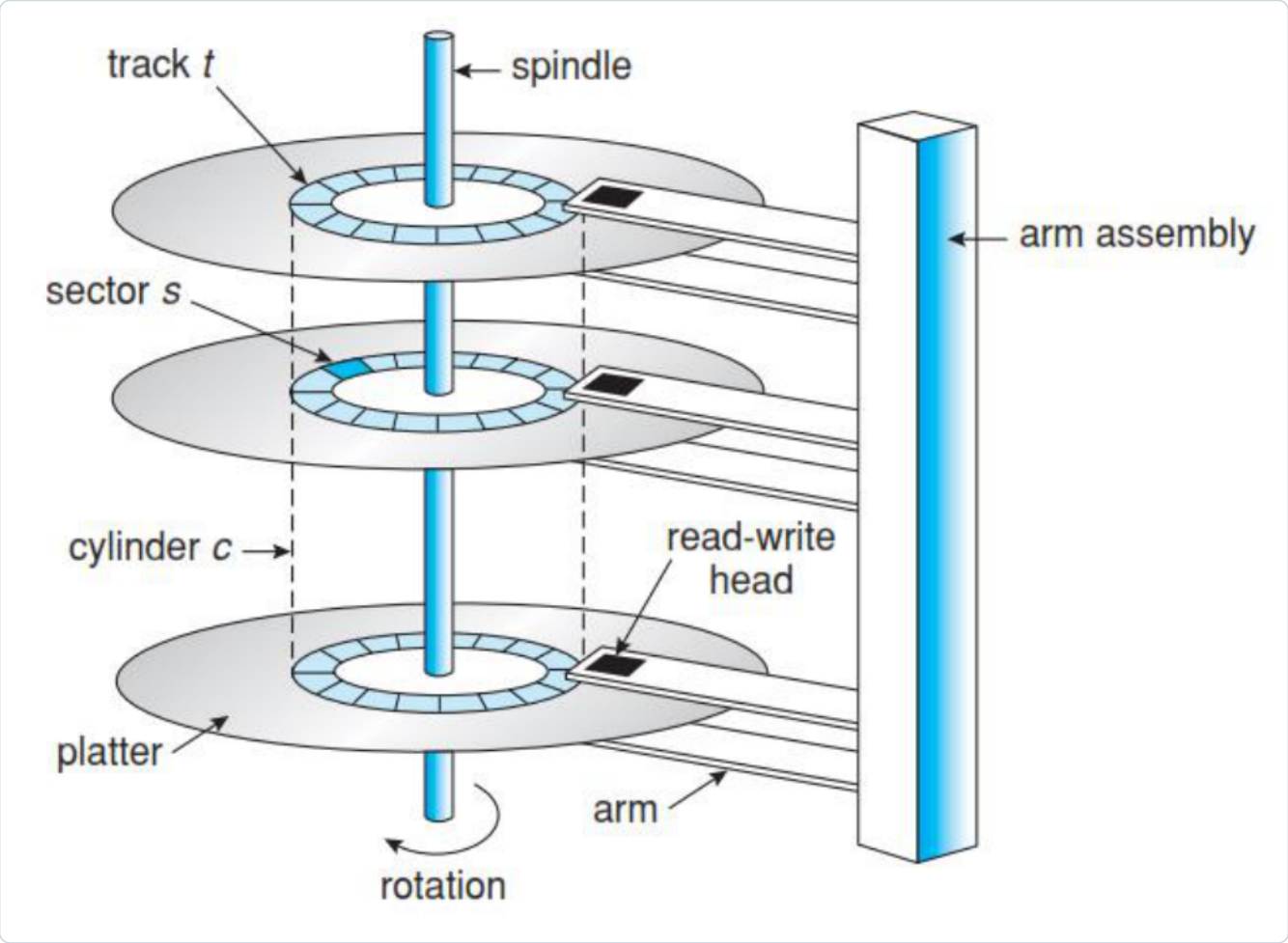
Because of all this device variation, the **operating system** needs to provide a wide range of functionality to applications, to allow them to control all aspects of the devices.

One key goal of an **operating system**'s I/O subsystem is to provide the simplest interface possible to the rest of the system. Because devices are a performance bottleneck, another key is to optimize I/O for maximum concurrency.

## 6.1. Magnetic Disks

Magnetic disks provide the bulk of secondary storage for modern computer systems. Conceptually, disks are relatively simple (see following figure). Each disk platter has a flat circular shape, like a CD. Common platter diameters range from 1.8 to 3.5 inches.

The two surfaces of a platter are covered with a magnetic material. We store information by recording it magnetically on the platters.




A read–write head —flies‖ just above each surface of every platter. The heads are attached to a disk arm that moves all the heads as a unit. The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.

The set of tracks that are at one arm position makes up a cylinder. There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors. The storage capacity of common disk drives is measured in gigabytes.

When the disk is in use, a drive motor spins it at high speed. Disk speed has two parts. The transfer rate is the rate at which data flow between the drive and the computer.

The positioning time, or random-access time, consists of two parts: the time necessary to move the disk arm to the desired cylinder, called the seek time, and the time necessary for the desired sector to rotate to the disk head, called the rotational latency.

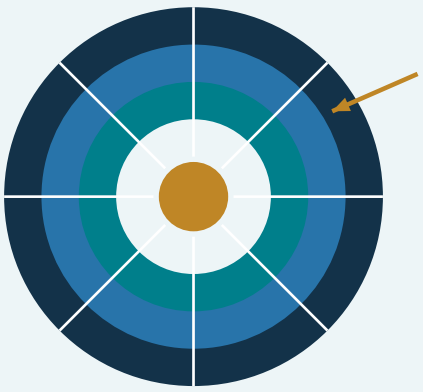
Typical disks can transfer several megabytes of data per second, and they have seek times and rotational latencies of several milliseconds.

Because the disk head flies on an extremely thin cushion of air (measured in microns), there is a danger that the head will make contact with the disk surface.

Although the disk platters are coated with a thin protective layer, the head will sometimes damage the magnetic surface. This accident is called a head crash. A head crash normally cannot be repaired; the entire disk must be replaced.

EXPLORE THE CONCEPT

## A magnetic disk has tracks and sectors



Tracks: concentric rings

Sectors: track subdivisions

Head: accesses a location

Sketch is schematic.

This schematic shows the organization used to describe disk storage.

# 6.1.1. Disk Structure

Modern magnetic disk drives are addressed as large one- dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer.

The size of a logical block is usually 512 bytes, although some disks can be low- level formatted to have a different logical block size, such as 1,024 bytes. The one-dimensional array of logical blocks is mapped onto the sectors of the disk sequentially.

Sector 0 is the first sector of the first track on the outermost cylinder. The mapping proceeds in order through that track, then through the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.

By using this mapping, we can—at least in theory— convert a logical block number into an old-style disk address that consists of a cylinder number, a track number within that cylinder, and a sector number within that track.

In practice, it is difficult to perform this translation, for two reasons. First, most disks have some defective sectors, but the mapping hides this by substituting spare sectors from elsewhere on the disk. Second, the number of sectors per track is not a constant on some drives.

Let’s look more closely at the second reason. On media that use constant linear velocity (CLV), the density of bits per track is uniform. The farther a track is from the center of the disk, the greater its length, so the more sectors it can hold.

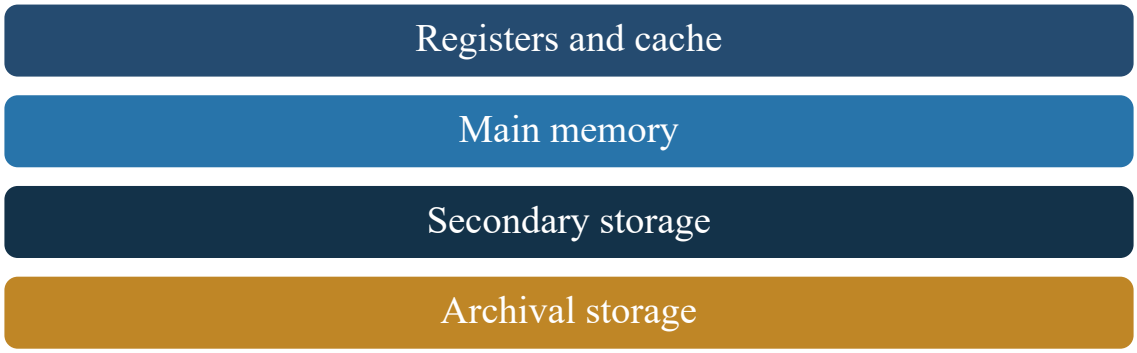
As we move from outer zones to inner zones, the number of sectors per track decreases. Tracks in the outermost zone typically hold 40 percent more sectors than do tracks in the innermost zone. The drive increases its rotation speed as the head moves from the outer to the inner tracks to keep the same rate of data moving under the head.

Alternatively, the disk rotation speed can stay constant; in this case, the density of bits decreases from inner tracks to outer tracks to keep the data rate constant. This method is used in hard disks and is known as constant angular velocity (CAV).

The number of sectors per track has been increasing as disk technology improves, and the outer zone of a disk usually has several hundred sectors per track. Similarly, the number of cylinders per disk has been increasing; large disks have tens of thousands of cylinders.

VISUAL EXPLANATION

**Storage hierarchy**




Conceptual hierarchy: access speed, cost, capacity, and volatility guide storage choices.

# 6.2. Disk Scheduling

One of the responsibilities of the **operating system** is to use the hardware efficiently. For the disk drives, meeting this responsibility entails having fast access time and large disk bandwidth.

The disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer. We can improve both the access time and the bandwidth by managing the order in which disk I/O requests are serviced.

Whenever a process needs I/O to or from the disk, it issues a system call to the **operating system**. The request specifies several pieces of information:

- Whether this operation is input or output
- What the disk address for the transfer is
- What the memory address for the transfer is
- What the number of sectors to be transferred is

For a multiprogramming system with many processes, the disk queue may often have several pending requests. Thus, when one request is completed, the **operating system** chooses which pending request to service next. How does the **operating system** make this choice?

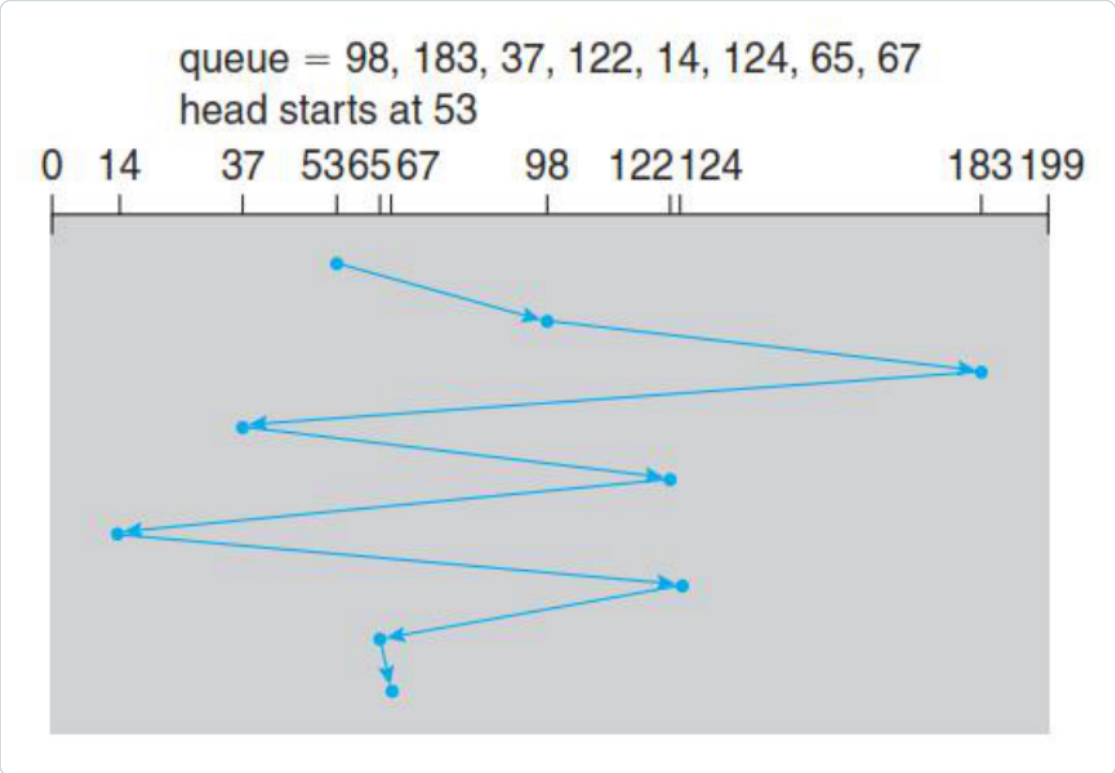
Any one of several disk-scheduling algorithms can be used, and we discuss them next.

# 6.2.1. FCFS Scheduling

The simplest form of disk scheduling is, of course, the first-come, first-served (FCFS) algorithm. This algorithm is intrinsically fair, but it generally does not provide the fastest service. Consider, for example, a disk queue with requests for I/O to blocks on cylinders

98, 183, 37, 122, 14, 124, 65, 67,

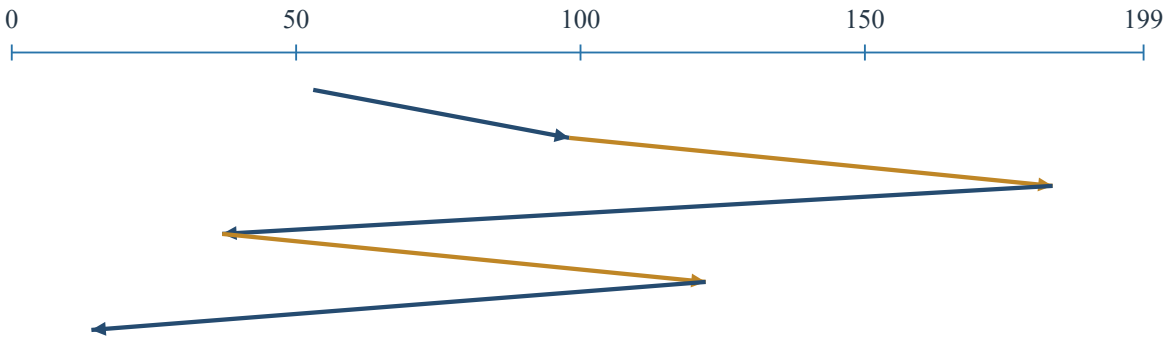
in that order. If the disk head is initially at cylinder 53, it will first move from 53 to 98, then to 183, 37, 122, 14, 124, 65, and finally to 67, for a total head movement of 640 cylinders. This schedule is diagrammed in the following figure.




The wild swing from 122 to 14 and then back to 124 illustrates the problem with this schedule. If the requests for cylinders 37 and 14 could be serviced together, before or after the requests for 122 and 124, the total head movement could be decreased substantially, and performance could be thereby improved.

VISUAL EXPLANATION

## FCFS disk requests




Illustrative head path in request order; the positions here are a separate visual example.

EXPLORE THE CONCEPT

## Disk requests specify locations

| | |
|---|---|
| Current cylinder | 53 |
| Next request | 98 |
| Movement | 45 cylinders |

Head movement is measured between successive cylinder positions.

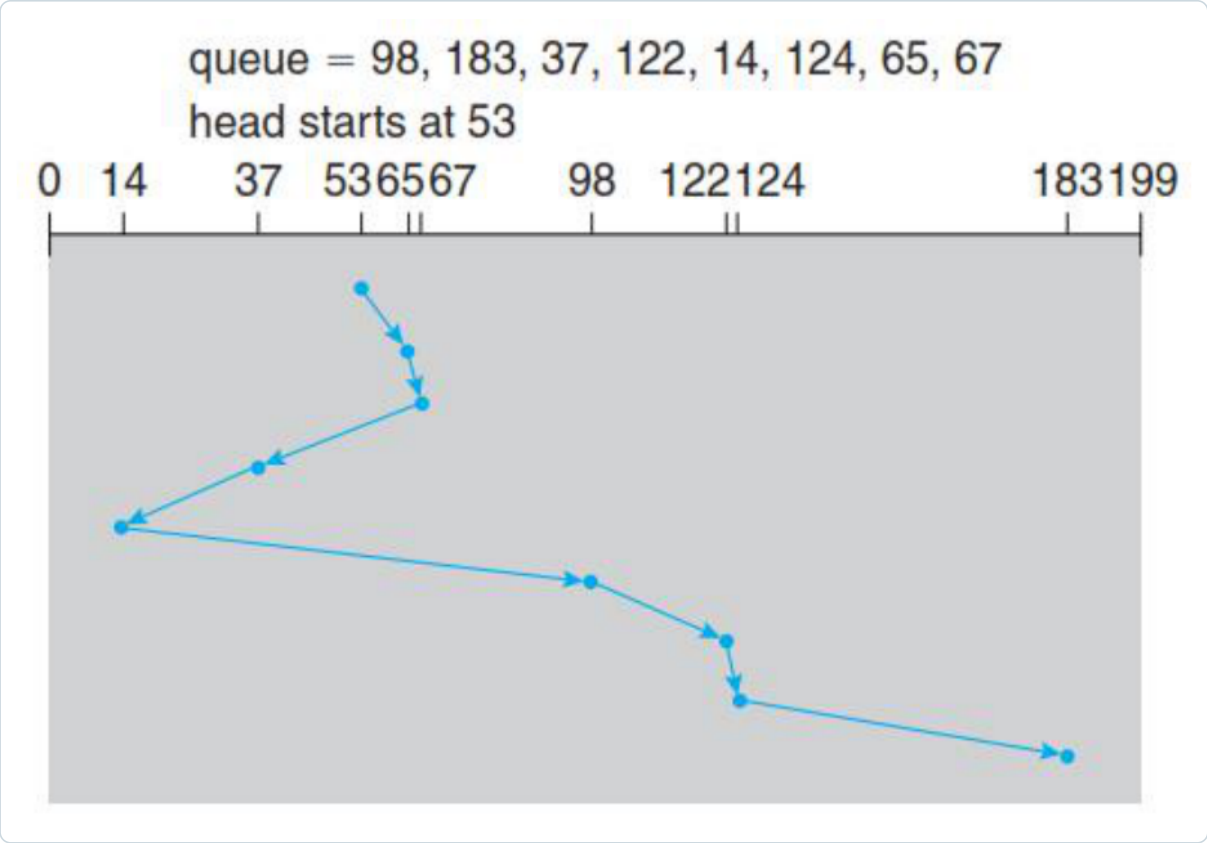
# 6.2.2. SSTF Scheduling

It seems reasonable to service all the requests close to the current head position before moving the head far away to service other requests. This assumption is the basis for the shortest-seek-time-first (SSTF) algorithm.

The SSTF algorithm selects the request with the least seek time from the current head position. In other words, SSTF chooses the pending request closest to the current head position.

For our example request queue, the closest request to the initial head position (53) is at cylinder 65. Once we are at cylinder 65, the next closest request is at cylinder 67. From there, the request at cylinder 37 is closer than the one at 98, so 37 is served next.

Continuing, we service the request at cylinder 14, then 98, 122, 124, and finally 183. This scheduling method results in a total head movement of only 236 cylinders—little more than one-third of the distance needed for FCFS scheduling of this request queue. Clearly, this algorithm gives a substantial improvement in performance.




Although the SSTF algorithm is a substantial improvement over the FCFS algorithm, it is not optimal. In the example, we can do better

by moving the head from 53 to 37, even though the latter is not closest, and then to 14, before turning around to service 65, 67, 98, 122, 124, and

183. This strategy reduces the total head movement to 208 cylinders.

## 6.2.3. SCAN Scheduling

In the SCAN algorithm, the disk arm starts at one end of the disk and moves toward the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk.

At the other end, the direction of head movement is reversed, and servicing continues. The head continuously scans back and forth across the disk.

The SCAN algorithm is sometimes called the elevator algorithm, since the disk arm behaves just like an elevator in a building, first servicing all the requests going up and then reversing to service requests the other way.

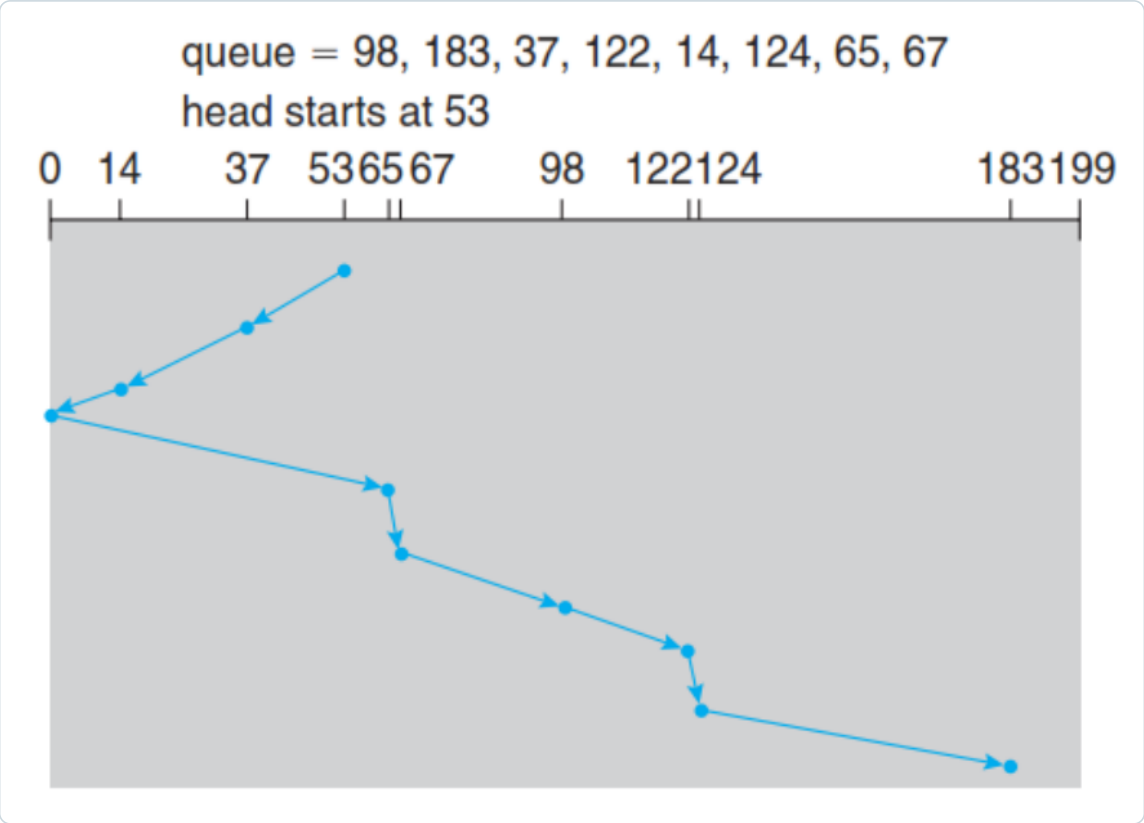
Let’s return to our example to illustrate. Before applying SCAN to schedule the requests on cylinders 98, 183, 37, 122, 14, 124, 65, and 67, we need to know the direction of head movement in addition to the head’s current position.

Assuming that the disk arm is moving toward 0 and that the initial head position is again 53, the head will next service 37 and then 14.

At cylinder 0, the arm will reverse and will move toward the other end of the disk, servicing the requests at 65, 67, 98, 122, 124, and

183. If a request arrives in the queue just in front of the head, it will be serviced almost immediately;

a request arriving just behind the head will have to wait until the arm moves to the end of the disk, reverses direction, and comes back.

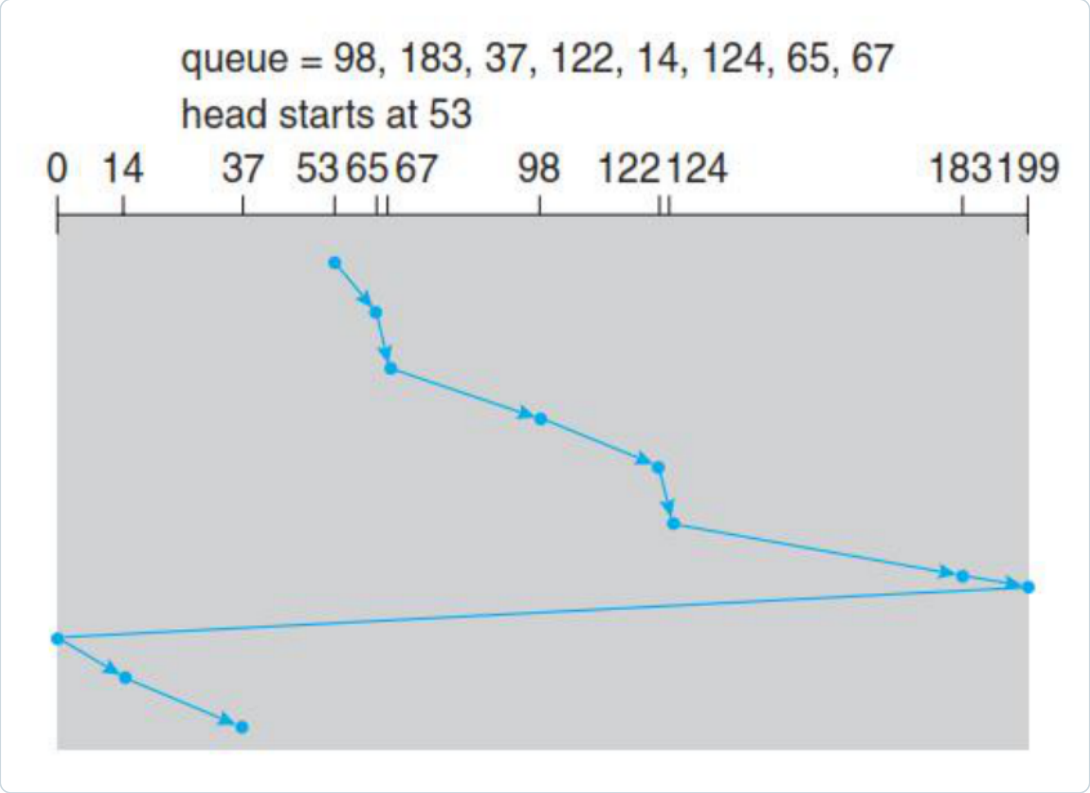



## 6.2.4. C-SCAN Scheduling

Circular SCAN (C-SCAN) scheduling is a variant of SCAN designed to provide a more uniform wait time. Like SCAN, C-SCAN moves the head from one end of the disk to the other, servicing requests along the way.

When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip (see the following figure). The C-SCAN scheduling

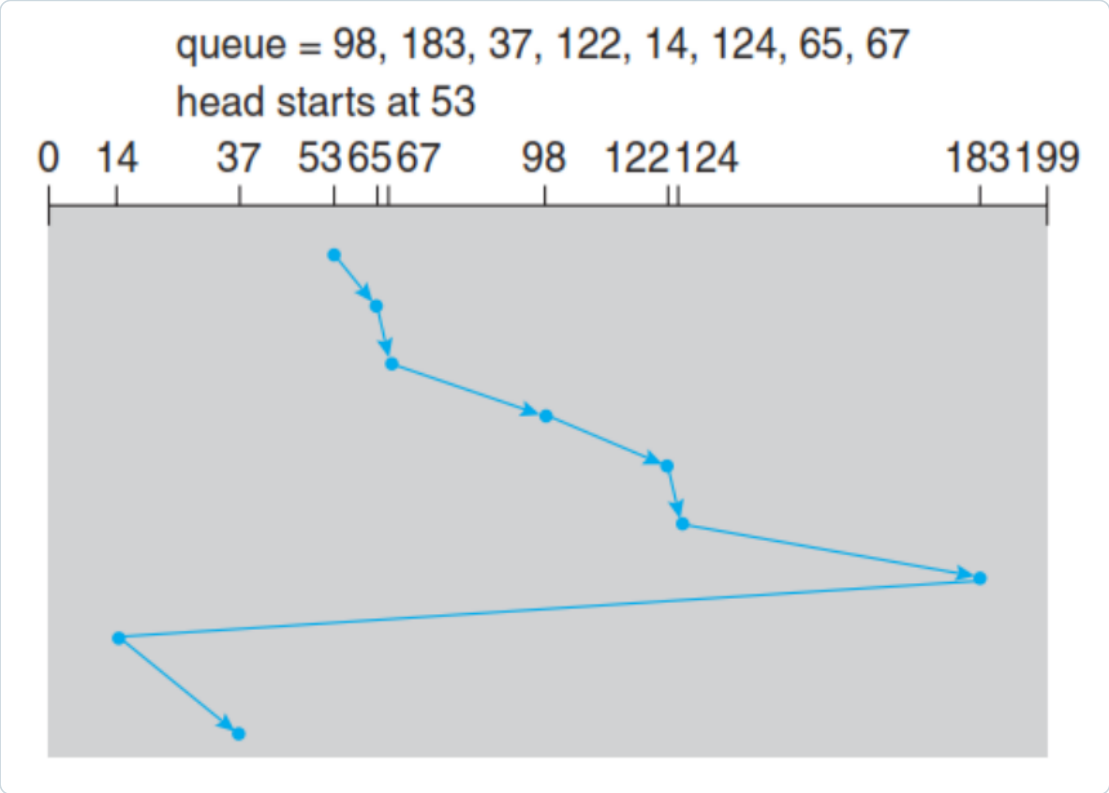
algorithm essentially treats the cylinders as a circular list that wraps around from the final cylinder to the first one.




## 6.2.5. LOOK Scheduling

As we described them, both SCAN and C-SCAN move the disk arm across the full width of the disk. In practice, neither algorithm is often implemented this way. More commonly, the arm goes only as far as the final request in each direction.

Then, it reverses direction immediately, without going all the way to the end of the disk. Versions of SCAN and C-SCAN that follow this pattern are called LOOK and C-LOOK scheduling, because they look for a request before continuing to move in a given direction (see the following figure).




VISUAL EXPLANATION

### SCAN and LOOK

SCAN
sweep to a disk end

LOOK
reverse at last request

C-SCAN
service one direction

Return movement
then begin next sweep

SCAN and LOOK differ in where the head reverses. The diagrams below show the full examples.

## 6.2.6. Selection of a Disk-Scheduling Algorithm

Given so many disk-scheduling algorithms, how do we choose the best one? SSTF is common and has a natural appeal because it increases performance over FCFS.

SCAN and C-SCAN perform better for systems that place a heavy load on the disk, because they are less likely to cause a starvation problem.

For any particular list of requests, we can define an optimal order of retrieval, but the computation needed to find an optimal schedule may not justify the savings over SSTF or SCAN. With any scheduling algorithm, however, performance depends heavily on the number and types of requests.

Optimal disk-scheduling can be replaced with a different algorithm if necessary. Because of the complexities, the disk-scheduling algorithm should be written as a separate module of the **operating system**. Either SSTF or LOOK is a reasonable choice for the default algorithm.

It is difficult for the **operating system** to schedule for improved rotational latency, though, because modern disks do not disclose the physical location of logical blocks.

Disk manufacturers have been alleviating this problem by implementing disk-scheduling algorithms in the controller hardware built into the disk drive. If the **operating system** sends a batch of requests to the controller, the controller can queue them and then schedule them to improve both the seek time and the rotational latency.

# 6.3. File -System

The file system is the most visible aspect of an **operating system**. It provides the mechanism for on-line storage of and access to both data and programs of the **operating system** and all the users of the computer system.

The file system consists of two distinct parts: a collection of files, each storing related data, and a directory structure, which organizes and provides information about all the files in the system.

## 6.3.1. File Concept

A file is a named collection of related information that is recorded on secondary storage. From a user's perspective, a file is the smallest allotment of logical secondary storage; that is, data cannot be written to secondary storage unless they are within a file.

Commonly, files represent programs (both source and object forms) and data. Data files may be numeric, alphabetic, alphanumeric, or binary. Files may be free form, such as text files, or may be formatted rigidly.

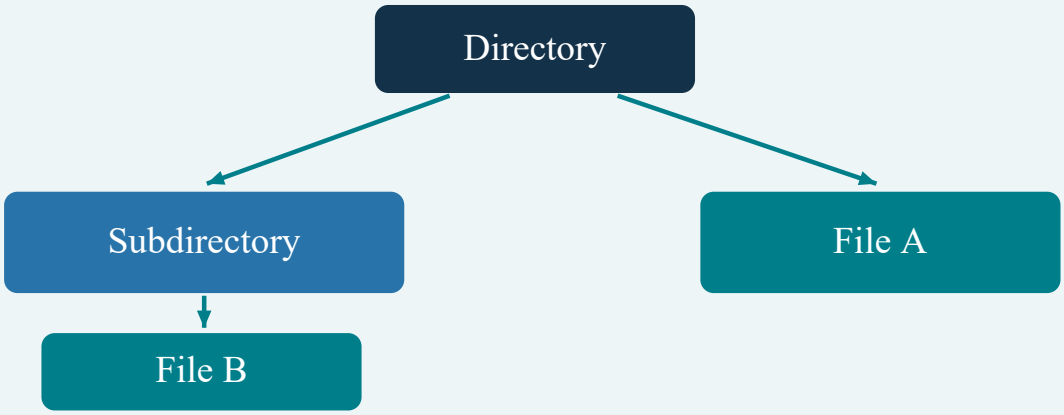
In general, a file is a sequence of bits, bytes, lines, or records, the meaning of which is defined by the file's creator and user. The concept of a file is thus extremely general.

The information in a file is defined by its creator. Many different types of information may be stored in a file — source or executable programs, numeric or text data, photos, music, video, and so on. A file has a certain defined structure, which depends on its type.

A text file is a sequence of characters organized into lines (and possibly pages). A source file is a sequence of functions, each of which is further organized as declarations followed by executable statements. An executable file is a series of code sections that the loader can bring into memory and execute.

EXPLORE THE CONCEPT

### Directories organize files




Directories provide a logical organization for files and subdirectories.

## 6.3.2. File Attributes

A file's attributes vary from one **operating system** to another but typically consist of these:

- Name. The symbolic file name is the only information kept in human readable form.
- Identifier. This unique tag, usually a number, identifies the file within the file system; it is the non-human- readable name for the file.

- Type. This information is needed for systems that support different typesof files.
- Location. This information is a pointer to a device and to the location of the file on that device.
- Size. The current size of the file (in bytes, words, or blocks) and possibly the maximum allowed size are included in this attribute.
- Protection. Access-control information determines who can do reading, writing, executing, and so on.
- Time, date, and user identification. This information may be kept for creation, last modification, and last use. These data can be useful for protection, security, and usage monitoring.

## 6.3.3. File Operations

A file is an abstract data type. To define a file properly, we need to consider the operations that can be performed on files. The **operating system** can provide system calls to create, write, read, reposition, delete, and truncate files. Let's examine what the **operating system** must do to perform each of these six basic file operations:

- **Creating** a file. Two steps are necessary to create a file. First, space in the file system must be found for the file. Second, an entry for the new file must be made in the directory.
- **Writing** a file. To write a file, we make a system call specifying both the name of the file and the information to be written to the file. Given the name of the file, the system searches the directory to find the file's location.

The system must keep a write pointer to the location in the file where the next write is to take place. The write pointer must be updated whenever a write occurs.

- **Reading** a file. To read from a file, we use a system call that specifies the name of the file and where (in memory) the next block of the file should be put.

  Again, the directory is searched for the associated entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place. Once the read has taken place, the read pointer is updated.

  Because a process is usually either reading from or writing to a file, the current operation location can be kept as a per-process currentfile-position pointer. Both the read and write operations use this same pointer, saving space and reducing system complexity.

- **Repositioning** within a file. The directory is searched for the appropriate entry, and the current-file-position pointer is repositioned to a given value. Repositioning within a file need not involve any actual I/O. This file operation is also known as a file seek.

- **Deleting** a file. To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.

- **Truncating** a file. The user may want to erase the contents of a file but keep its attributes. Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain

unchanged—except for file length—but lets the file be reset to length zero and its file space released.

Most of the file operations mentioned involve searching the directory for The **operating system** keeps a table, called the open-file table, containing information about all open files.

When a file operation is requested, the file is specified via an index into this table, so no searching is required. When the file is no longer being actively used, it is closed by the process, and the **operating system** removes its entry from the open-file table.

The entry associated with the named file. Several pieces of information are associated with an open file:

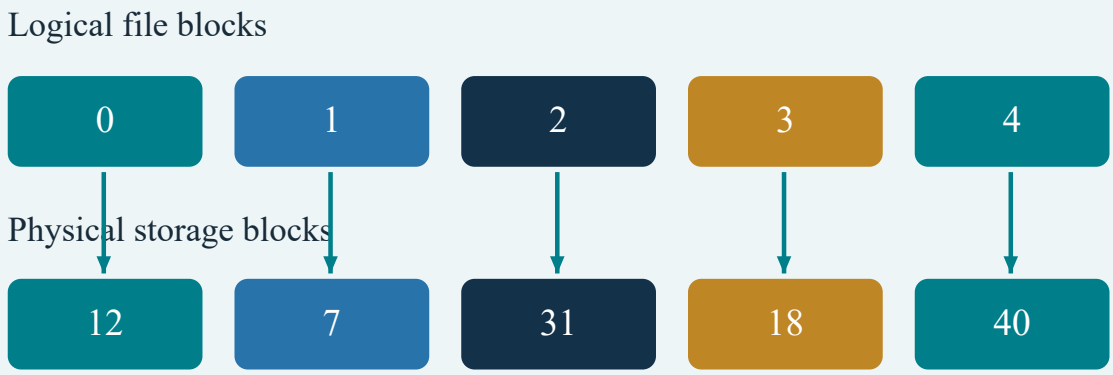
- File pointer. On systems that do not include a file offset as part of the read() and write() system calls, the system must track the last read–write location as a current-file-position pointer. This pointer is unique to each process operating on the file and therefore must be kept separate from the on-disk file attributes.
- File-open count. As files are closed, the **operating system** must reuse its open-file table entries, or it could run out of space in the table.

  Multiple processes may have opened a file, and the system must wait for the last file to close before removing the open-file table entry. The file-open count tracks the number of opens and closes and reaches zero on the last close. The system can then remove the entry.

- Disk location of the file. Most file operations require the system to modify data within the file. The information needed to locate the file on disk is kept in memory so that the system does not have to read it from disk for each operation.
- Access rights. Each process opens a file in an access mode. This information is stored on the per-process table so the **operating system** can allow or deny subsequent I/O requests.

EXPLORE THE CONCEPT

### File blocks map to storage blocks




Illustrative mapping: a logical file need not occupy consecutive physical blocks.

## 6.3.4. File Type

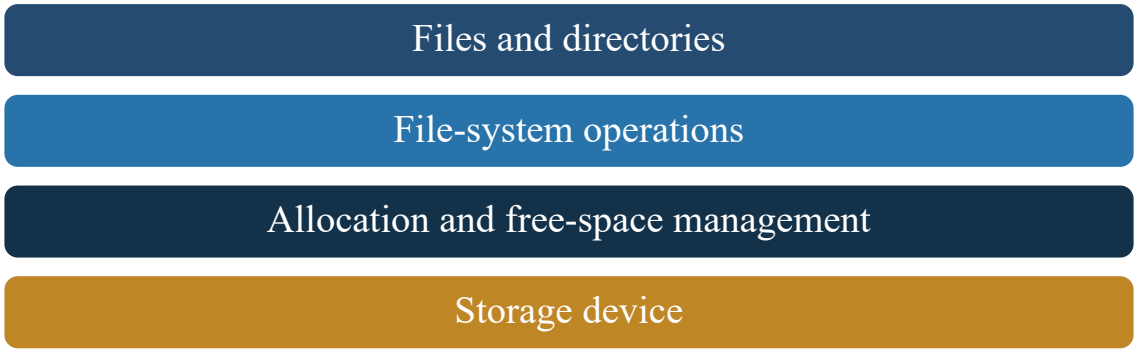
File type refers to the ability of the **operating system** to distinguish different types of file such as text files source files and binary files etc. Many operating systems support many types of files.

**Operating system** like MS-DOS and UNIX has the following types of files:

- Ordinary files o These are the files that contain user information. o These may have text, databases or executable program. o The user can apply various operations on such files like add, modify, delete or even remove the entire file.
- Directory files o These files contain list of file names and other information related to these files.
- Special files: o These files are also known as device files. o These files represent physical device like disks, terminals, printers, networks, tape drive etc.

VISUAL EXPLANATION

**The file-system view**




The file system organizes persistent data and maps logical file operations to storage.

# References:

—**Operating system** concepts‖, Abraham Silberschatz, Peter Bear, and Greg Gagen, Ten Edition.

—**Operating system** tutorials‖, tutorialspoint.com